

Lab 1:Strengthening System Security

Tools Used

- Kali Linux
- PAM / libpam-pwquality
- auditd / auditctl / ausearch
- sudo / systemctl
- chage / passwd
- grep / awk / find / getent

Exercise 1: Locate and Open the adduser.conf File

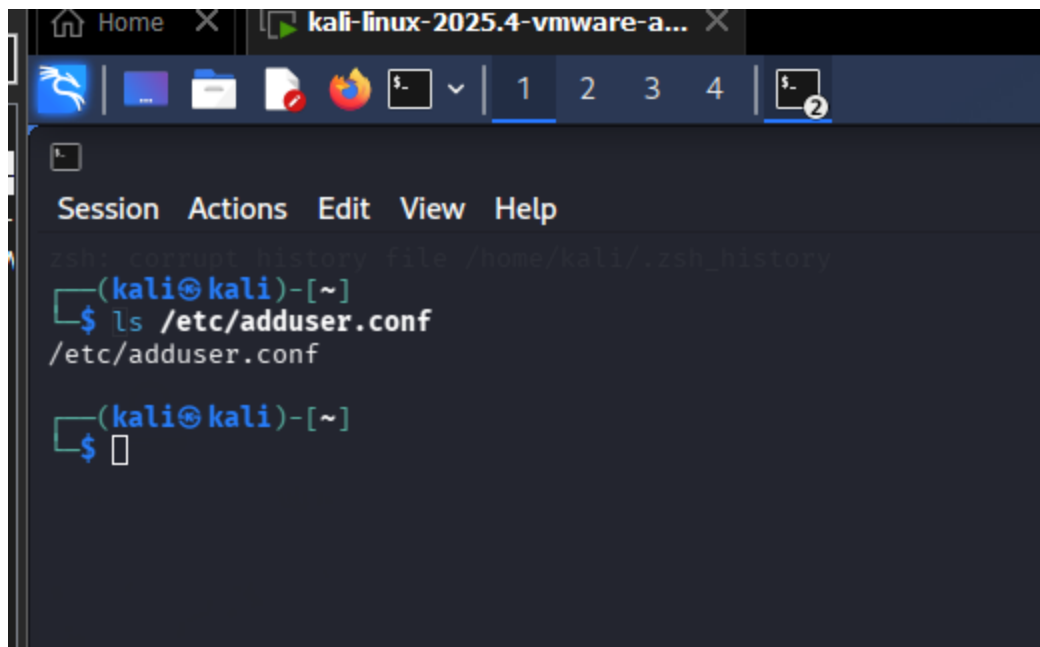
Command Used:

ls /etc/adduser.conf

Opened File Editor

nano

sudo nano /etc/adduser.conf



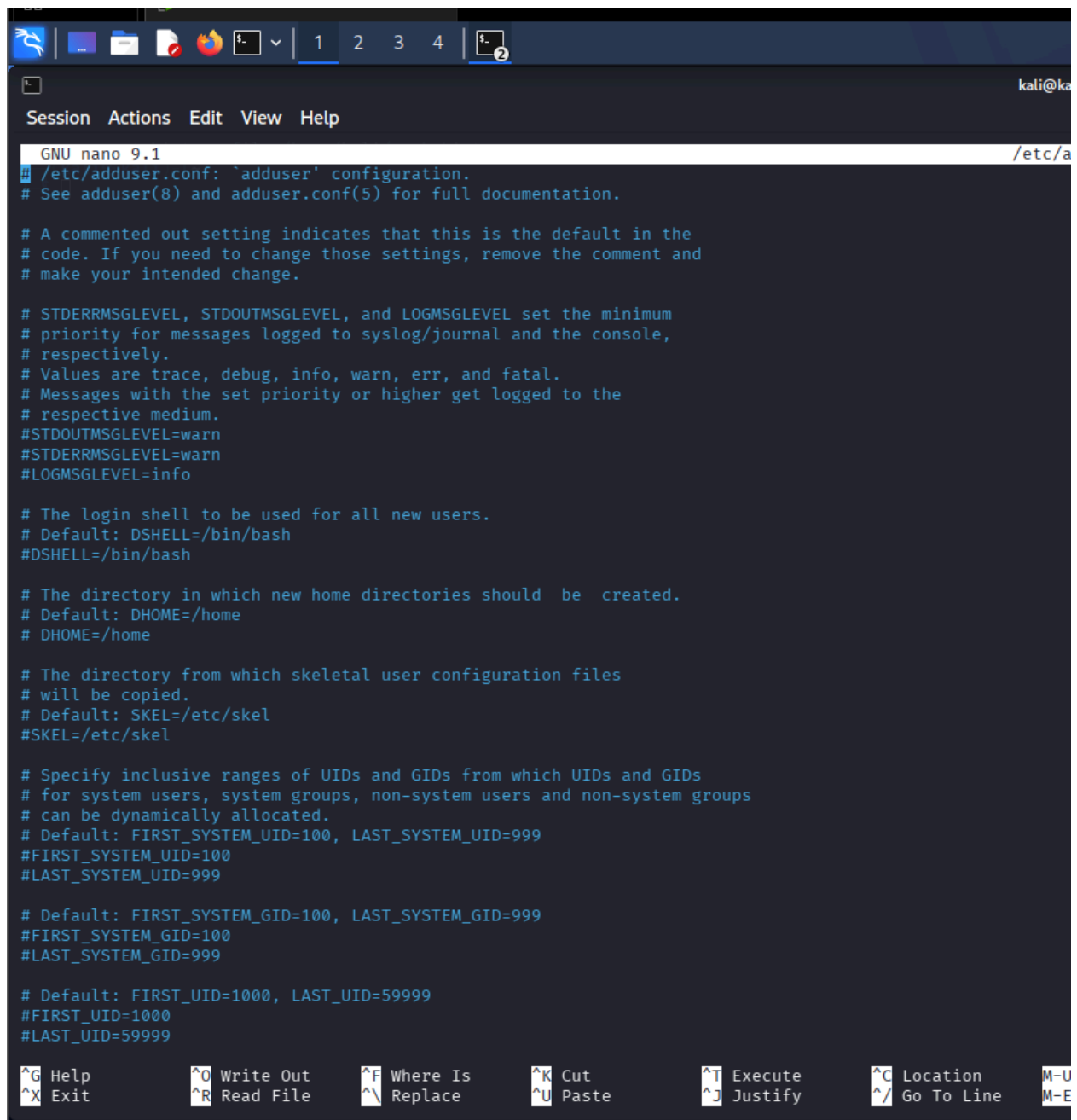
A terminal window titled "kali-linux-2025.4-vmware-a..." with a dark theme. The window has a menu bar with "Session", "Actions", "Edit", "View", and "Help". The terminal shows the following commands and output:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ ls /etc/adduser.conf
/etc/adduser.conf
(kali@kali)~$
```



A terminal window showing the following commands and output:

```
(kali@kali)~$ sudo nano /etc/adduser.conf
[sudo] password for kali:
(kali@kali)~$
```



```
GNU nano 9.1 /etc/a
# /etc/adduser.conf: 'adduser' configuration.
# See adduser(8) and adduser.conf(5) for full documentation.

# A commented out setting indicates that this is the default in the
# code. If you need to change those settings, remove the comment and
# make your intended change.

# STDERRMSGLEVEL, STDOUTMSGLEVEL, and LOGMSGLEVEL set the minimum
# priority for messages logged to syslog/journal and the console,
# respectively.
# Values are trace, debug, info, warn, err, and fatal.
# Messages with the set priority or higher get logged to the
# respective medium.
#STDOUTMSGLEVEL=warn
#STDERRMSGLEVEL=warn
#LOGMSGLEVEL=info

# The login shell to be used for all new users.
# Default: DSHELL=/bin/bash
#DSHELL=/bin/bash

# The directory in which new home directories should be created.
# Default: DHOME=/home
# DHOME=/home

# The directory from which skeletal user configuration files
# will be copied.
# Default: SKEL=/etc/skel
#SKEL=/etc/skel

# Specify inclusive ranges of UIDs and GIDs from which UIDs and GIDs
# for system users, system groups, non-system users and non-system groups
# can be dynamically allocated.
# Default: FIRST_SYSTEM_UID=100, LAST_SYSTEM_UID=999
#FIRST_SYSTEM_UID=100
#LAST_SYSTEM_UID=999

# Default: FIRST_SYSTEM_GID=100, LAST_SYSTEM_GID=999
#FIRST_SYSTEM_GID=100
#LAST_SYSTEM_GID=999

# Default: FIRST_UID=1000, LAST_UID=59999
#FIRST_UID=1000
#LAST_UID=59999

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E
```

The adduser.conf file was successfully located and opened for review. This provided an overview of the default configuration settings used for creating new user accounts in Linux.

Exercise 2: Analyze Key Configuration Parameters

Commands Used to search:

1. Default Group Settings:
GROUP=100.
2. Home Directory Settings:
DHOME=/home

Skel Directory:

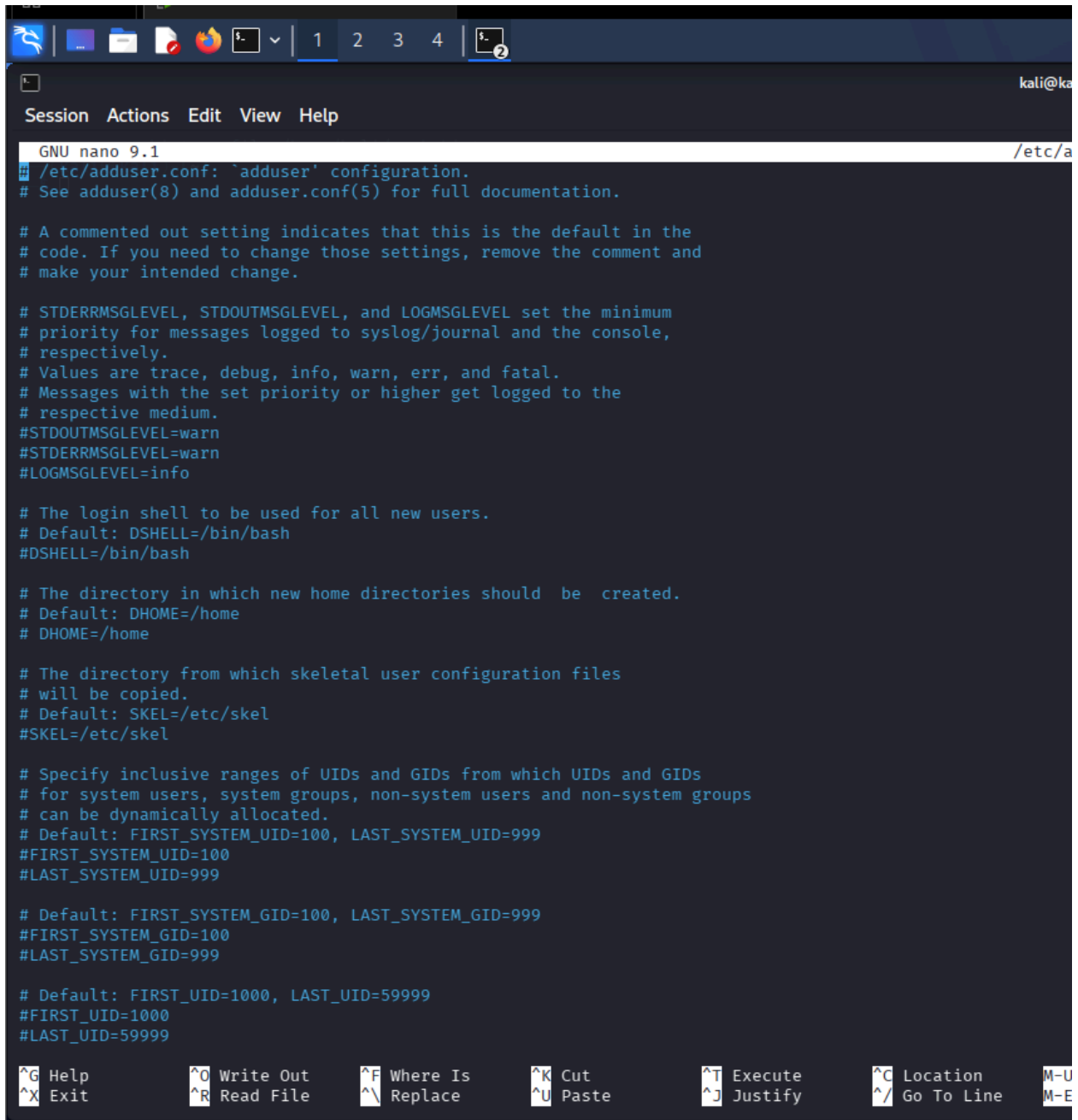
- Locate

SKEL=/etc/skel

4. Default Shell:
DSHELL=/bin/

5. Useful Fields in adduser.conf:

- FIRST_UID
- LAST_UID
- FIRST_GID
- LAST_GID
- USERGROUPS
- HOME
- LOGIN_SHELL



```
GNU nano 9.1 /etc/a
# /etc/adduser.conf: 'adduser' configuration.
# See adduser(8) and adduser.conf(5) for full documentation.

# A commented out setting indicates that this is the default in the
# code. If you need to change those settings, remove the comment and
# make your intended change.

# STDERRMSGLEVEL, STDOUTMSGLEVEL, and LOGMSGLEVEL set the minimum
# priority for messages logged to syslog/journal and the console,
# respectively.
# Values are trace, debug, info, warn, err, and fatal.
# Messages with the set priority or higher get logged to the
# respective medium.
#STDOUTMSGLEVEL=warn
#STDERRMSGLEVEL=warn
#LOGMSGLEVEL=info

# The login shell to be used for all new users.
# Default: DSHELL=/bin/bash
#DSHELL=/bin/bash

# The directory in which new home directories should be created.
# Default: DHOME=/home
# DHOME=/home

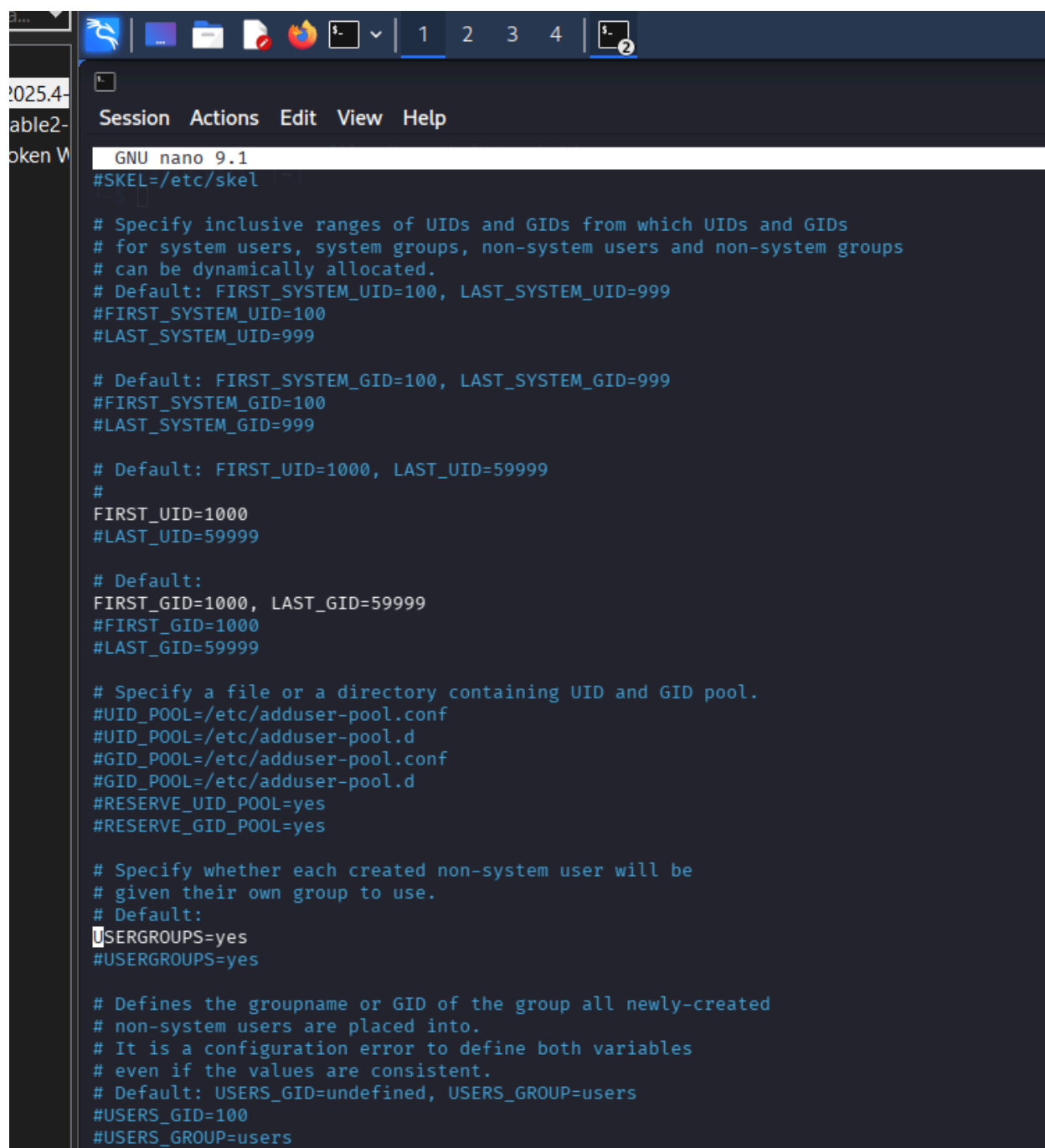
# The directory from which skeletal user configuration files
# will be copied.
# Default: SKEL=/etc/skel
#SKEL=/etc/skel

# Specify inclusive ranges of UIDs and GIDs from which UIDs and GIDs
# for system users, system groups, non-system users and non-system groups
# can be dynamically allocated.
# Default: FIRST_SYSTEM_UID=100, LAST_SYSTEM_UID=999
#FIRST_SYSTEM_UID=100
#LAST_SYSTEM_UID=999

# Default: FIRST_SYSTEM_GID=100, LAST_SYSTEM_GID=999
#FIRST_SYSTEM_GID=100
#LAST_SYSTEM_GID=999

# Default: FIRST_UID=1000, LAST_UID=59999
#FIRST_UID=1000
#LAST_UID=59999

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E
```



The image shows a terminal window with the nano text editor open. The editor displays the configuration file /etc/adduser.conf. The window's title bar includes standard Linux window controls and a taskbar with icons for a file manager, terminal, and other applications. The nano editor's menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The file content is as follows:

```
GNU nano 9.1
#SKEL=/etc/skel

# Specify inclusive ranges of UIDs and GIDs from which UIDs and GIDs
# for system users, system groups, non-system users and non-system groups
# can be dynamically allocated.
# Default: FIRST_SYSTEM_UID=100, LAST_SYSTEM_UID=999
#FIRST_SYSTEM_UID=100
#LAST_SYSTEM_UID=999

# Default: FIRST_SYSTEM_GID=100, LAST_SYSTEM_GID=999
#FIRST_SYSTEM_GID=100
#LAST_SYSTEM_GID=999

# Default: FIRST_UID=1000, LAST_UID=59999
#
#FIRST_UID=1000
#LAST_UID=59999

# Default:
#FIRST_GID=1000, LAST_GID=59999
#FIRST_GID=1000
#LAST_GID=59999

# Specify a file or a directory containing UID and GID pool.
#UID_POOL=/etc/adduser-pool.conf
#UID_POOL=/etc/adduser-pool.d
#GID_POOL=/etc/adduser-pool.conf
#GID_POOL=/etc/adduser-pool.d
#RESERVE_UID_POOL=yes
#RESERVE_GID_POOL=yes

# Specify whether each created non-system user will be
# given their own group to use.
# Default:
#USERGROUPS=yes
#USERGROUPS=yes

# Defines the groupname or GID of the group all newly-created
# non-system users are placed into.
# It is a configuration error to define both variables
# even if the values are consistent.
# Default: USERS_GID=undefined, USERS_GROUP=users
#USERS_GID=100
#USERS_GROUP=users
```

The configuration file was examined to identify key user management parameters. Some settings listed in the lab manual were not available in the current Kali Linux version, while others such as FIRST_UID, LAST_UID, FIRST_GID, LAST_GID, and USERGROUPS were successfully located.

Access Control Lists (ACLs)

Scenario

You are a system administrator managing a shared project folder for a development team. The project folder is shared among three team members: Alice, Bob, and Charlie.

- Alice: Needs full read, write, and execute permissions for the folder.
- Bob: Can only read files but should not modify or delete them.
- Charlie: Needs both read and write permissions but should not delete files.

configure the folder's permissions using Access Control Lists (ACLs) to meet these requirements.

Exercise 1: Create the Project Folder and Test Files

Step 1: Created the Users First because they don't exist

Commands Used:

```
sudo adduser alice  
sudo adduser bob  
sudo adduser charlie
```

Step 2: Created the Group

```
sudo groupadd developers
```

Step 3: Add Users to the Group

```
sudo usermod -aG developers alice
```

```
sudo usermod -aG developers bob
```

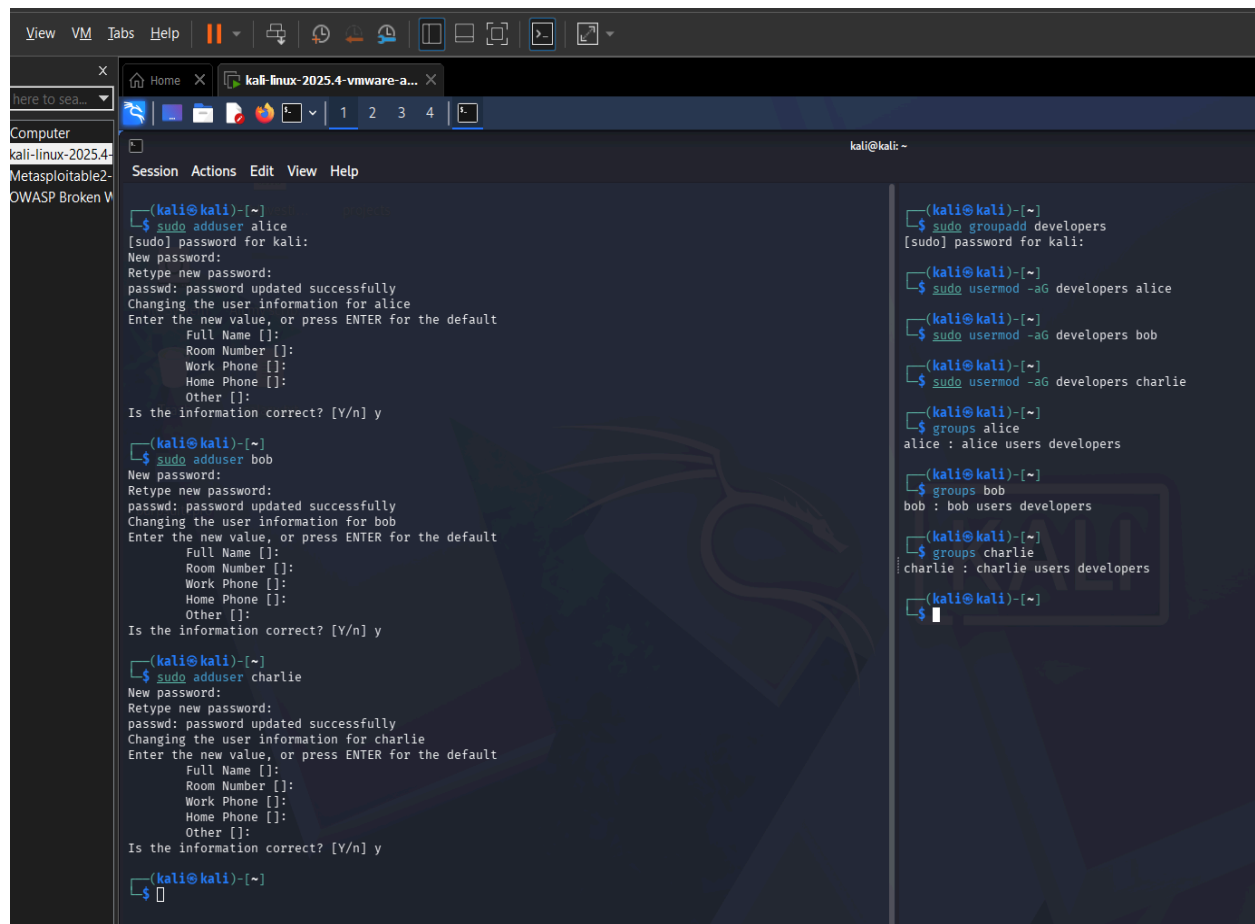
```
sudo usermod -aG developers charlie
```

Step 4: Verified if successfully created

```
groups alice
```

```
groups bob
```

```
groups charlie
```



The screenshot shows a Kali Linux terminal window with a dark theme. The terminal displays the following commands and their outputs:

```
(kali@kali)-[~]
$ sudo groupadd developers
[sudo] password for kali:
$ sudo usermod -aG developers alice
$ sudo usermod -aG developers bob
$ sudo usermod -aG developers charlie
$ groups alice
alice : alice users developers
$ groups bob
bob : bob users developers
$ groups charlie
charlie : charlie users developers
$
```

Successfully created users.Group and added users to the group.

Step 5: Create the project folder

Commands Used:

sudo mkdir /projects/team_project

To create test files in the folder:

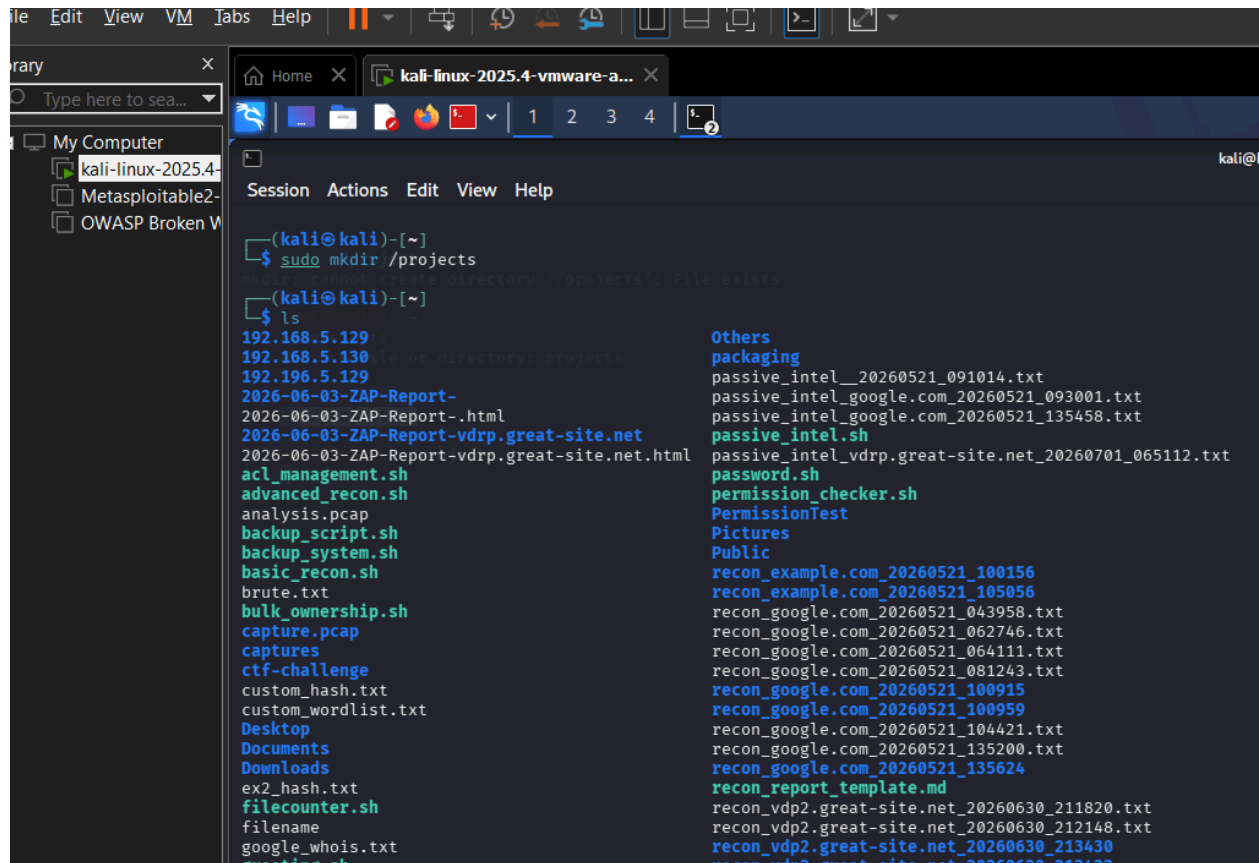
sudo touch /projects/team_project/{file1.txt,file2.txt}

Change ownership of the folder to a generic group (e.g.,
developers

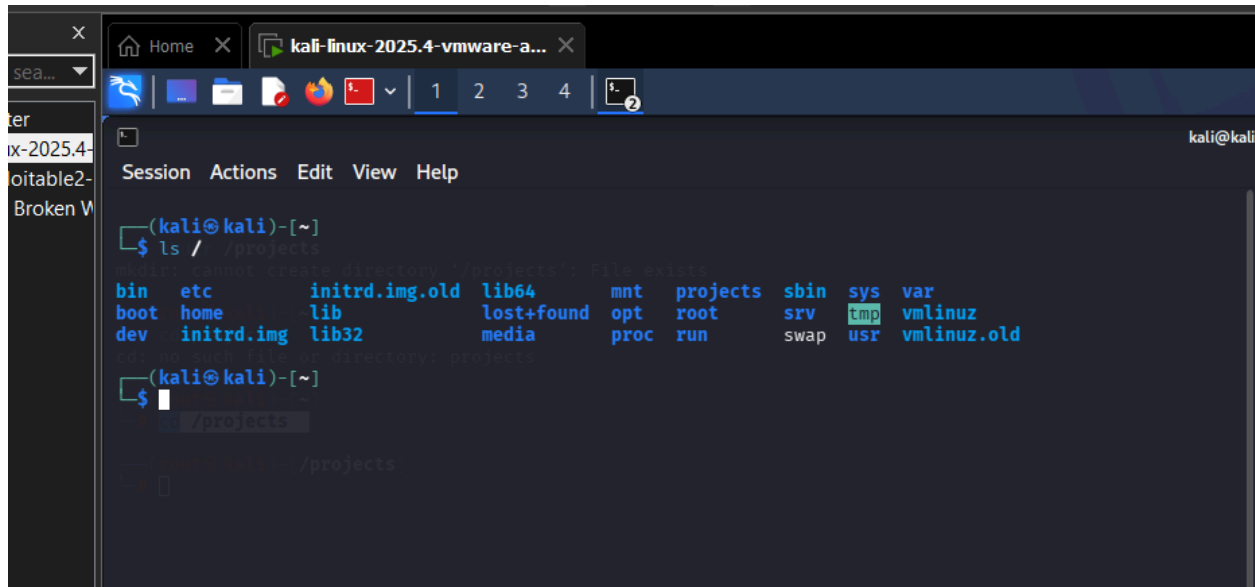
sudo chown -R root:developers /projects/team_project

Set default permissions to allow group access:

sudo chmod 770 /projects/team_project



```
kali@kali:~$ sudo mkdir /projects
(kali@kali)~$ ls
192.168.5.129
192.168.5.130
192.196.5.129
2026-06-03-ZAP-Report-
2026-06-03-ZAP-Report-.html
2026-06-03-ZAP-Report-vdrp.great-site.net
2026-06-03-ZAP-Report-vdrp.great-site.net.html
acl_management.sh
advanced_recon.sh
analysis.pcap
backup_script.sh
backup_system.sh
basic_recon.sh
brute.txt
bulk_ownership.sh
capture.pcap
captures
ctf-challenge
custom_hash.txt
custom_wordlist.txt
Desktop
Documents
Downloads
ex2_hash.txt
filecounter.sh
filename
google_whois.txt
greeting.sh
Others
packaging
passive_intel__20260521_091014.txt
passive_intel_google.com_20260521_093001.txt
passive_intel_google.com_20260521_135458.txt
passive_intel.sh
passive_intel_vdrp.great-site.net_20260701_065112.txt
password.sh
permission_checker.sh
PermissionTest
Pictures
Public
recon_example.com_20260521_100156
recon_example.com_20260521_105056
recon_google.com_20260521_043958.txt
recon_google.com_20260521_062746.txt
recon_google.com_20260521_064111.txt
recon_google.com_20260521_081243.txt
recon_google.com_20260521_100915
recon_google.com_20260521_100959
recon_google.com_20260521_104421.txt
recon_google.com_20260521_135200.txt
recon_google.com_20260521_135624
recon_report_template.md
recon_vdp2.great-site.net_20260630_211820.txt
recon_vdp2.great-site.net_20260630_212148.txt
recon_vdp2.great-site.net_20260630_213430
recon_vdp2.great-site.net_20260630_213633
```



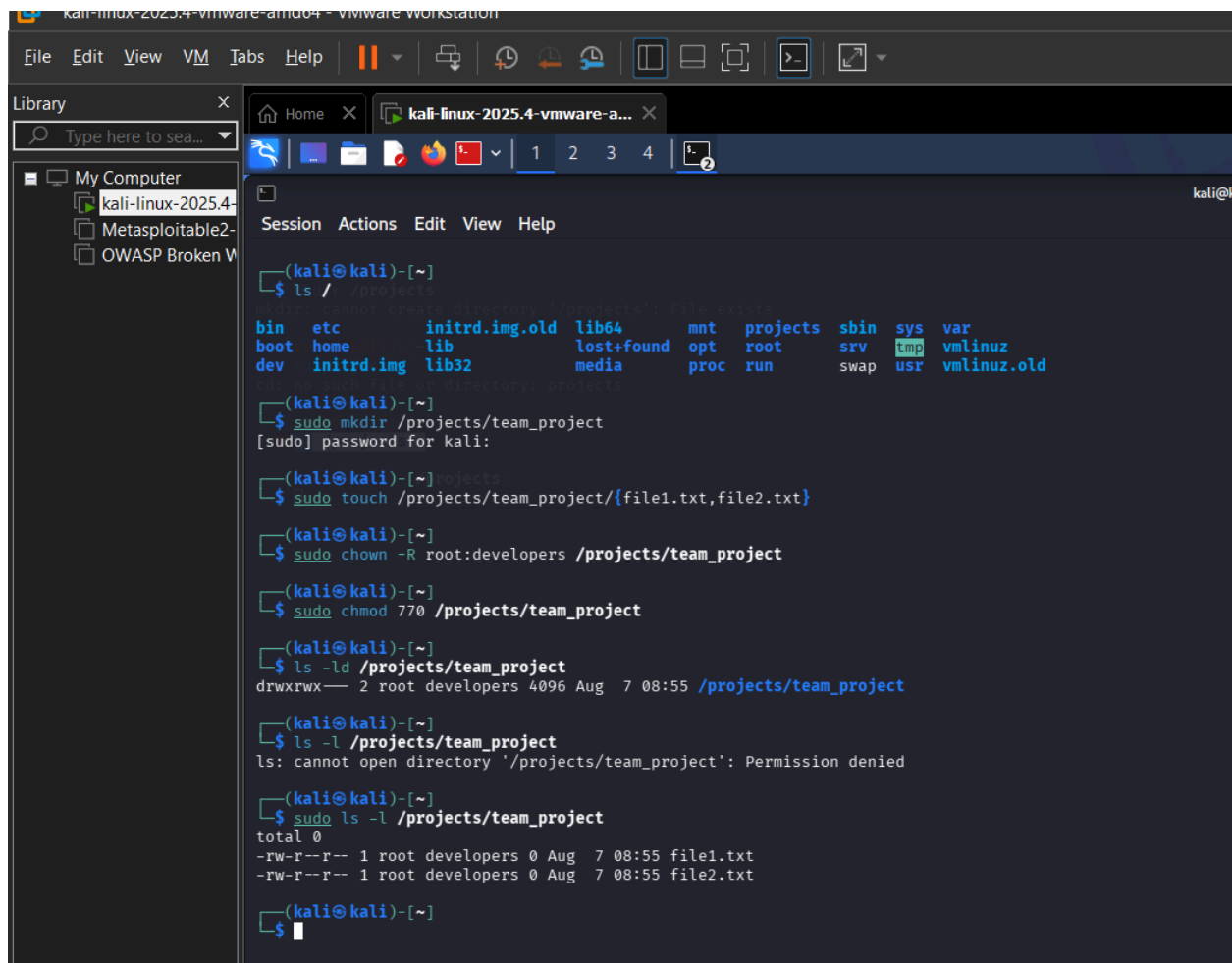
Successfully created the /projects directory using:

`sudo mkdir /projects`

this placed it in the root of the filesystem. Initially, I couldn't find it with the `ls` command because I was listing my current directory instead of the root directory. I then used

`ls /`

and confirmed that the projects directory had been created successfully.



```
(kali@kali)-[~]
$ ls /
bin      etc      initrd.img.old  lib64      mnt      projects  sbin  sys  var
boot    home     lib             lost+found  opt      root      srv   tmp  vmlinuz
dev      initrd.img  lib32          media      proc     run       swap  usr  vmlinuz.old

(kali@kali)-[~]
$ sudo mkdir /projects/team_project
[sudo] password for kali:

(kali@kali)-[~]
$ sudo touch /projects/team_project/{file1.txt,file2.txt}

(kali@kali)-[~]
$ sudo chown -R root:developers /projects/team_project

(kali@kali)-[~]
$ sudo chmod 770 /projects/team_project

(kali@kali)-[~]
$ ls -ld /projects/team_project
drwxrwx--- 2 root developers 4096 Aug  7 08:55 /projects/team_project

(kali@kali)-[~]
$ ls -l /projects/team_project
ls: cannot open directory '/projects/team_project': Permission denied

(kali@kali)-[~]
$ sudo ls -l /projects/team_project
total 0
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file1.txt
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file2.txt

(kali@kali)-[~]
$
```

The team_project directory and the test files were created successfully, The permissions and ownership were applied correctly, although I got a "Permission denied" message when trying to access the directory, I had to use (sudo) because administrative privileges were required, which confirmed that the directory, permissions, ownership, and files were all set up successfully.

Exercise 2: Configure ACLs for Each User

Commands Used:

1. Grant Alice Full Permissions (Read, Write, Execute):
`sudo setfacl -m u:alice:rwX /projects/team_project`
2. Grant Bob Read-Only Permissions:
`sudo setfacl -m u:bob:rx /projects/team_project`

3. Grant Charlie Read and Write Permissions Without Deletion:

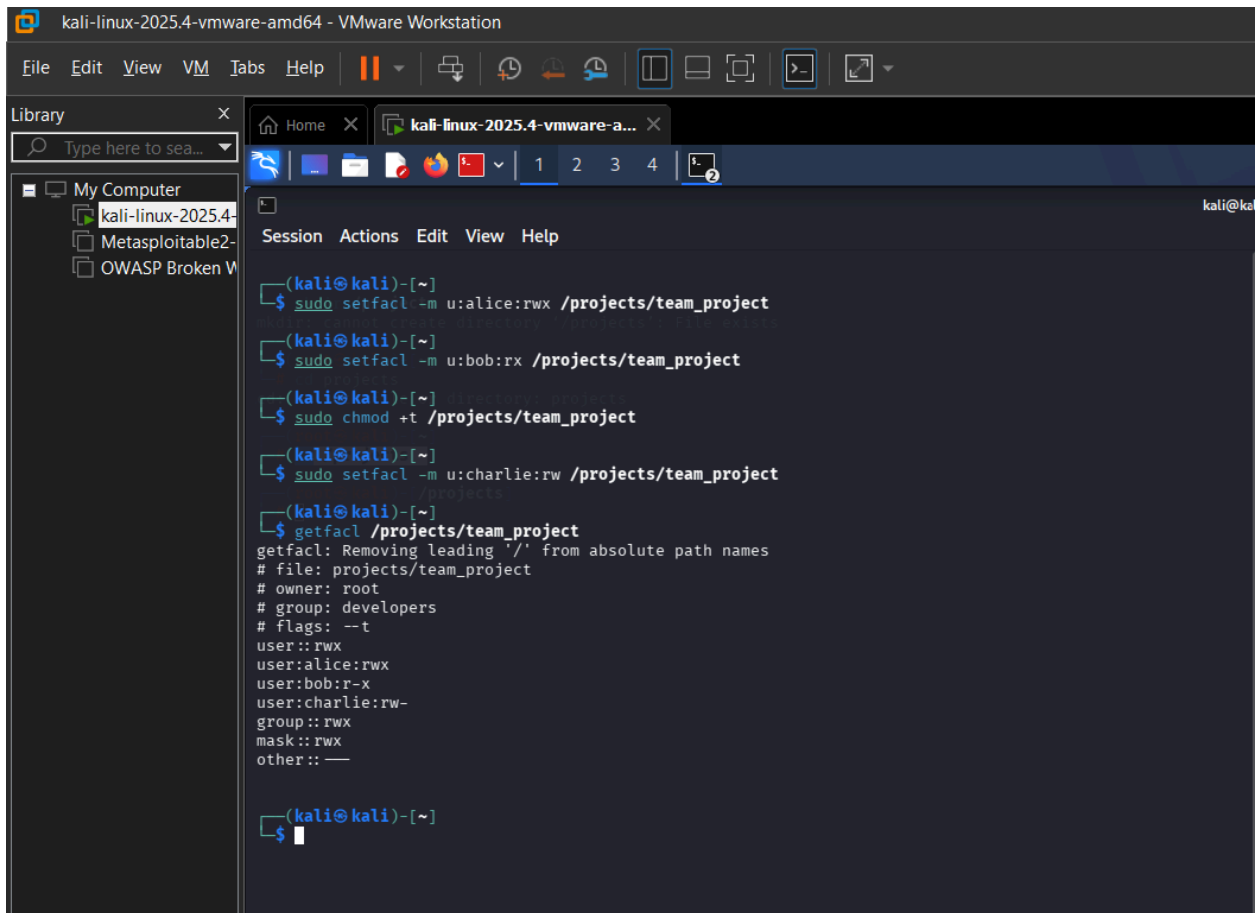
- To prevent deletion, set the sticky bit and assign specific write permissions:

```
sudo chmod +t /projects/team_project sudo setfacl -m u:charlie:rw /projects/team_project
```

4. Verify the ACL Settings:

- Check the ACLs for the directory:

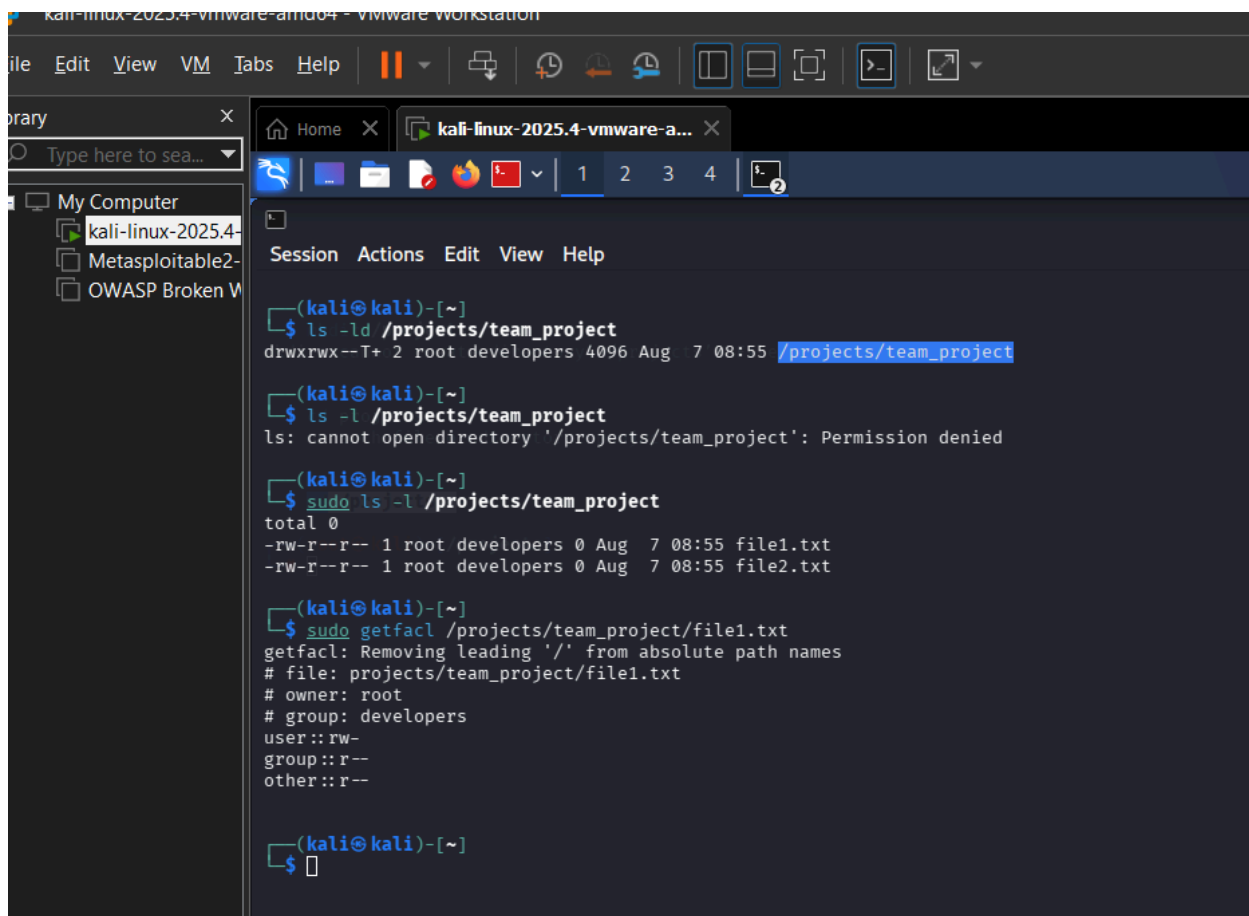
5. `getfacl /projects/team_project`



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

Session Actions Edit View Help
(kali@kali)-[~]
$ sudo setfacl -m u:alice:rw /projects/team_project
(kali@kali)-[~]
$ sudo setfacl -m u:bob:rx /projects/team_project
(kali@kali)-[~]
$ sudo chmod +t /projects/team_project
(kali@kali)-[~]
$ sudo setfacl -m u:charlie:rw /projects/team_project
(kali@kali)-[~]
$ getfacl /projects/team_project
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project
# owner: root
# group: developers
# flags: --t
user::rw
user:alice:rw
user:bob:r-x
user:charlie:rw-
group::rw
mask::rw
other::---
```

The ACLs were successfully configured for Alice, Bob, and Charlie according to the required permissions. The sticky bit was added to the shared project directory to help prevent users from deleting files they do not own. The `getfacl` output confirmed that all ACL settings were applied successfully.

A screenshot of a Kali Linux terminal window. The terminal shows the following commands and output:

```
(kali@kali)-[~]
$ ls -ld /projects/team_project
drwxrwx--T+ 2 root developers 4096 Aug  7 08:55 /projects/team_project

(kali@kali)-[~]
$ ls -l /projects/team_project
ls: cannot open directory '/projects/team_project': Permission denied

(kali@kali)-[~]
$ sudo ls -l /projects/team_project
total 0
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file1.txt
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file2.txt

(kali@kali)-[~]
$ sudo getfacl /projects/team_project/file1.txt
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project/file1.txt
# owner: root
# group: developers
user::rw-
group::r--
other::r--

(kali@kali)-[~]
$
```

The directory permissions and ownership were verified successfully. I had to use sudo to view the contents because the current user did not have permission to access the directory, confirming that the access restrictions were working as configured.

Exercise 3: Test the Permissions

1. Switch to each user and test their access rights:

- For Alice:

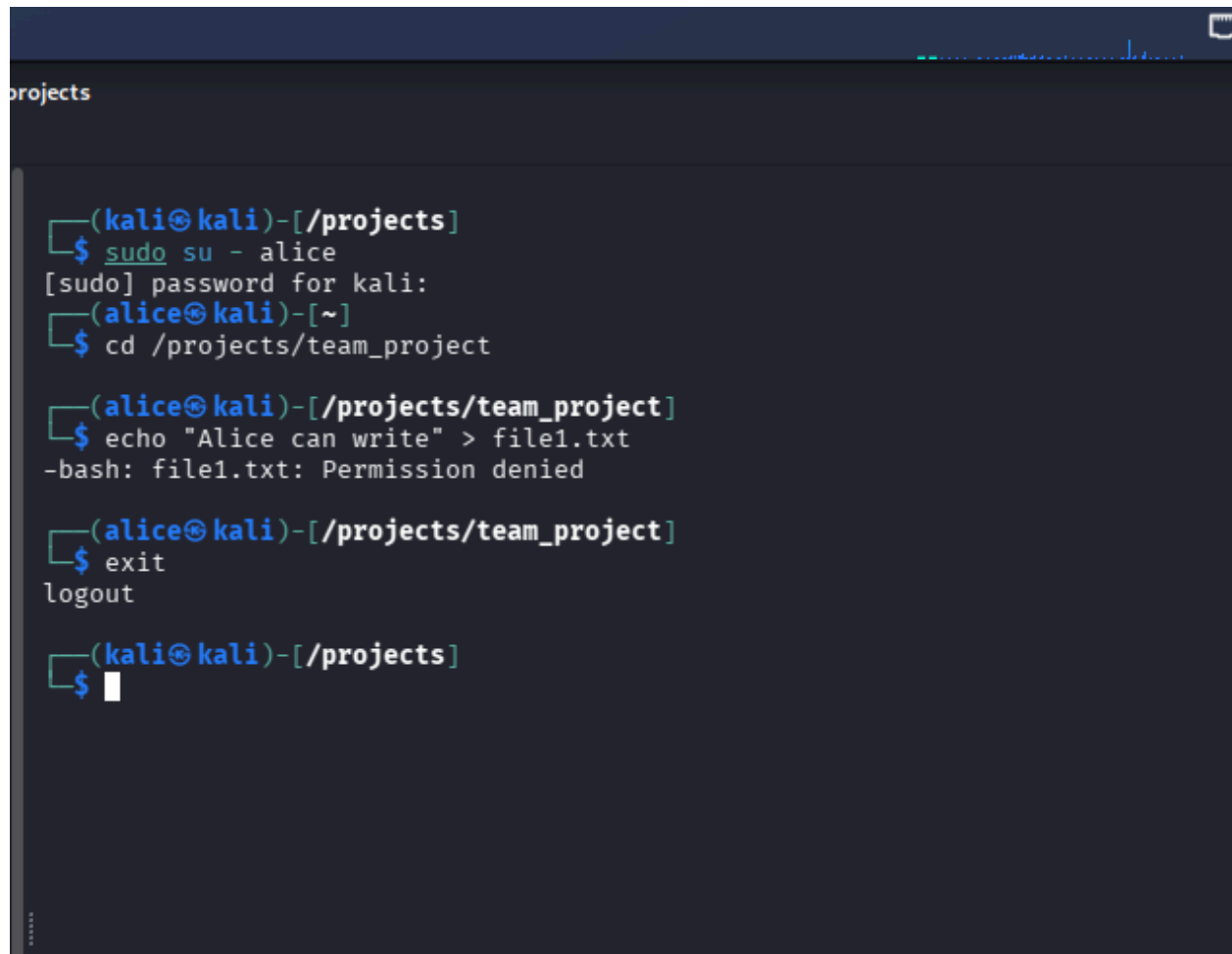
```
sudo su - alice cd /projects/team_project echo "Alice can write" > file1.txt
```

- For Bob:

```
sudo su - bob cd /projects/team_project cat file1.txt echo "Bob can write" >>
file1.txt # This should fail.
```

- For Charlie:

```
sudo su - charlie cd /projects/team_project echo "Charlie can write" > file2.txt rm  
file1.txt # This should fail due to the sticky bit.
```



```
(kali㉿kali)-[/projects]
└─$ sudo su - alice
[sudo] password for kali:
└─(alice㉿kali)-[~]
└─$ cd /projects/team_project

└─(alice㉿kali)-[/projects/team_project]
└─$ echo "Alice can write" > file1.txt
-bash: file1.txt: Permission denied

└─(alice㉿kali)-[/projects/team_project]
└─$ exit
logout

└─(kali㉿kali)-[/projects]
└─$
```

I switched to the Alice account and attempted to write to file1.txt,

Error Message

Received a **"Permission denied"** error. This showed that although the ACLs were configured on the directory, additional permissions were still required for the existing txt file.

What i did to rectify the error

appliedS ACLs to the txt files as well.

Commands Used:

```
sudo setfacl -m u:alice:rw /projects/team_project/file1.txt  
sudo setfacl -m u:bob:r /projects/team_project/file1.txt  
sudo setfacl -m u:charlie:rw /projects/team_project/file2.txt
```

verified:

```
sudo getfacl /projects/team_project/file1.txt
```

```
(kali㉿kali)-[~]  
$ sudo setfacl -m u:alice:rw /projects/team_project/file1.txt  
[sudo] password for kali:
```

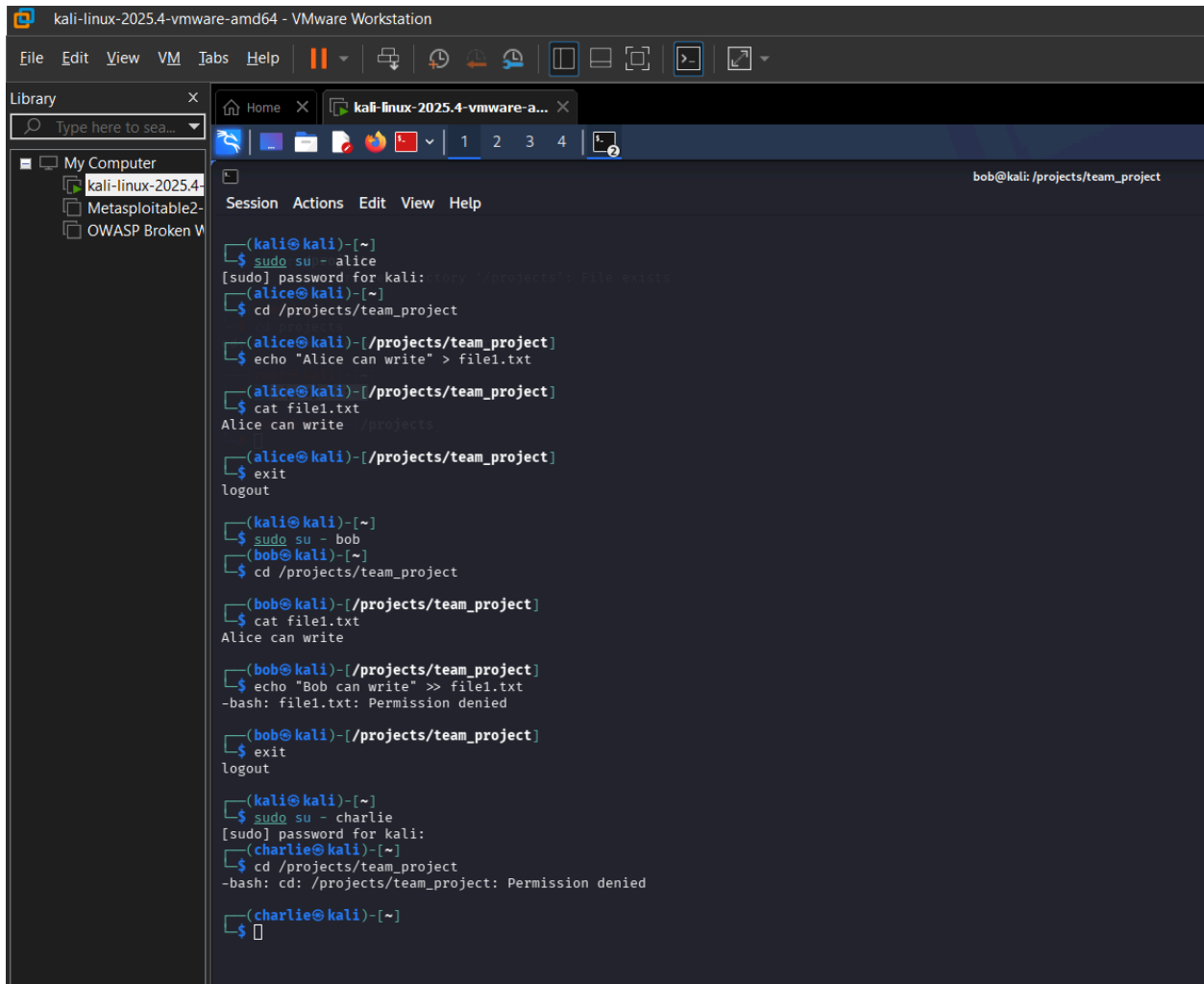
```
(kali㉿kali)-[/projects]
$ sudo setfacl -m u:bob:r /projects/team_project/file1.txt

(kali㉿kali)-[/projects]
$ sudo setfacl -m u:charlie:rw /projects/team_project/file2.txt

(kali㉿kali)-[/projects]
$ sudo getfacl /projects/team_project/file1.txt
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project/file1.txt
# owner: root
# group: developers
user::rw-
user:alice:rw-
user:bob:r--
group::r--
mask::rw-
other::r--

(kali㉿kali)-[/projects]
$
```

Tested for Permissions again



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken V
Home kali-linux-2025.4-vmware-a...
1 2 3 4
bob@kali: /projects/team_project
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo su - alice
[sudo] password for kali: 
(alice@kali)-[~]
$ cd /projects/team_project
(alice@kali)-[/projects/team_project]
$ echo "Alice can write" > file1.txt
(alice@kali)-[/projects/team_project]
$ cat file1.txt
Alice can write /projects
(alice@kali)-[/projects/team_project]
$ exit
logout
(kali@kali)-[~]
$ sudo su - bob
(bob@kali)-[~]
$ cd /projects/team_project
(bob@kali)-[/projects/team_project]
$ cat file1.txt
Alice can write
(bob@kali)-[/projects/team_project]
$ echo "Bob can write" >> file1.txt
-bash: file1.txt: Permission denied
(bob@kali)-[/projects/team_project]
$ exit
logout
(kali@kali)-[~]
$ sudo su - charlie
[sudo] password for kali: 
(charlie@kali)-[~]
$ cd /projects/team_project
-bash: cd: /projects/team_project: Permission denied
(charlie@kali)-[~]
$
```

Alice was able to access the project directory, write to **file1.txt**, and verify its contents successfully. This confirmed that her read, write, and execute permissions were configured correctly.

The ACLs were successfully applied to **file1.txt**, giving **Alice** read and write access while **Bob** was granted read-only access. The `getfacl` output confirmed that the permissions were configured correctly for each user.

Charlie was unable to access the **team_project** directory because he did not have execute (x) permission on the directory. As a result, Linux correctly denied access, preventing him from reaching the files inside.

Exercise 4: Manage Default ACLs

1. Set default ACLs so new files inherit permissions:

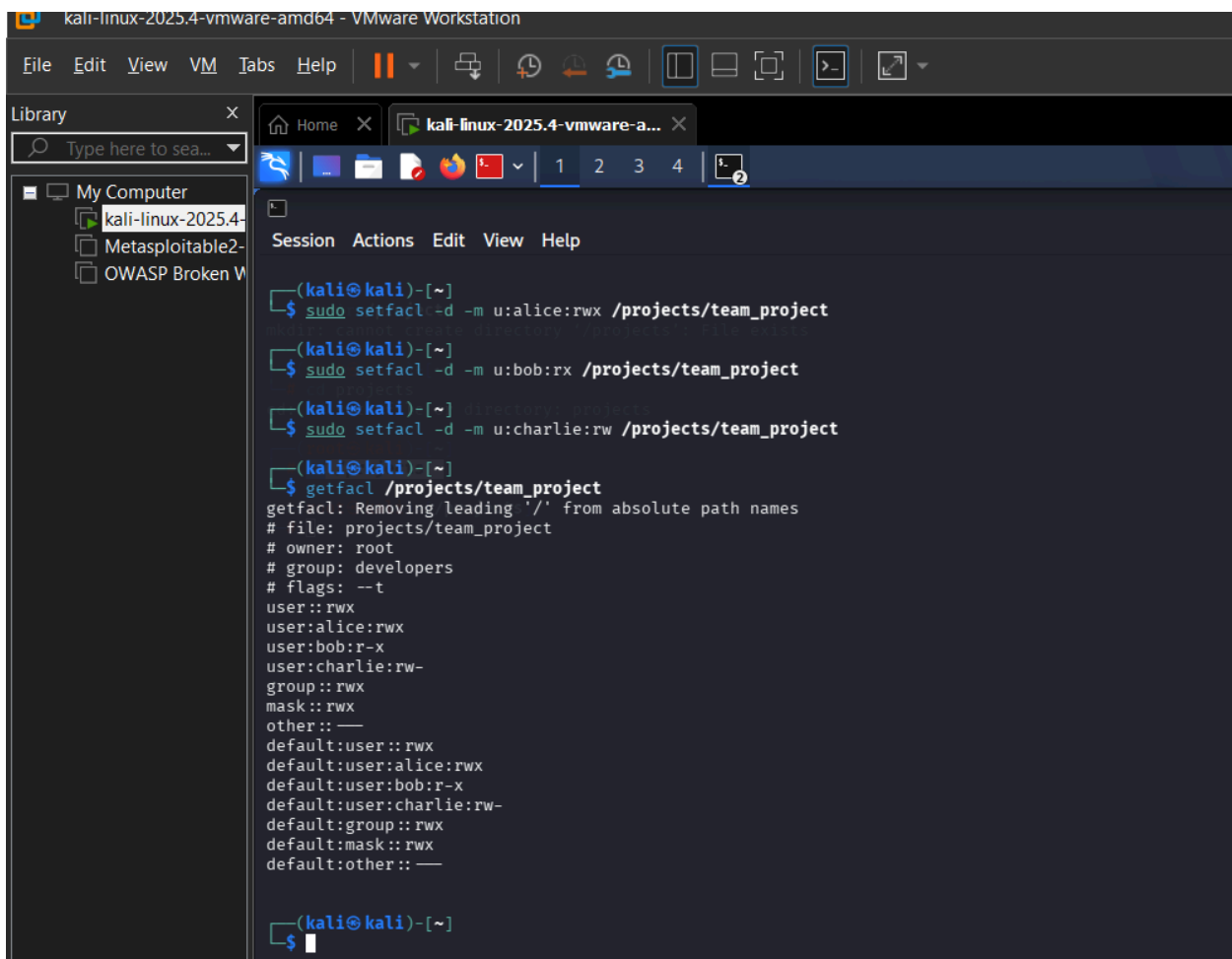
```
sudo setfacl -d -m u:alice:rw /projects/team_project
```

```
sudo setfacl -d -m u:bob:r-x /projects/team_project
```

```
sudo setfacl -d -m u:charlie:rw /projects/team_project
```

2. Verify default ACLs:

```
getfacl /projects/team_project
```



The screenshot shows a terminal window titled 'kali-linux-2025.4-vmware-amd64 - VMware Workstation'. The terminal displays the following commands and output:

```
(kali@kali)-[~]  
$ sudo setfacl -d -m u:alice:rw /projects/team_project  
(kali@kali)-[~]  
$ sudo setfacl -d -m u:bob:r-x /projects/team_project  
(kali@kali)-[~]  
$ sudo setfacl -d -m u:charlie:rw /projects/team_project  
(kali@kali)-[~]  
$ getfacl /projects/team_project  
getfacl: Removing leading '/' from absolute path names  
# file: projects/team_project  
# owner: root  
# group: developers  
# flags: --t  
user::rw  
user:alice:rw  
user:bob:r-x  
user:charlie:rw-  
group::rw  
mask::rw  
other::---  
default:user::rw  
default:user:alice:rw  
default:user:bob:r-x  
default:user:charlie:rw-  
default:group::rw  
default:mask::rw  
default:other::---
```

The default ACLs were successfully configured for Alice, Bob, and Charlie on the team_project directory. The getfacl output confirmed that all new files and directories created inside the project folder will automatically inherit these permissions

Sudo and Privilege Management

Scenario

As a system administrator, you are tasked with managing user privileges for a team of engineers. Each user has different access requirements for administrative exercises.

1. John: Needs full administrative privileges to perform any Exercise.
2. Mary: Can manage system updates but cannot modify user accounts or access sensitive files.
3. Paul: Can restart and manage specific services like Apache and MySQL but cannot make any other system changes.

Your Exercise is to configure sudo privileges to meet these requirements.

Create User Accounts

Commands Used:

```
sudo useradd -m john  
sudo useradd -m mary  
sudo useradd -m paul
```

Set Passwords:

```
sudo passwd john  
sudo passwd mary  
sudo passwd paul
```

Verified Users:

```
grep -E "john|mary|paul" /etc/passwd
```

```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
library
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

Session Actions Edit View Help
(kali@olivil)-[~]
$ sudo useradd -m john
[sudo] password for kali:
(kali@olivil)-[~]
$ sudo useradd -m mary
(kali@olivil)-[~]
$ sudo useradd -m paul
(kali@olivil)-[~]
$ sudo passwd john
New password:
Retype new password:
passwd: password updated successfully
(kali@olivil)-[~]
$ sudo passwd mary
New password:
Retype new password:
passwd: password updated successfully
(kali@olivil)-[~]
$ sudo passwd paul
New password:
Retype new password:
passwd: password updated successfully
(kali@olivil)-[~]
$ grep -E "john|mary|paul" /etc/passwd
john:x:1009:1009::/home/john:/bin/sh
mary:x:1010:1010::/home/mary:/bin/sh
paul:x:1011:1011::/home/paul:/bin/sh
(kali@olivil)-[~]
$
```

The user accounts John, Mary, and Paul were created successfully, and secure passwords were assigned to each account. The `/etc/passwd` output confirmed that all three users were created with their own home directories and default login shells.

Exercise 2: Configure Sudo Privilege

Granted John Full Administrative Privileges

Commands Used For John:

1. Granted John Full Administrative Privileges by adding him to the sudo group
- `sudo usermod -aG sudo john`

Verifies John was successfully added to the sudo group

- `groups john`

Tested for John's sudo access by:

Switching to John's account

`su - john`

Then ran:

`sudo whoami`

I then exit john's account.

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ sudo usermod -aG sudo john  
[sudo] password for kali:  
  
(kali@olivil)-[~]  
$ sudo usermod -aG sudo john  
  
(kali@olivil)-[~]  
$ groups john  
john : john sudo  
  
(kali@olivil)-[~]  
$ su - john  
Password:  
su: Authentication failure  
  
(kali@olivil)-[~]  
$ su - john  
Password:  
$ sudo whoami  
[sudo] password for john:  
root  
$ exit  
  
(kali@olivil)-[~]  
$
```

John was successfully added to the sudo group and granted full administrative privileges. After logging in as John, the sudo whoami command returned root, confirming that the configuration was successful.

2. Grant Mary Privileges to Manage System Updates

Commands Used:

- Edits the sudoers file:

sudo visudo

- Added the following rule at the end:

mary ALL=(ALL) NOPASSWD: /usr/bin/apt update, /usr/bin/apt upgrade
- Tested Mary's access:

su - mary sudo apt update # Should work without a password. sudo useradd testuser # Should fail.

3. Grant Paul Access to Manage Specific Services:

- Edits the sudoers file:

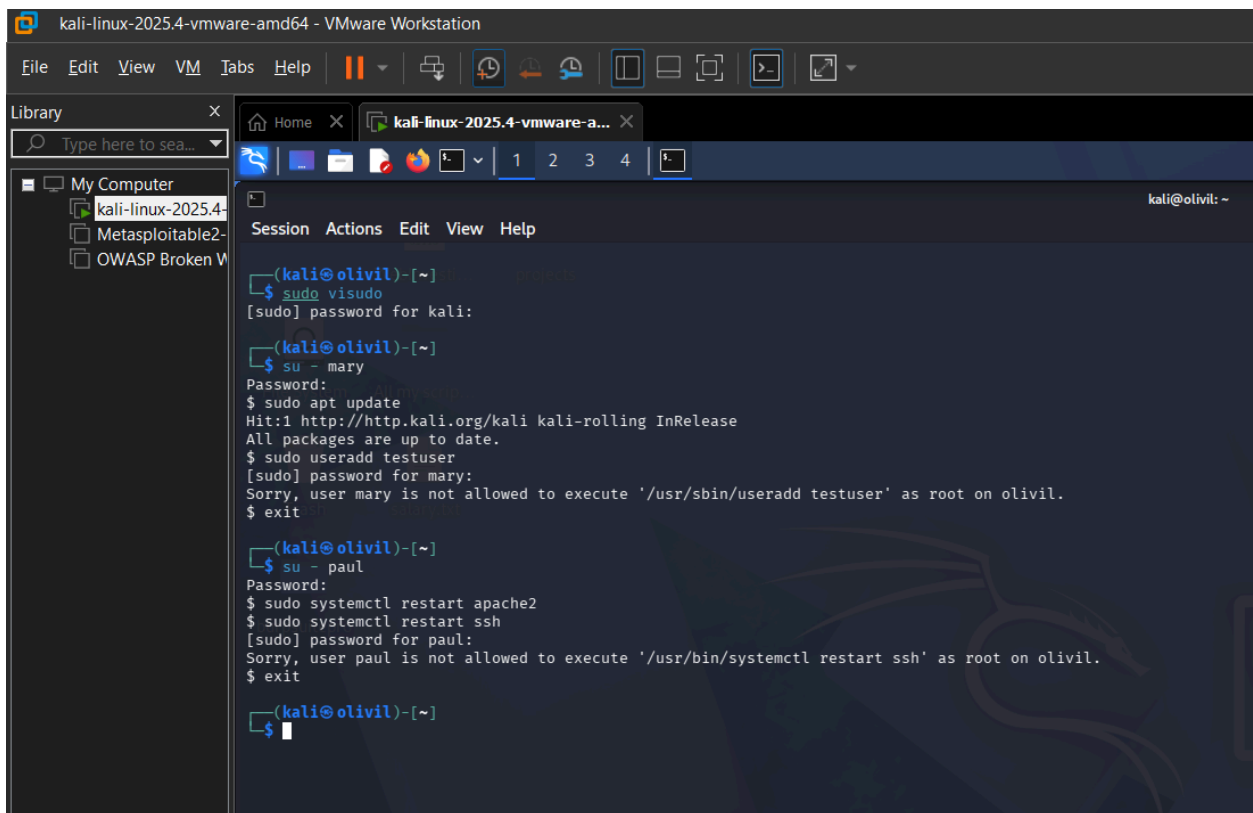
sudo visudo

- Add the following rule:

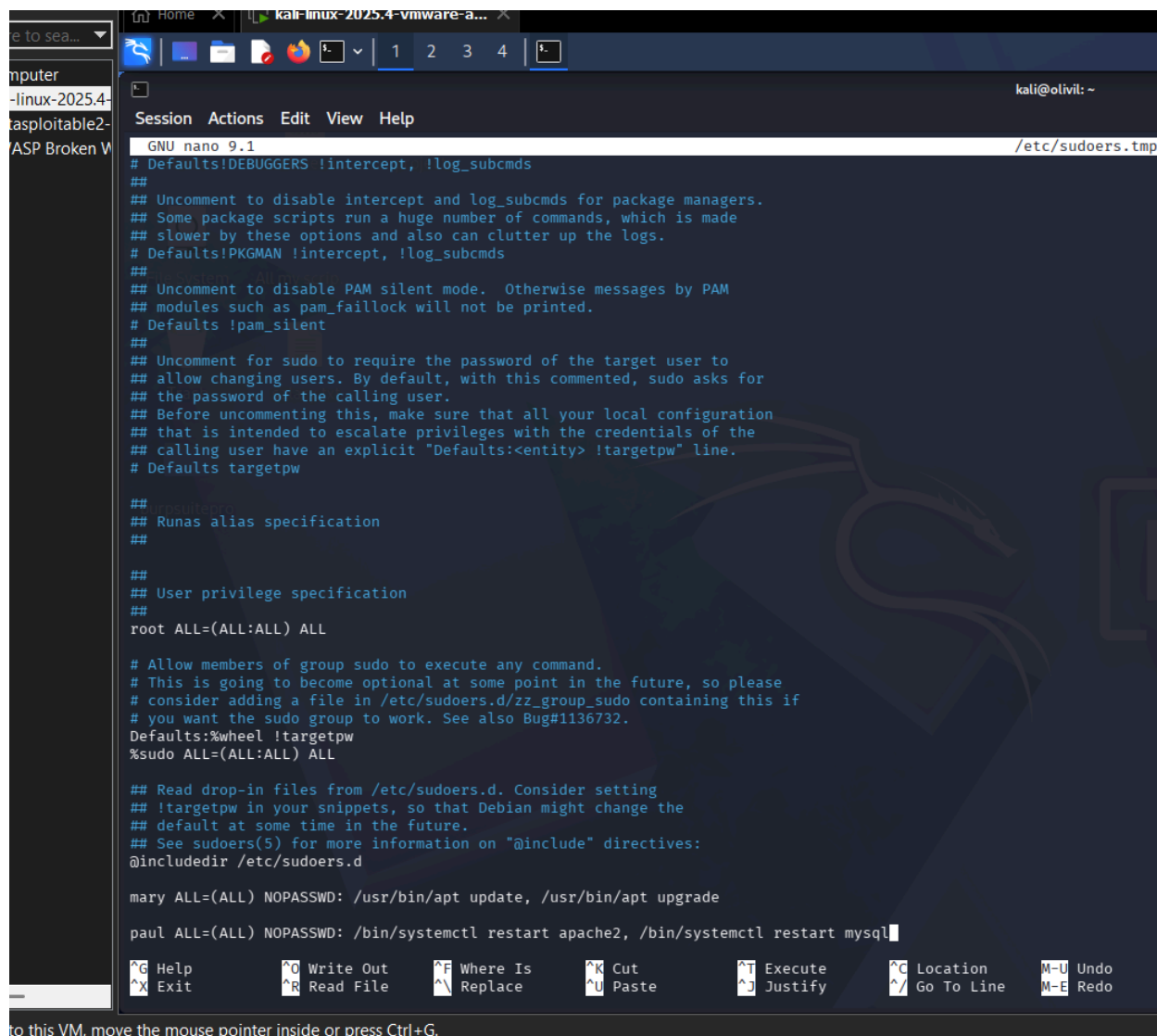
paul ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart mysql

- Test Paul's access:

su - paul sudo systemctl restart apache2
.sudo systemctl restart sshl.



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken V
kali@olivil: ~
Session Actions Edit View Help
(kali@olivil)-[~]
$ sudo visudo
[sudo] password for kali:
(kali@olivil)-[~]
$ su - mary
Password:
$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
All packages are up to date.
$ sudo useradd testuser
[sudo] password for mary:
Sorry, user mary is not allowed to execute '/usr/sbin/useradd testuser' as root on olivil.
$ exit
(kali@olivil)-[~]
$ su - paul
Password:
$ sudo systemctl restart apache2
$ sudo systemctl restart ssh
[sudo] password for paul:
Sorry, user paul is not allowed to execute '/usr/bin/systemctl restart ssh' as root on olivil.
$ exit
(kali@olivil)-[~]
$
```



```
GNU nano 9.1 /etc/sudoers.tmp
# Defaults!DEBBUGERS !intercept, !log_subcmds
##
## Uncomment to disable intercept and log_subcmds for package managers.
## Some package scripts run a huge number of commands, which is made
## slower by these options and also can clutter up the logs.
# Defaults!PKGMAN !intercept, !log_subcmds
##
## Uncomment to disable PAM silent mode. Otherwise messages by PAM
## modules such as pam_faillock will not be printed.
# Defaults !pam_silent
##
## Uncomment for sudo to require the password of the target user to
## allow changing users. By default, with this commented, sudo asks for
## the password of the calling user.
## Before uncommenting this, make sure that all your local configuration
## that is intended to escalate privileges with the credentials of the
## calling user have an explicit "Defaults:<entity> !targetpw" line.
# Defaults targetpw

##
## Runas alias specification
##

##
## User privilege specification
##
root ALL=(ALL:ALL) ALL

# Allow members of group sudo to execute any command.
# This is going to become optional at some point in the future, so please
# consider adding a file in /etc/sudoers.d/zz_group_sudo containing this if
# you want the sudo group to work. See also Bug#1136732.
Defaults:%wheel !targetpw
%sudo ALL=(ALL:ALL) ALL

## Read drop-in files from /etc/sudoers.d. Consider setting
## !targetpw in your snippets, so that Debian might change the
## default at some time in the future.
## See sudoers(5) for more information on "@include" directives:
@include /etc/sudoers.d

mary ALL=(ALL) NOPASSWD: /usr/bin/apt update, /usr/bin/apt upgrade

paul ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart mysql
```

- Mary successfully ran apt update without being granted permission to manage user accounts. Her attempt to create a testuser was denied, confirming that her sudo privileges were restricted as required.
- Paul successfully restarted the Apache service without a password, while his attempt to restart SSH was denied. This confirmed that his sudo privileges were limited to the specific services assigned to him.

Exercise 3: Restrict Access to the Sudo Command

1. Prevented unauthorized users from using sudo

- Ensured only specific users are part of the sudo

Commands Used:

getent group sudo

2. Removed a user from the Sudo

sudo deluser Ahmed sudo

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ getent group sudo  
sudo:x:27:kali,olivil,john,Ahmed  
  
(kali@olivil)-[~]  
$ sudo deluser Ahmed sudo  
[sudo] password for kali:  
  
(kali@olivil)-[~]  
$ getent group sudo  
sudo:x:27:kali,olivil,john  
  
(kali@olivil)-[~]  
$
```

Ahmed, who was one of the users, was successfully removed from the sudo group. The final output confirmed that kali, olivil, and John remain members with sudo access.

Exercise 4: Logging and Monitoring Sudo Usage

1. Enabled sudo logs in using

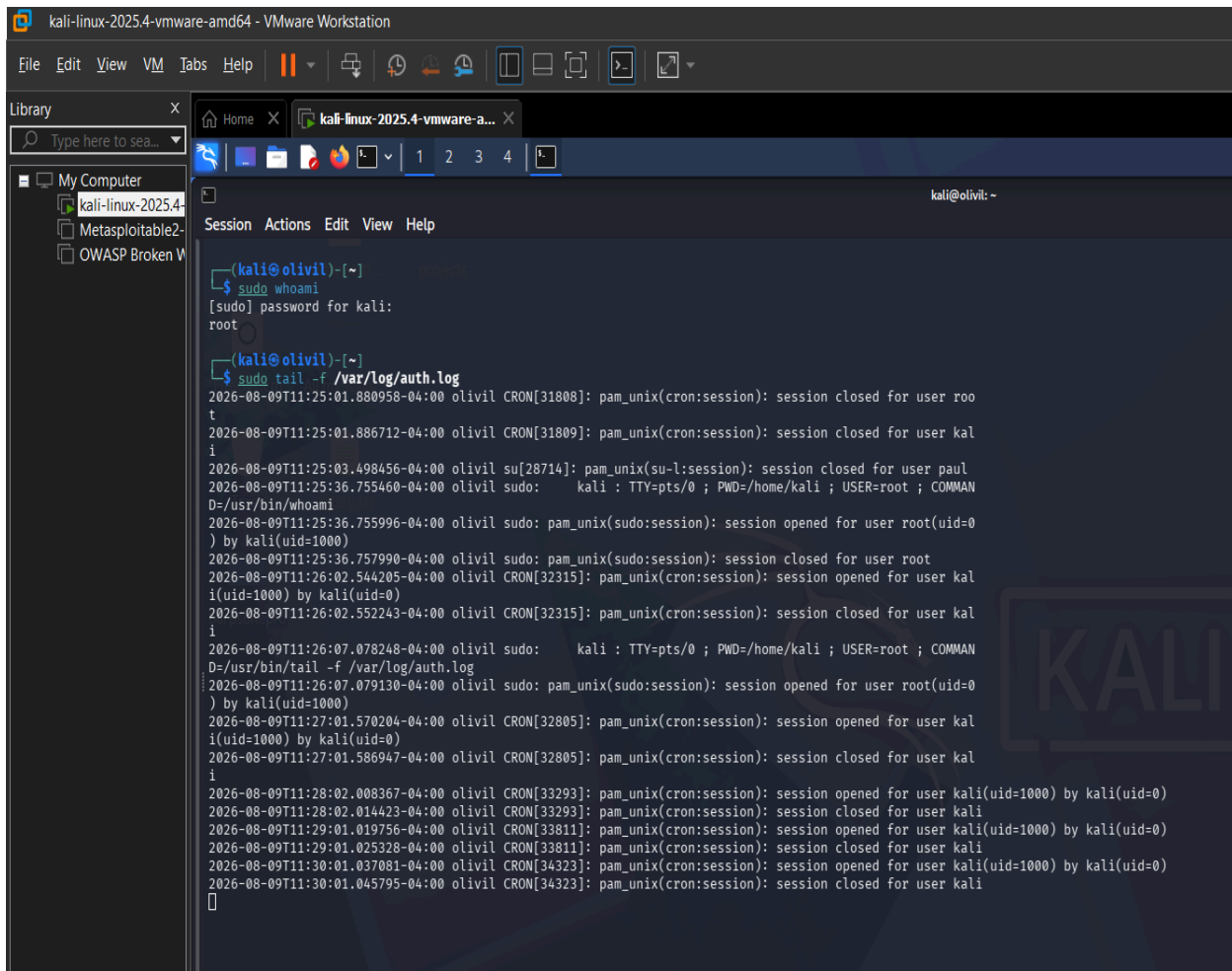
Commands Used:

/var/log/auth.log

2. Tested logging by running a command as a sudo user and checked the log.

sudo tail -f /var/log/auth.log

3. Look for entries indicating which commands were executed using sudo



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken W

kali@olivil: ~
$ sudo whoami
[sudo] password for kali:
root

kali@olivil: ~
$ sudo tail -f /var/log/auth.log
2026-08-09T11:25:01.880958-04:00 olivil CRON[31808]: pam_unix(cron:session): session closed for user root
2026-08-09T11:25:01.886712-04:00 olivil CRON[31809]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:25:03.498456-04:00 olivil su[28714]: pam_unix(su-l:session): session closed for user paul
2026-08-09T11:25:36.755460-04:00 olivil sudo:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/whoami
2026-08-09T11:25:36.755996-04:00 olivil sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
2026-08-09T11:25:36.757990-04:00 olivil sudo: pam_unix(sudo:session): session closed for user root
2026-08-09T11:26:02.544205-04:00 olivil CRON[32315]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:26:02.552243-04:00 olivil CRON[32315]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:26:07.078248-04:00 olivil sudo:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
2026-08-09T11:26:07.079130-04:00 olivil sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
2026-08-09T11:27:01.570204-04:00 olivil CRON[32805]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:27:01.586947-04:00 olivil CRON[32805]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:28:02.008367-04:00 olivil CRON[33293]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:28:02.014423-04:00 olivil CRON[33293]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:29:01.019756-04:00 olivil CRON[33811]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:29:01.025328-04:00 olivil CRON[33811]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:30:01.037081-04:00 olivil CRON[34323]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:30:01.045795-04:00 olivil CRON[34323]: pam_unix(cron:session): session closed for user kali
```

The sudo command was successfully executed as olivil, returning root. The /var/log/auth.log output also recorded the sudo whoami command and the

corresponding root session, confirming that sudo activity is being logged successfully

How the Sudoers File Is Used to Manage Privileges

The sudoers file is used to control which users can run administrative commands and what level of access they have. In this lab, it was used to give John full administrative privileges, while Mary and Paul were given limited access to only the specific commands required for their tasks. This helps improve system security by following the principle of least privilege and preventing unauthorized system changes.

Securing Network Services and Ports

1. List Open Ports.

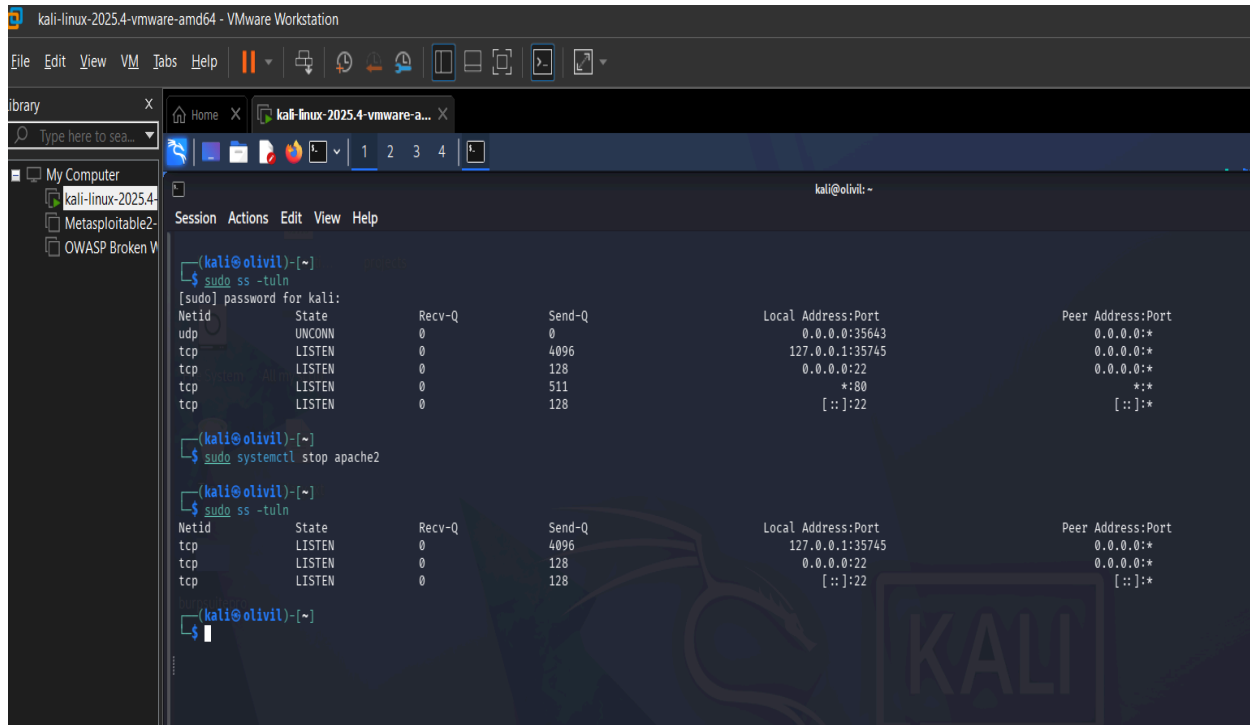
Command Used:

```
sudo ss -tln
```

2. Closed Unnecessary Ports by:

- Stopping an unnecessary service Apache:

```
sudo systemctl stop apache2 sudo systemctl disable apache2
```



- The ss command successfully displayed the active listening ports on the Kali system. Ports 22 (SSH) and 80 (HTTP/Apache) were identified as open TCP ports, along with local UDP/TCP services.
- Apache was successfully stopped and disabled, and port 80 was no longer listening. This confirmed that the unnecessary network service was successfully closed.

Exercise 4: SSH and Remote Access

1. Secured the SSH Service by:
 - Editing the SSH configuration file.

Commands Used:

`sudo nano /etc/ssh/sshd_config`

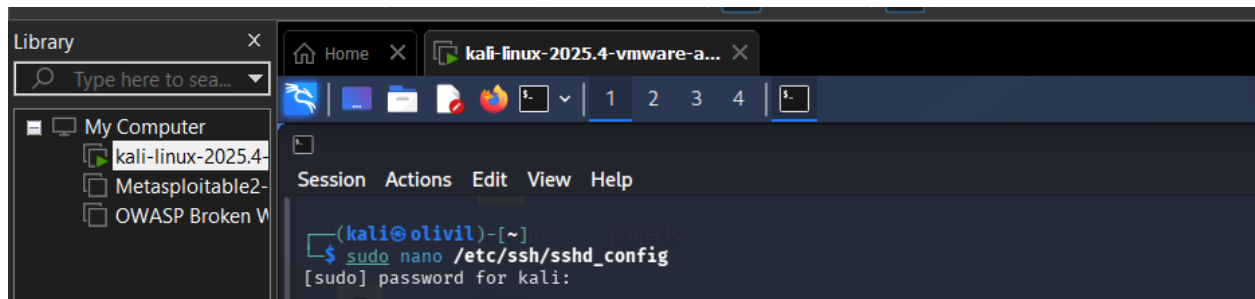
- Changed the SSH port #Port22 to 2222

Then saved and exit:

- Ctrl + O

- Press Enter
- Ctrl + X

2. Restarted SSH using:
`sudo systemctl restart sshd`



The image shows a Kali Linux virtual machine running in VMware. The terminal window displays the contents of the `/etc/ssh/sshd_config` file, which is being edited with the `nano` text editor. The file contains various configuration options for the SSH daemon, including listening address, host keys, ciphers, logging, and authentication settings. The `Port 2222` line is highlighted. The bottom of the terminal shows a status bar with keyboard shortcuts for nano editor actions.

```
File Edit View VM Tabs Help
kali-linux-2025.4-vmware-a...
Type here to sea...
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken W

Session Actions Edit View Help
GNU nano 9.1 /etc/ssh/sshd_config *

# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH

#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

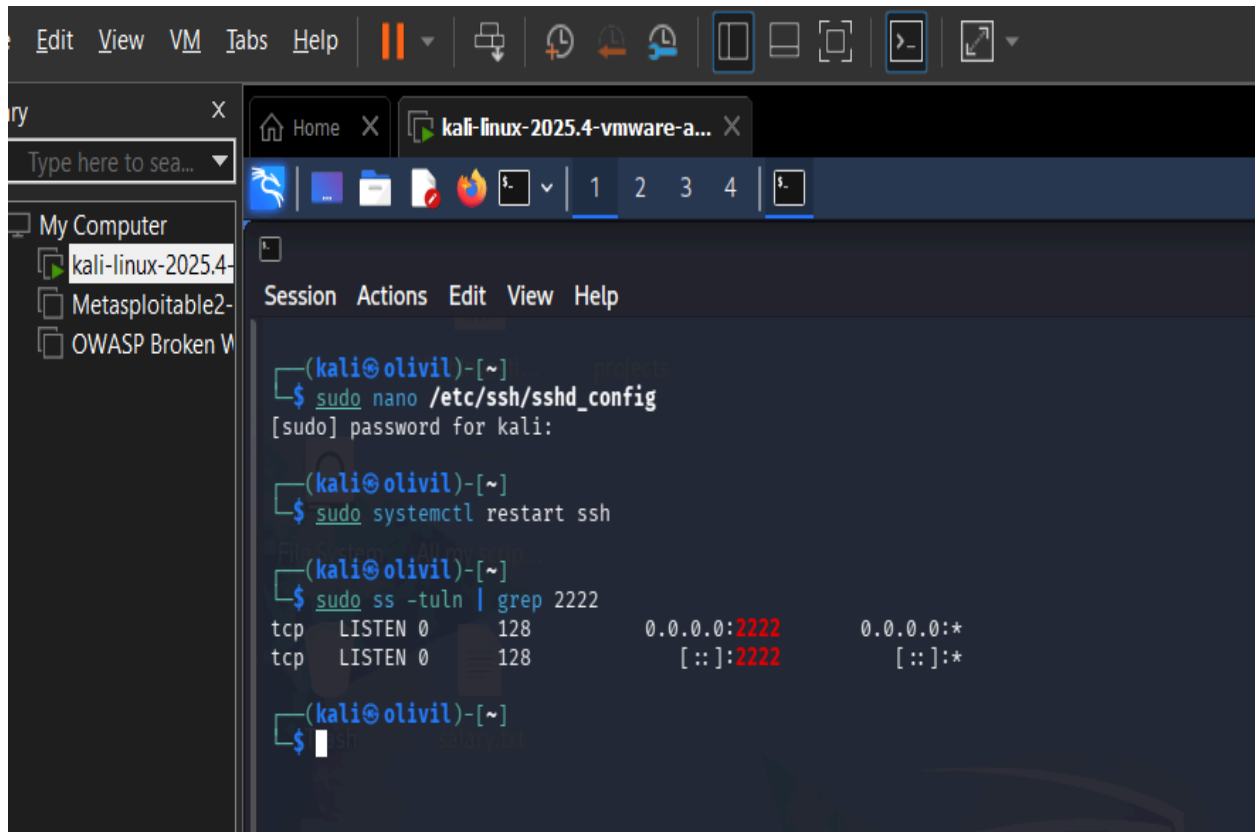
#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E
```

to direct input to this VM, move the mouse pointer inside or press Ctrl+G.



SSH was successfully reconfigured to use port 2222 instead of port 22. The ss output confirmed that SSH is listening on port 2222 for both IPv4 and IPv6 connections.

Exercise 5: System Update and Patching

1. Check for Updates.

Commands used:

`sudo apt update`

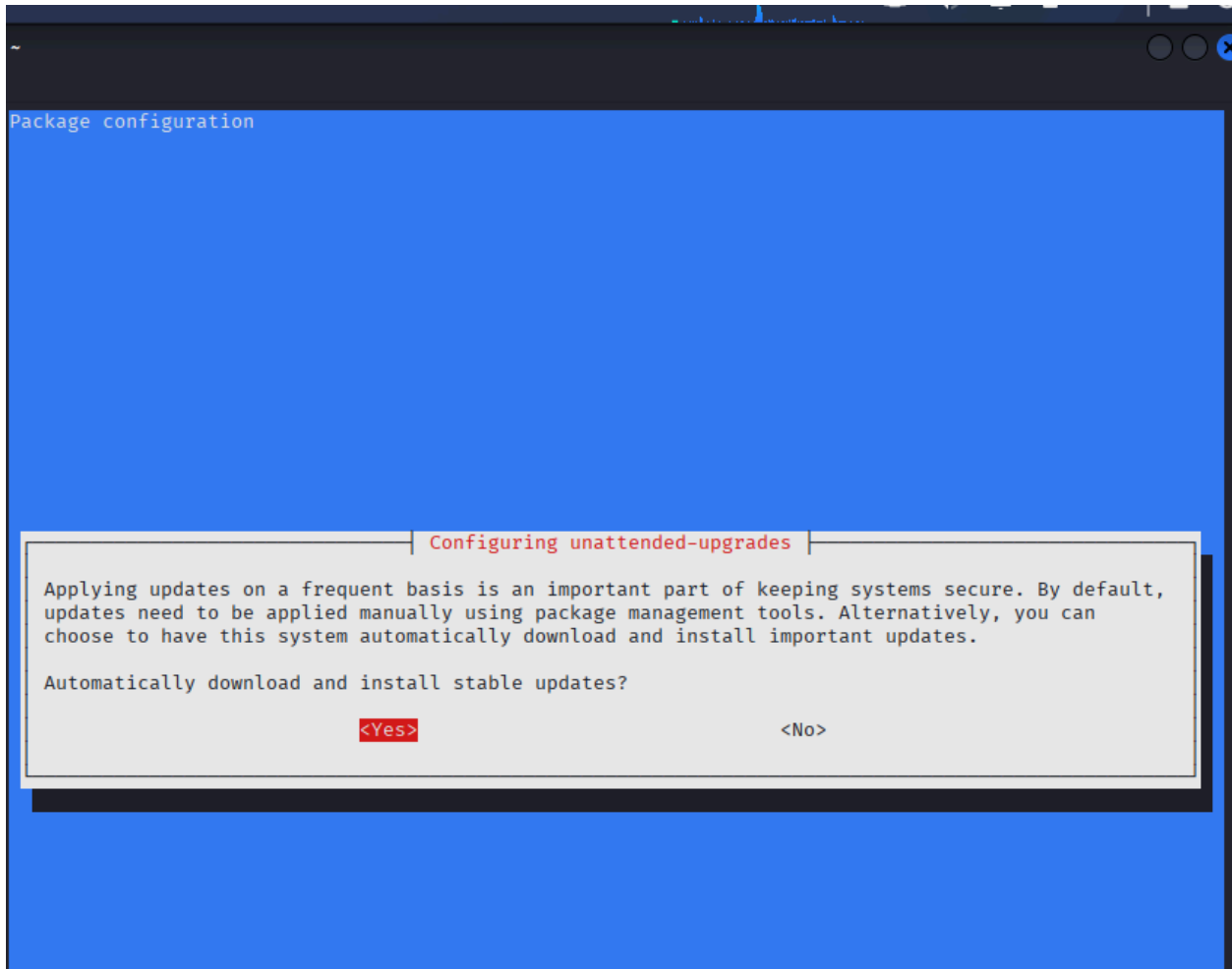
`sudo apt upgrade`

2. Automate Updates:

- Install unattended-upgrades:

`sudo apt install unattended-upgrades`

`sudo dpkg-reconfigure unattended-upgrades`



The system was successfully updated using `apt update` and `apt upgrade`. The `unattended-upgrades` package was installed and automatic stable updates were enabled to help keep the system secure.

Implementing Two-Factor Authentication (2FA) with Google Authenticator

Step 1: Installed Google Authenticator

1. Update the system package index:
`sudo apt update`
2. Install the Google Authenticator PAM module:
`sudo apt install libpam-google-authenticator -y`

The screenshot shows a Kali Linux terminal window with the following commands and output:

```
(kali@olivil)-[~]
$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Ign:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb)
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [168 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [889 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Fetched 18.6 MB in 4min 56s (62.8 kB/s)
24 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@olivil)-[~]
$ sudo apt install libpam-google-authenticator -y
Installing:
  libpam-google-authenticator

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 24
  Download size: 46.9 kB
  Space needed: 132 kB / 51.5 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 libpam-google-authenticator amd64 20250213-1.11-0.1 [46.9 kB]
Fetched 46.9 kB in 24s (1,960 B/s)
Selecting previously unselected package libpam-google-authenticator.
(Reading database... 454924 files and directories currently installed.)
Preparing to unpack .../libpam-google-authenticator_20250213-1.11-0.1_amd64.deb...
Unpacking libpam-google-authenticator (20250213-1.11-0.1)...
Setting up libpam-google-authenticator (20250213-1.11-0.1)...
Processing triggers for kali-menu (2026.3.2)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.

(kali@olivil)-[~]
$
```

The Google Authenticator PAM module was successfully installed after updating the system package index. The system is now ready for the user-specific 2FA configuration.

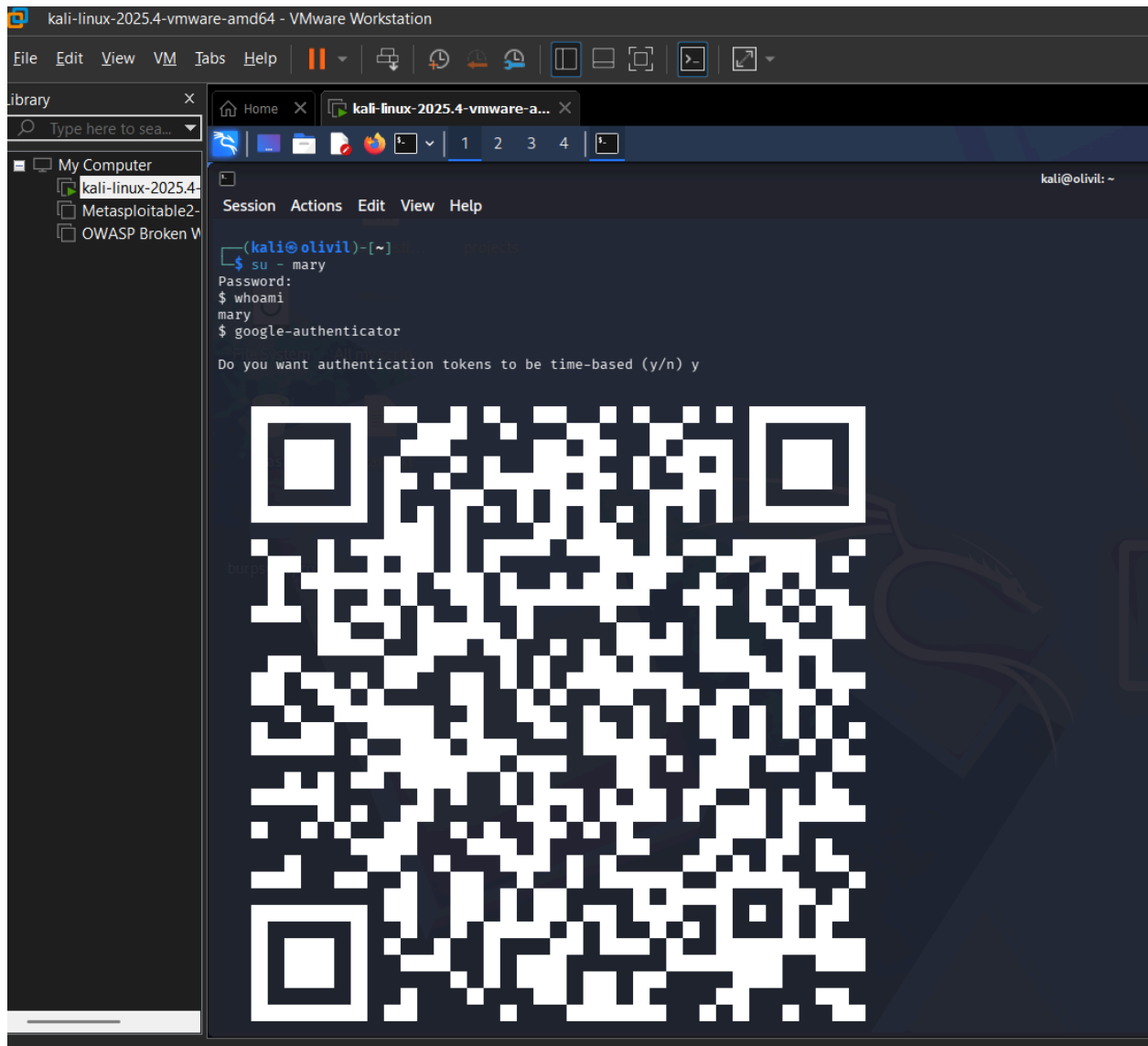
Step 2: Configure Google Authenticator for Your User

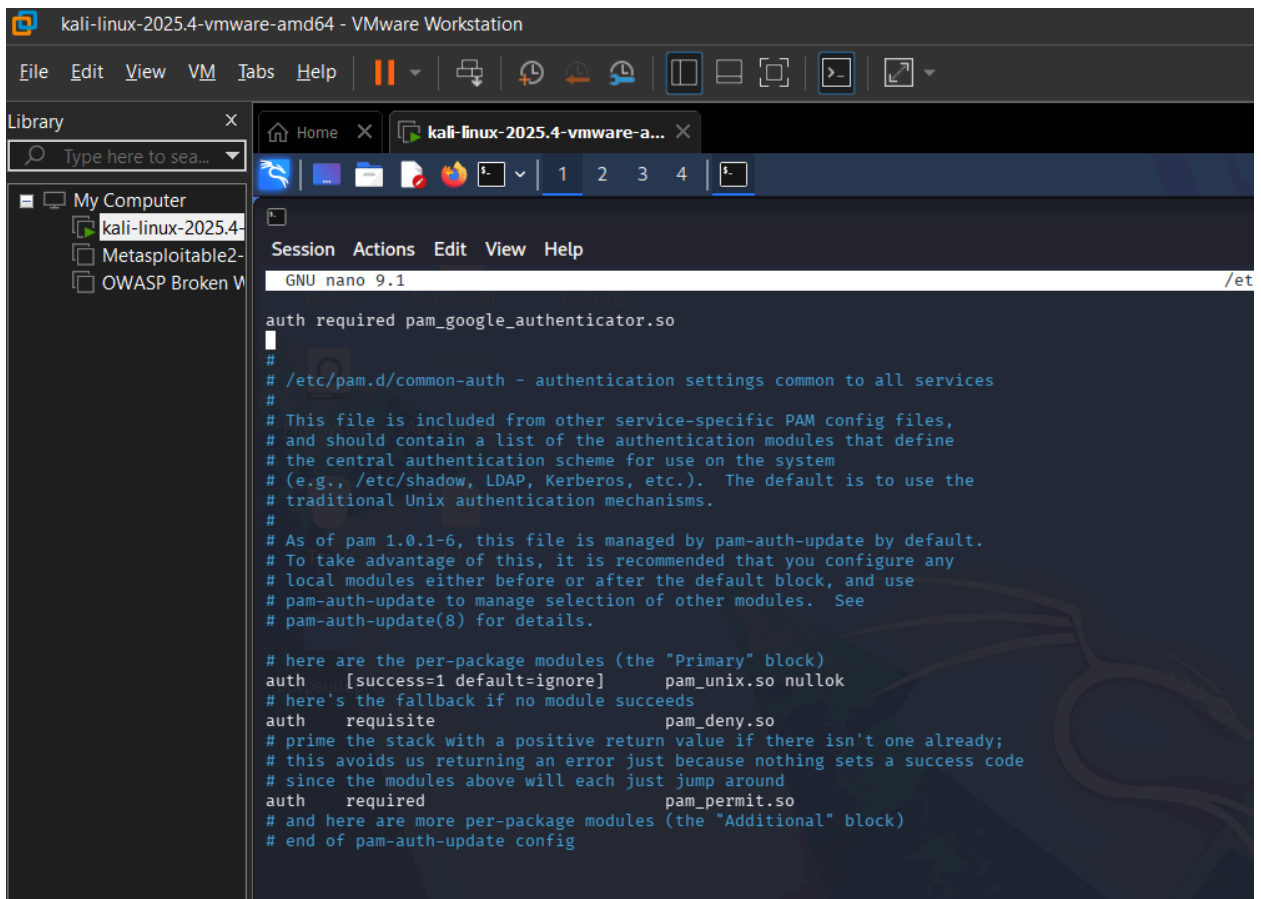
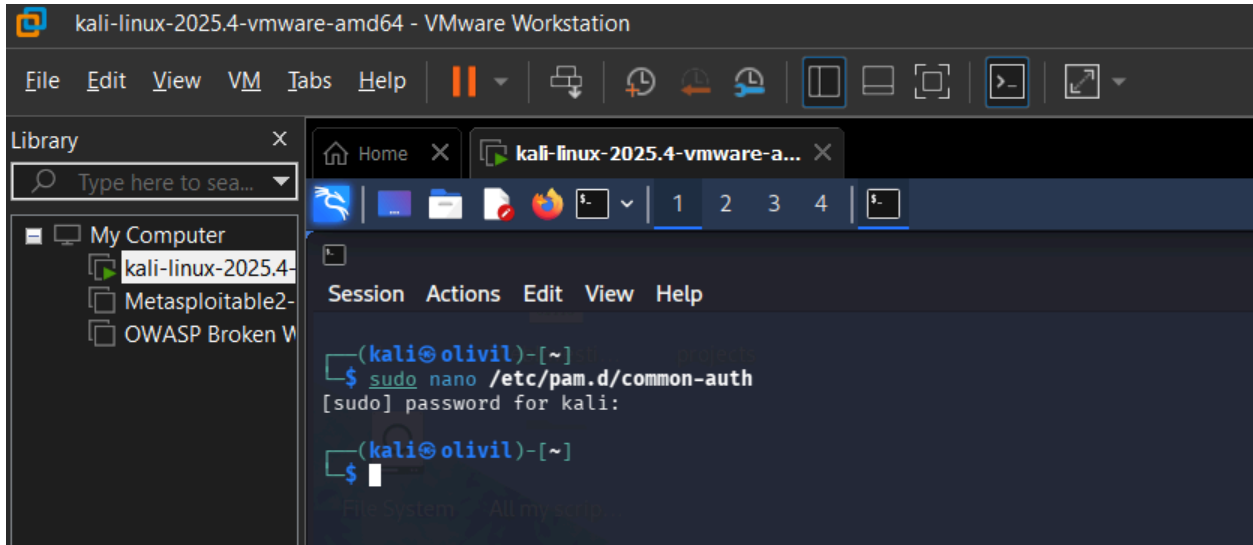
1. Switch to the user account you want to configure 2FA for:
su - <username>

2. Run the Google Authenticator setup:
google-authenticator

3. Follow the prompts:

- QR Code: Scan the QR code using the Google Authenticator app on your smartphone.
- Emergency Codes: Save the emergency backup codes provided (these can be used if you lose access to the app).
- Enable Time-Based Tokens: Answer yes when asked if you want to enable time-based tokens.

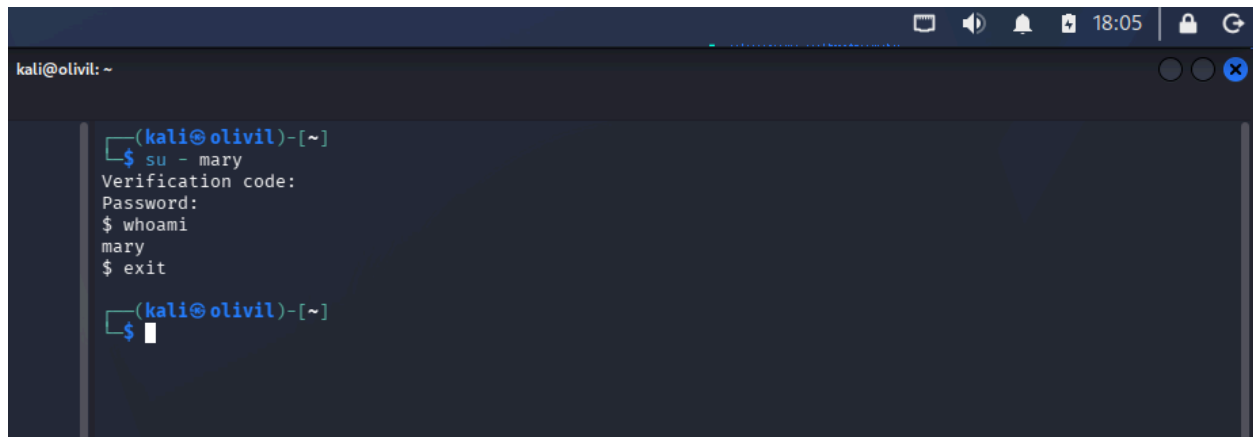




The PAM configuration was successfully updated to include Google Authenticator, enabling 2FA authentication for the system.

Step 4: Test 2FA Configuration

1. Open a new terminal or SSH session and attempt to log in as the configured user.
2. After entering the username and password, you should be prompted for the 2FA token generated by the Google Authenticator app.

A terminal window titled 'kali@olivil: ~' with standard Linux window controls. The prompt is '(kali@olivil)-[~]'. The user enters 'su - mary'. The prompt changes to 'Verification code:'. The user enters a code. The prompt changes to 'Password:'. The user enters a password. The prompt changes to '\$ whoami'. The output is 'mary'. The user enters '\$ exit'. The prompt returns to '(kali@olivil)-[~]'.

```
(kali@olivil)-[~]
$ su - mary
Verification code:
Password:
$ whoami
mary
$ exit
(kali@olivil)-[~]
$
```

Mary successfully logged in by providing both her Google Authenticator verification code and password, confirming that the 2FA setup is working correctly

Implementing Key-Based Authentication and Securing SSH Access

Scenario

You are a cybersecurity analyst at SecureTech Ltd. In addition to disabling root login, the company wants to implement key-based authentication for enhanced SSH security. Your Exercise is to configure SSH for key-based authentication, test the setup, and ensure the root user is securely disabled.

Part 1: SSH Key-Based Authentication

Commands used:

1. Verified the current user and account details

```
whoami  
id  
echo $USER
```

2. Generated a 4096-bit RSA SSH key pair

```
ssh-keygen -t rsa -b 4096
```

This created:

```
/home/kali/.ssh/id_rsa  
/home/kali/.ssh/id_rsa.pub
```

3. Checked the SSH service

```
sudo systemctl status ssh
```

```
kali@olivil: ~  
$ whoami  
kali  
  
kali@olivil: ~  
$ id  
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),300(hark),130(kaboxer)  
  
kali@olivil: ~  
$ echo $USER  
kali  
  
kali@olivil: ~  
$ ssh-keygen -t rsa -b 4096  
Generating public/private rsa key pair.  
Enter file in which to save the key (/home/kali/.ssh/id_rsa):  
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in /home/kali/.ssh/id_rsa  
Your public key has been saved in /home/kali/.ssh/id_rsa.pub  
The key fingerprint is:  
SHA256:MLLvoMqH/wGLIFJf/iGClenk6z2Y+q0U1HwJZFzerw kali@olivil  
The key's randomart image is:  
+--[RSA 4096]--+  
|.o+o.|  
|..++|  
|+++o|  
|oo=0+ooo|  
|*B+.. .S|  
|0.o o.E|  
|.=-....|  
|=o.= o.|  
|=*Boo..|  
+---[SHA256]---+  
  
kali@olivil: ~  
$ sudo systemctl status ssh  
[sudo] password for kali:  
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)  
   Active: active (running) since Sat 2026-08-15 06:55:56 EDT; 32min ago  
     Invocation: c9528cdf1bb840f5b449a2f94dcab702  
       Docs: man:sshd(8)  
             man:sshd_config(5)  
    Process: 997 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)  
   Main PID: 1009 (sshd)
```

The kali user was verified successfully, and an RSA 4096-bit SSH key pair was generated. The private key was stored in `/home/kali/.ssh/id_rsa`, while the public key was stored in `/home/kali/.ssh/id_rsa.pub`. The SSH service was checked and confirmed to be active and running, with the SSH server listening on port 22.

Part 2: Copy Public Key and Test SSH Login

Commands used

1. Copied the public key to the SSH server

ssh-copy-id kali@localhost

2. Accepted the server's host key

yes

3. Authenticate to install the key

kali@localhost's password:

The key was successfully installed, confirmed by:

Number of key(s) added: 1

4. Tested SSH key-based authentication

ssh kali@localhost

5. Exited the SSH session

exit

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ ssh-copy-id kali@localhost  
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_rsa.pub"  
The authenticity of host 'localhost (::1)' can't be established.  
ED25519 key fingerprint is: SHA256:6L5RA5iA+bVS5/4jJDR1KbKK4Uznkb47CV3Bw+lZU4  
This host key is known by the following other names/addresses:  
~/.ssh/known_hosts:1: [hashed name]  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed  
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys  
kali@localhost's password:  
Permission denied, please try again.  
kali@localhost's password:  
  
Number of key(s) added: 1  
  
Now try logging into the machine, with: "ssh 'kali@localhost'"  
and check to make sure that only the key(s) you wanted were added.  
  
(kali@olivil)-[~]  
$ ssh kali@localhost  
Linux olivil 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64  
  
The programs included with the Kali GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
You have mail.  
Last login: Fri Jul 10 05:09:58 2026 from ::1  
(kali@olivil)-[~]  
$ exit  
Connection to localhost closed.  
  
(kali@olivil)-[~]  
$
```

The SSH public key was successfully copied to the kali account on the local SSH server. Although the first password attempt was unsuccessful, the key installation completed successfully, with one key added. The SSH connection was then tested successfully using `ssh kali@localhost`, confirming that the configured SSH key could authenticate the kali user. The SSH session was closed normally using `exit`.

Part 3: Configure SSH for Key-Based Authentication

Commands used:

1. Edited the SSH configuration file

```
sudo nano /etc/ssh/sshd_config
```

2. Configured public-key authentication

```
PubkeyAuthentication yes
```

3. Disabled password authentication

```
PasswordAuthentication no
```

Then:

CTR + O = SAVE

CTR + X =SAVE

4. Verified the SSH configuration

```
sudo sshd -t
```

5. Restarted the SSH service

```
sudo systemctl restart ssh
```

6. Verified that SSH is running

```
sudo systemctl status ssh --no-pager
```

7. Tested key-based login

```
ssh kali@localhost
```

8. Verified the logged-in user

```
whoami
```

```
(kali@olivil)-[~]  
$ sudo nano /etc/ssh/sshd_config  
[sudo] password for kali:  
(kali@olivil)-[~]
```

```
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key
```

```
# Ciphers and keying
#RekeyLimit default none
```

```
# Logging
#SyslogFacility AUTH
```

```
#LogLevel INFO
```

```
# Authentication:
```

```
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

```
PubkeyAuthentication yes
```

```
# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2
```

```
#AuthorizedPrincipalsFile none
```

```
#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody
```

```
# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes
```

```
# To disable tunneled clear text passwords, change to "no" here!
```

```
PasswordAuthentication yes
#PermitEmptyPasswords no
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^F Where Is
^_ Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```

```
^C Location
^_ Go To Line
```

```
M-U Undo
M-E Redo
```

```
M-A Set Mark
M-6 Copy
```

```
M-]
^B
```

```

(kali@olivil)-[~]
$ sudo sshd -t

(kali@olivil)-[~]
$ sudo systemctl restart ssh

(kali@olivil)-[~]
$ sudo systemctl status ssh --no-pager
ssh.service - OpenBSD Secure Shell server
  Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
  Active: active (running) since Sat 2026-08-15 07:55:41 EDT; 1min 35s ago
  Invocation: d4f137b13de749179203c493cea672b6
  Docs: man:sshd(8)
       man:sshd_config(5)
  Process: 27433 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 27436 (sshd)
  Tasks: 1 (limit: 2066)
  Memory: 1.3M (peak: 2.3M)
  CPU: 57ms
  CGroup: /system.slice/ssh.service
          └─27436 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 15 07:55:41 olivil systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Aug 15 07:55:41 olivil sshd[27436]: Server listening on 0.0.0.0 port 22.
Aug 15 07:55:41 olivil sshd[27436]: Server listening on :: port 22.
Aug 15 07:55:41 olivil systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@olivil)-[~]
$ ssh kali@localhost
linux olivil 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have mail.
Last login: Sat Aug 15 07:37:32 2026 from ::1
(kali@olivil)-[~]
$ whoami

```

The SSH server was configured to allow public-key authentication and disable password-based SSH authentication. The configuration was successfully validated using `sshd -t` with no errors. The SSH service restarted successfully and remained active and running. A new SSH session was then established successfully using the configured SSH key, and `whoami` confirmed that the authenticated user was `kali`.

Part 2: Disable Root Login

Commands used:

```
sudo nano /etc/ssh/sshd_config
```

Changed:

```
PermitRootLogin no
```

Then:

```
CRL + O = SAVE
```

```
CRL + X = EXIT
```

Configuration tested with:

```
sudo sshd -t
```

SSH restarted with:

```
sudo systemctl restart ssh
```

SSH service verified with:

```
sudo systemctl status ssh --no-pager
```

Normal key-based login tested with:

```
ssh kali@localhost
```

```
whoami
```

Root login tested with:

```
ssh root@localhost
```

```
kali-linux-2025.4-vmware-a...
kali@olivil:
Session Actions Edit View Help
(kali@olivil)-[~]
$ sudo nano /etc/ssh/sshd_config
[sudo] password for kali:
(kali@olivil)-[~]
$ sudo sshd -t
(kali@olivil)-[~]
$ sudo systemctl restart ssh
(kali@olivil)-[~]
$ sudo systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-15 08:41:17 EDT; 17s ago
     Invocation: 0c6f71a8147847dc91aa953708c886fa
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 50380 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 50391 (sshd)
      Tasks: 1 (limit: 2066)
     Memory: 1.4M (peak: 2.8M)
        CPU: 53ms
    CGroup: /system.slice/ssh.service
            └─50391 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 15 08:41:17 olivil systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Aug 15 08:41:17 olivil sshd[50391]: Server listening on 0.0.0.0 port 22.
Aug 15 08:41:17 olivil systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Aug 15 08:41:17 olivil sshd[50391]: Server listening on :: port 22.
(kali@olivil)-[~]
$ sudo sshd -t
```

```
(kali@olivil)-[~]
$ ssh kali@localhost
Linux olivil 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have mail.
Last login: Sat Aug 15 07:58:35 2026 from ::1
(kali@olivil)-[~]
$ whoami
kali
(kali@olivil)-[~]
$ exit
Connection to localhost closed.

(kali@olivil)-[~]
$ ssh root@localhost
root@localhost's password:
Permission denied, please try again.
root@localhost's password: 
```

SSH was configured to prevent direct root login by setting PermitRootLogin no. The configuration passed the SSH syntax check, and the SSH service restarted successfully. Key-based authentication for the kali user continued to work normally. A direct SSH attempt as root returned Permission denied, confirming that root SSH access was successfully disabled.

Configuring User Account Security with login.defs

Scenario

You are a system administrator for CyberShield Inc., a company that handles sensitive client data. To comply with security policies, you are Exercised with strengthening user account security by configuring parameters in the login.defs

Your goal is to ensure that all user accounts adhere to these policies for better security management.

Step 1 : Explore login.defs

1. Check the existing password, UID, and GID settings

```
grep -E  
'^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE|UID_MIN|UID_MAX|GID_MIN|GID_MAX)' /etc/login.defs
```

2. Edit the login.defs configuration

```
sudo nano /etc/login.defs
```

Configured:

FROM:

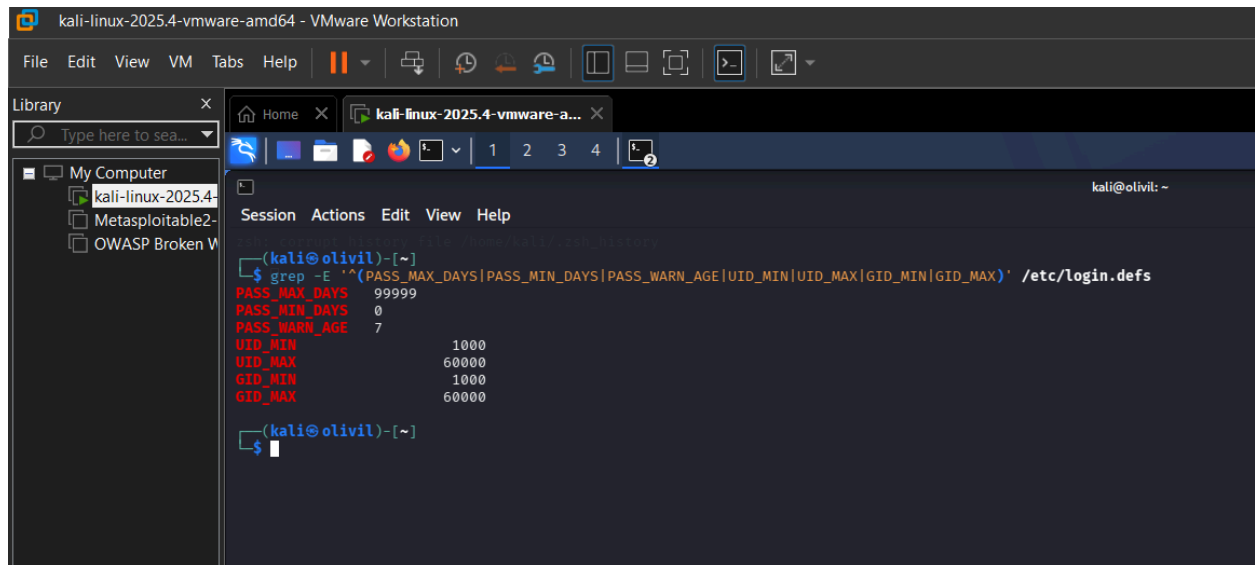
```
PASS_MAX_DAYS 99999  
PASS_MIN_DAYS 0  
PASS_WARN_AGE 7  
UID_MIN 1000  
UID_MAX 60000  
GID_MIN 1000  
GID_MAX 60000
```

TO:

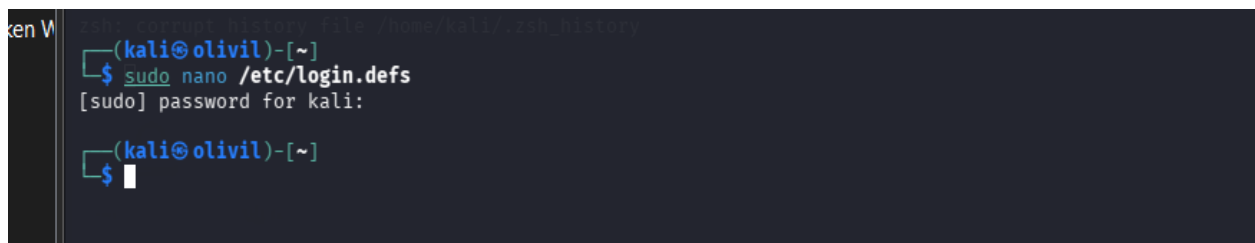
```
PASS_MAX_DAYS 90  
PASS_MIN_DAYS 7  
PASS_WARN_AGE 14  
UID_MIN 1000  
UID_MAX 60000  
GID_MIN 1000  
GID_MAX 60000
```

CTRL + O = ENTER

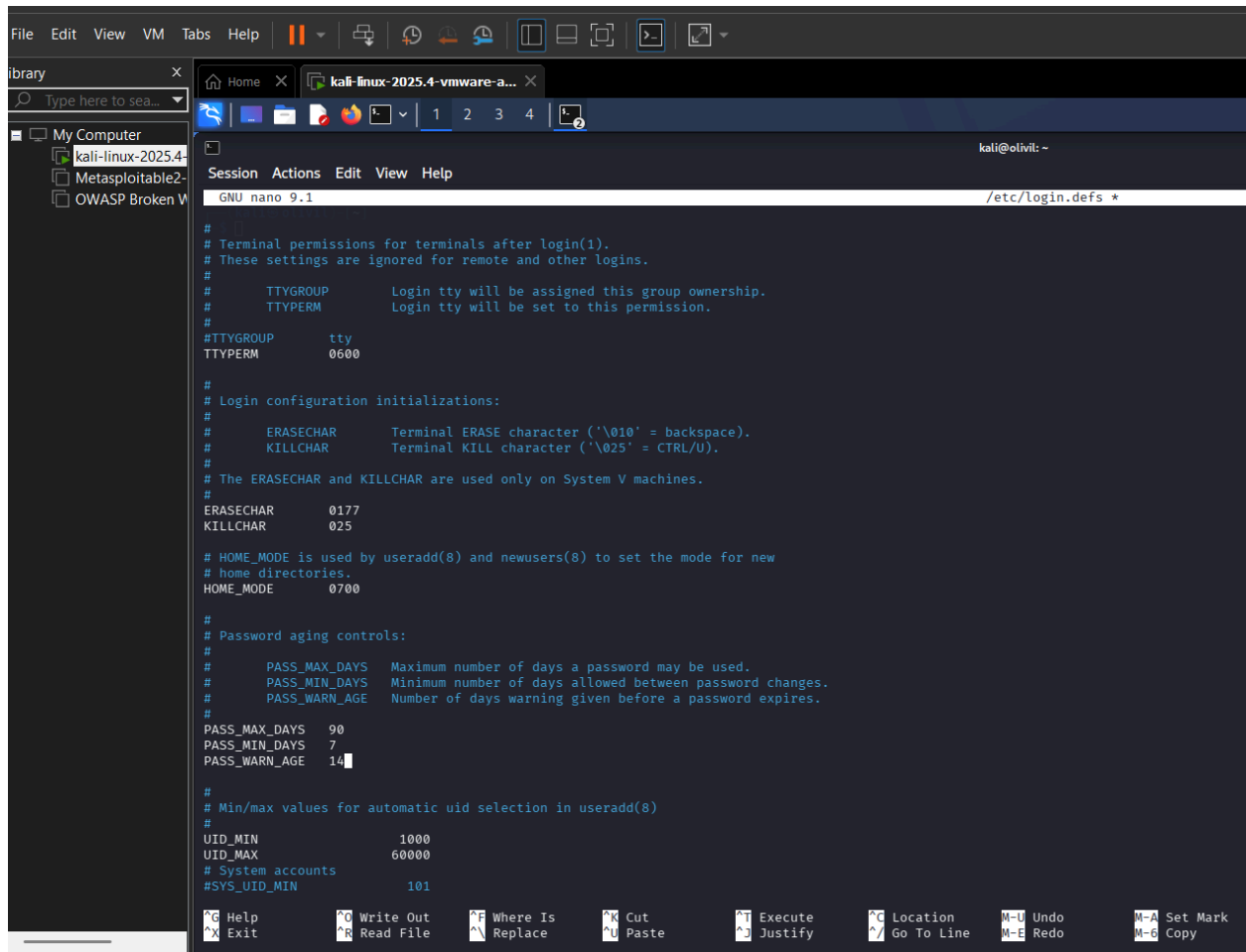
CTRL + X = EXIT FILE



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken W
kali@olivil: ~
Session Actions Edit View Help
(kali@olivil)-[~]
$ grep -E '^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE|UID_MIN|UID_MAX|GID_MIN|GID_MAX)' /etc/login.defs
PASS_MAX_DAYS 99999
PASS_MIN_DAYS 0
PASS_WARN_AGE 7
UID_MIN 1000
UID_MAX 60000
GID_MIN 1000
GID_MAX 60000
(kali@olivil)-[~]
$
```



```
ken W
(kali@olivil)-[~]
$ sudo nano /etc/login.defs
[sudo] password for kali:
(kali@olivil)-[~]
$
```



The screenshot shows a Kali Linux virtual machine environment. A terminal window is open, displaying the contents of the `/etc/login.defs` file. The file is being edited using the `nano` text editor. The terminal shows the following configuration:

```
# Terminal permissions for terminals after login(1).
# These settings are ignored for remote and other logins.
#
#       TTYGROUP      Login tty will be assigned this group ownership.
#       TTYPERM       Login tty will be set to this permission.
#
#TTYGROUP      tty
#TTYPERM       0600

#
# Login configuration initializations:
#
#       ERASECHAR      Terminal ERASE character ('\010' = backspace).
#       KILLCHAR       Terminal KILL character ('\025' = CTRL/U).
#
# The ERASECHAR and KILLCHAR are used only on System V machines.
#
#ERASECHAR     0177
#KILLCHAR      025

# HOME_MODE is used by useradd(8) and newusers(8) to set the mode for new
# home directories.
#HOME_MODE     0700

#
# Password aging controls:
#
#       PASS_MAX_DAYS  Maximum number of days a password may be used.
#       PASS_MIN_DAYS  Minimum number of days allowed between password changes.
#       PASS_WARN_AGE  Number of days warning given before a password expires.
#
#PASS_MAX_DAYS 90
#PASS_MIN_DAYS 7
#PASS_WARN_AGE 14

#
# Min/max values for automatic uid selection in useradd(8)
#
#UID_MIN       1000
#UID_MAX       60000
# System accounts
#SYS_UID_MIN   101
```

The terminal window also shows a sidebar with a file explorer and a bottom status bar with various keyboard shortcuts.

I first reviewed the existing login.defs settings to establish the current account-security configuration. I then opened the file and updated the password policy to require a maximum password age of 90 days, a minimum age of 7 days, and a 14-day warning period. The existing UID and GID ranges already matched the required security policy, so they were left unchanged.

Step 2: Tested User and Applied Password Policy

Commands used:

1. Verify the updated login.defs settings

```
grep -E  
'^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE|UID_MIN|UID_MAX|GID_MIN|GID_MAX)' /etc/login.defs
```

2. Attempt to create the test user

```
sudo useradd testuser (which user already exist from previous labs)
```

3. Verified the existing testuser account

```
id testuser
```

4. Checked the user's original password-aging settings

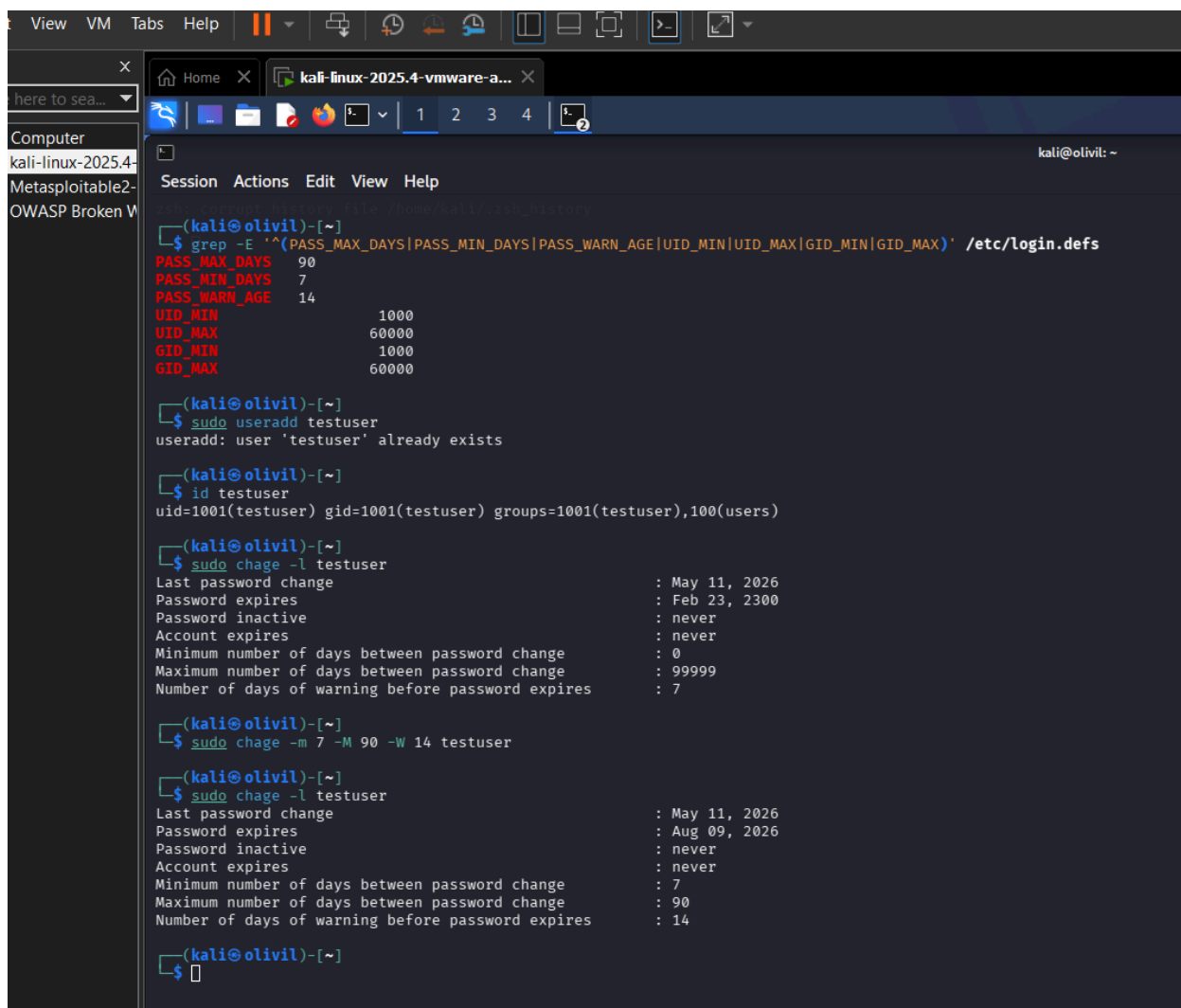
```
sudo chage -l testuser
```

5. Applied the required password policy

```
sudo chage -m 7 -M 90 -W 14 testuser
```

6. Verified the updated password-aging settings

```
sudo chage -l testuser
```



```
(kali@olivil)-[~]
$ grep -E '^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE|UID_MIN|UID_MAX|GID_MIN|GID_MAX)' /etc/login.defs
PASS_MAX_DAYS    90
PASS_MIN_DAYS    7
PASS_WARN_AGE    14
UID_MIN          1000
UID_MAX          60000
GID_MIN          1000
GID_MAX          60000

(kali@olivil)-[~]
$ sudo useradd testuser
useradd: user 'testuser' already exists

(kali@olivil)-[~]
$ id testuser
uid=1001(testuser) gid=1001(testuser) groups=1001(testuser),100(users)

(kali@olivil)-[~]
$ sudo chage -l testuser
Last password change                : May 11, 2026
Password expires                    : Feb 23, 2300
Password inactive                    : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7

(kali@olivil)-[~]
$ sudo chage -m 7 -M 90 -W 14 testuser

(kali@olivil)-[~]
$ sudo chage -l testuser
Last password change                : May 11, 2026
Password expires                    : Aug 09, 2026
Password inactive                    : never
Account expires                     : never
Minimum number of days between password change : 7
Maximum number of days between password change : 90
Number of days of warning before password expires : 14

(kali@olivil)-[~]
$
```

verified that the required login.defs settings were correctly configured. The testuser account already existed, so instead of creating a duplicate account, I checked its existing settings and applied the required password policy using **chage**. The final verification confirmed a 7-day minimum password age, 90-day maximum password age, and 14-day warning period.

Step 3: Set and Test the testuser Password

Commands used:

1. Set a password for testuser

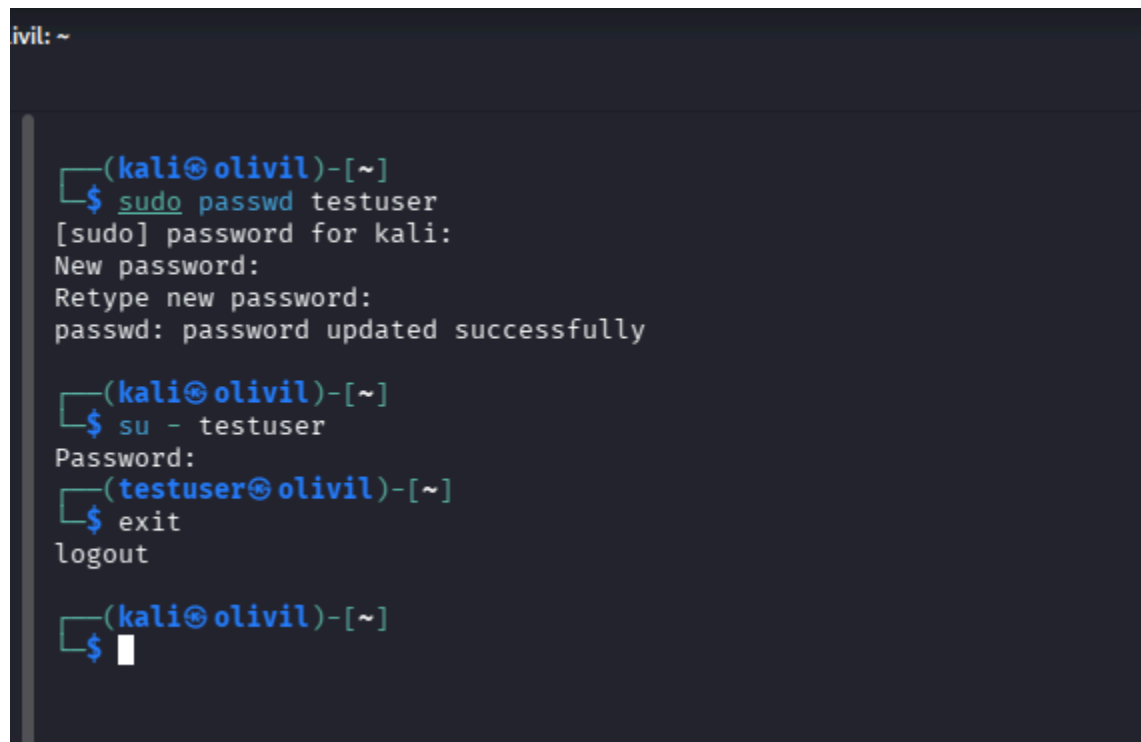
`sudo passwd testuser`

2. Switch to the testuser account

`su - testuser`

3. Exit the testuser session

`exit`



```
ivilt: ~  
  
(kali@olivil)-[~]  
$ sudo passwd testuser  
[sudo] password for kali:  
New password:  
Retype new password:  
passwd: password updated successfully  
  
(kali@olivil)-[~]  
$ su - testuser  
Password:  
(testuser@olivil)-[~]  
$ exit  
logout  
  
(kali@olivil)-[~]  
$
```

successfully set a password for the testuser account and verified that the account could be accessed by switching to it with `su - testuser`. After confirming successful login, I exited the test account and returned to the normal kali user session.

Configuring System Auditing with auditd

Scenario

You are the security administrator for SecureNet Corp., a company dealing with sensitive data. To improve security and comply with regulatory standards, you are Exercised with setting up system auditing to track critical events. You will use the

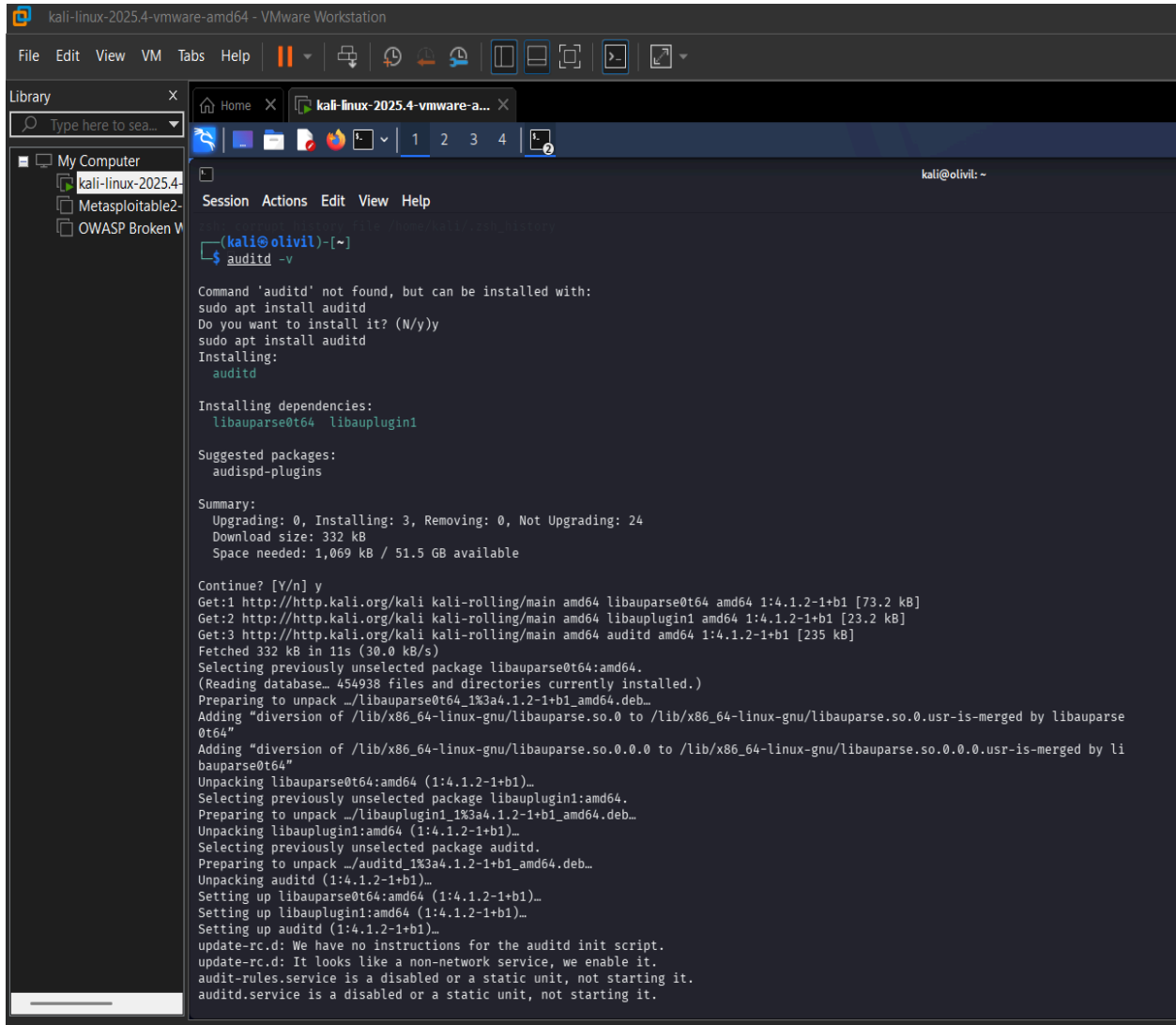
auditd

Your goal is to configure
auditd

Part 1: Install and Configure auditd

Commands used

```
sudo apt install auditd
sudo systemctl start auditd
sudo systemctl status auditd --no-pager
sudo systemctl enable auditd
sudo systemctl is-active auditd
sudo systemctl is-enabled auditd
```



```
ware-a... X
2 3 4 |
kali@olivil: ~

(kali@olivil)-[~]
$ auditd -v
auditd: invalid option -- 'v'
Usage: auditd [-f] [-l] [-n] [-s disable|enable|nochange] [-c <config_file>]

(kali@olivil)-[~]
$ sudo systemctl start auditd
[sudo] password for kali:

(kali@olivil)-[~]
$ sudo systemctl status auditd --no-pager
● auditd.service - Security Audit Logging Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-08-15 10:10:04 EDT; 15s ago
     Invocation: 2e29a89a94294c97b53cab0e6a563177
       Docs: man:auditd(8)
             https://github.com/linux-audit/audit-documentation
   Process: 36817 ExecStart=/usr/sbin/auditd (code=exited, status=0/SUCCESS)
    Main PID: 36818 (auditd)
       Tasks: 2 (limit: 2066)
      Memory: 912K (peak: 2.5M)
         CPU: 26ms
    CGroup: /system.slice/auditd.service
            └─36818 /usr/sbin/auditd

Aug 15 10:10:04 olivil systemd[1]: Starting auditd.service - Security Audit...
Aug 15 10:10:04 olivil auditd[36818]: No plugins found, not dispatching events
Aug 15 10:10:04 olivil auditd[36818]: Init complete, auditd 4.1.2 listening...
Aug 15 10:10:04 olivil systemd[1]: Started auditd.service - Security Audit...
Hint: Some lines were ellipsized, use -l to show in full.

(kali@olivil)-[~]
$ sudo systemctl enable auditd
Created symlink '/etc/systemd/system/multi-user.target.wants/auditd.service' → '/usr/lib/systemd/system/auditd.service'.

(kali@olivil)-[~]
$ sudo systemctl is-active auditd
active

(kali@olivil)-[~]
$ sudo systemctl is-enabled auditd
enabled

(kali@olivil)-[~]
```

auditd was installed successfully and the audit service was started. The service status showed that it was running, and it was enabled to start automatically when Kali boots.

Part 2 Configure Audit Rules

View existing rules

```
sudo auditctl -l
```

Output: No rules

Open audit rules file

```
sudo nano /etc/audit/rules.d/audit.rules
```

Rules added

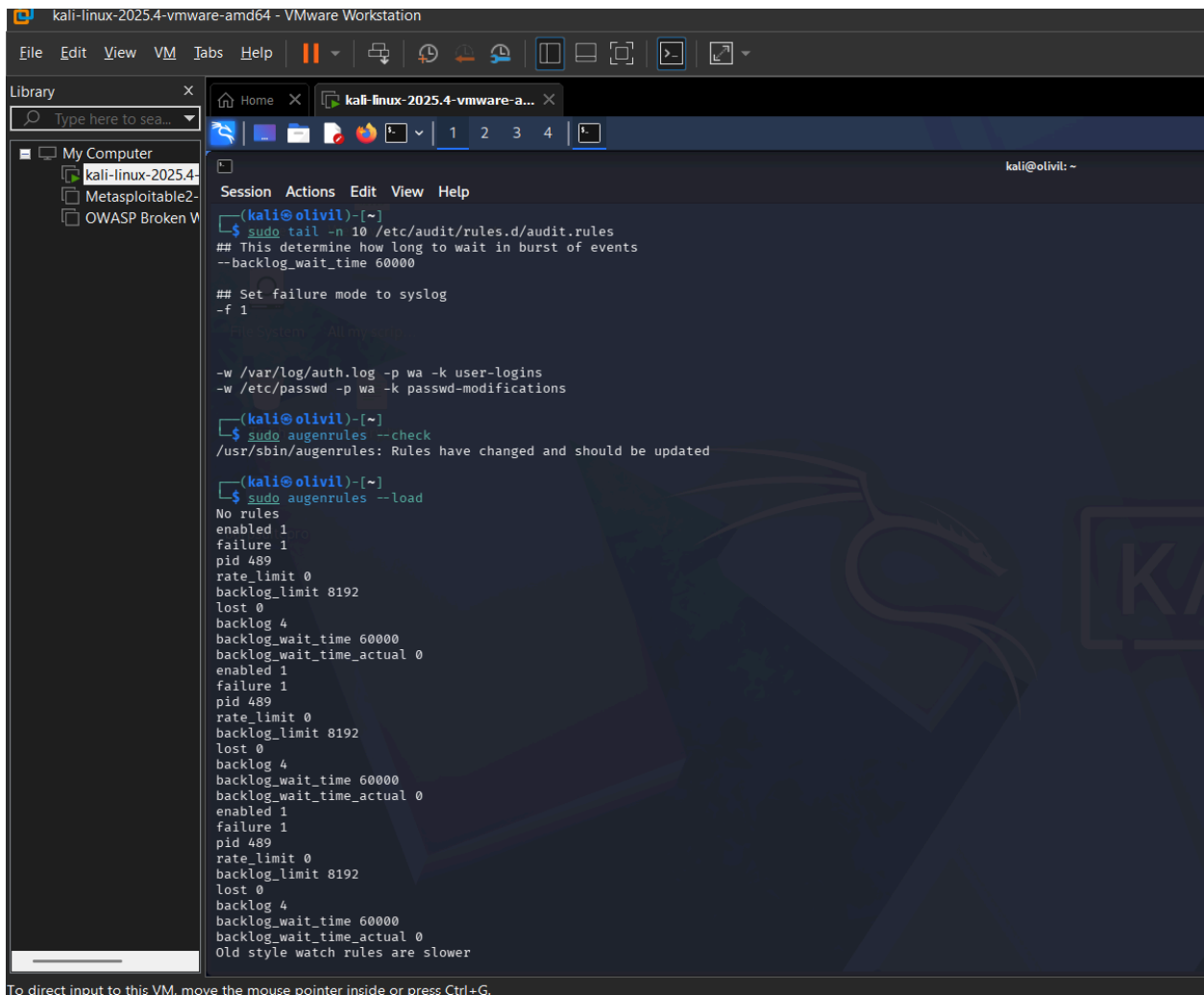
```
-w /var/log/auth.log -p wa -k user-logins  
-w /etc/passwd -p wa -k passwd-modifications
```

Apply the rules

```
sudo augenrules --load
```

Verify loaded rules

```
sudo auditctl -l
```

```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken V
kali@olivil: ~
$ sudo tail -n 10 /etc/audit/rules.d/audit.rules
## This determine how long to wait in burst of events
--backlog_wait_time 60000

## Set failure mode to syslog
-f 1

-w /var/log/auth.log -p wa -k user-logins
-w /etc/passwd -p wa -k passwd-modifications

(kali@olivil)-[~]
$ sudo augenrules --check
/usr/sbin/augenrules: Rules have changed and should be updated

(kali@olivil)-[~]
$ sudo augenrules --load
No rules
enabled 1
failure 1
pid 489
rate_limit 0
backlog_limit 8192
lost 0
backlog 4
backlog_wait_time 60000
backlog_wait_time_actual 0
enabled 1
failure 1
pid 489
rate_limit 0
backlog_limit 8192
lost 0
backlog 4
backlog_wait_time 60000
backlog_wait_time_actual 0
enabled 1
failure 1
pid 489
rate_limit 0
backlog_limit 8192
lost 0
backlog 4
backlog_wait_time 60000
backlog_wait_time_actual 0
Old style watch rules are slower
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

The existing audit rules were checked and no rules were initially loaded. Two rules were then added to monitor authentication logs and modifications to /etc/passwd. The rules were successfully loaded and verified with auditctl.

Part 3 Generate and Check Audit Logs

Commands Used:

1. Test user login

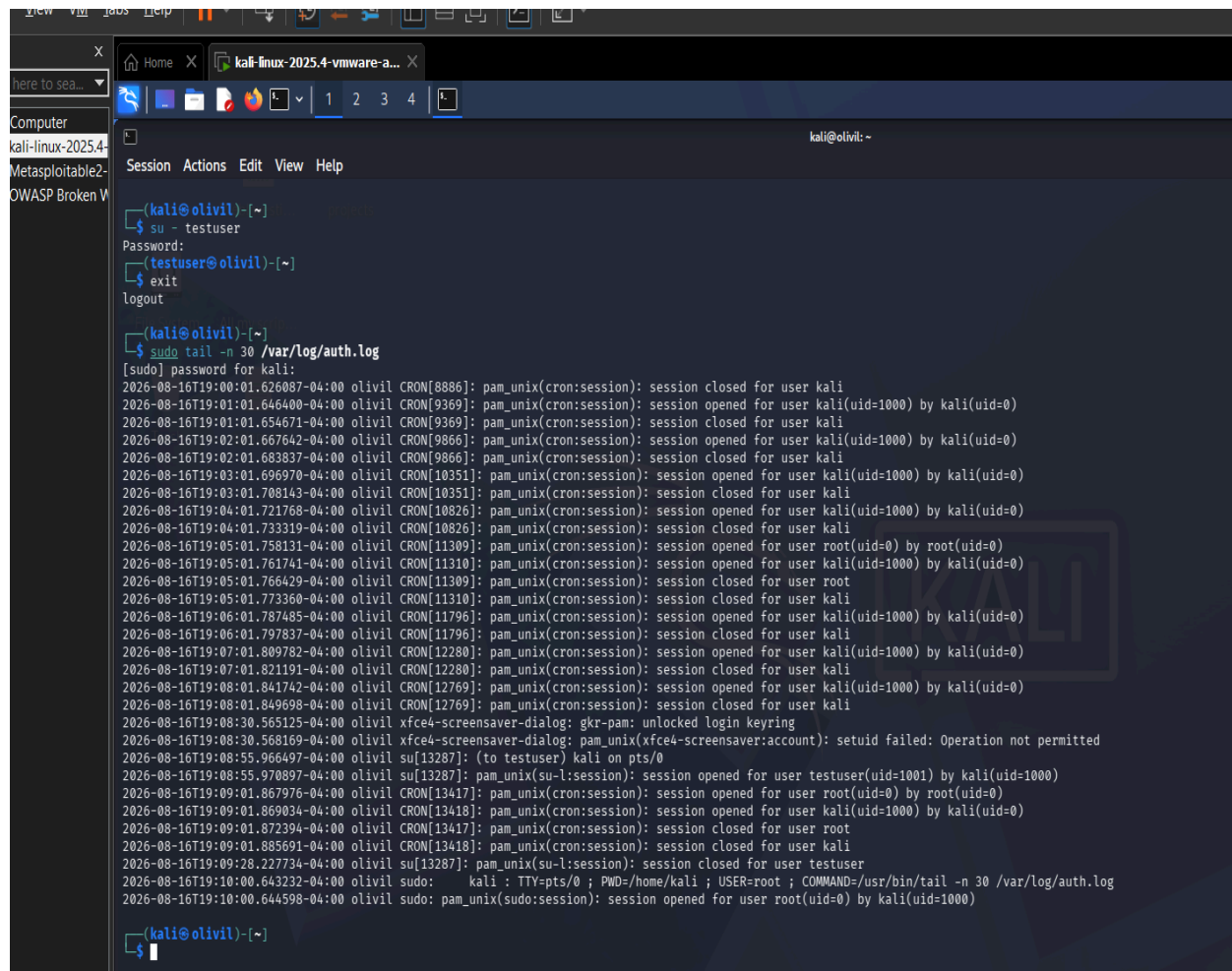
```
su - testuser
```

Then:

```
exit
```

2. Check authentication activity

```
sudo tail -n 30 /var/log/auth.log
```



```
kali@olivil: ~  
$ su - testuser  
Password:  
kali@olivil: ~  
$ exit  
logout  
kali@olivil: ~  
$ sudo tail -n 30 /var/log/auth.log  
[sudo] password for kali:  
2026-08-16T19:00:01.626087-04:00 olivil CRON[8886]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:01:01.646400-04:00 olivil CRON[9369]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:01:01.654671-04:00 olivil CRON[9369]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:02:01.667642-04:00 olivil CRON[9866]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:02:01.683837-04:00 olivil CRON[9866]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:03:01.696970-04:00 olivil CRON[10351]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:03:01.708143-04:00 olivil CRON[10351]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:04:01.721768-04:00 olivil CRON[10826]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:04:01.733319-04:00 olivil CRON[10826]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:05:01.758131-04:00 olivil CRON[11309]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)  
2026-08-16T19:05:01.761741-04:00 olivil CRON[11310]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:05:01.766429-04:00 olivil CRON[11309]: pam_unix(cron:session): session closed for user root  
2026-08-16T19:05:01.773360-04:00 olivil CRON[11310]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:06:01.787485-04:00 olivil CRON[11796]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:06:01.797837-04:00 olivil CRON[11796]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:07:01.809782-04:00 olivil CRON[12280]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:07:01.821191-04:00 olivil CRON[12280]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:08:01.841742-04:00 olivil CRON[12769]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:08:01.849698-04:00 olivil CRON[12769]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:08:30.565125-04:00 olivil xfce4-screensaver-dialog: gkr-pam: unlocked login keyring  
2026-08-16T19:08:30.568169-04:00 olivil xfce4-screensaver-dialog: pam_unix(xfce4-screensaver:account): setuid failed: Operation not permitted  
2026-08-16T19:08:55.966497-04:00 olivil su[13287]: (to testuser) kali on pts/0  
2026-08-16T19:08:55.970897-04:00 olivil su[13287]: pam_unix(su-l:session): session opened for user testuser(uid=1001) by kali(uid=1000)  
2026-08-16T19:09:01.867976-04:00 olivil CRON[13417]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)  
2026-08-16T19:09:01.869034-04:00 olivil CRON[13418]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)  
2026-08-16T19:09:01.872394-04:00 olivil CRON[13417]: pam_unix(cron:session): session closed for user root  
2026-08-16T19:09:01.885691-04:00 olivil CRON[13418]: pam_unix(cron:session): session closed for user kali  
2026-08-16T19:09:28.227734-04:00 olivil su[13287]: pam_unix(su-l:session): session closed for user testuser  
2026-08-16T19:10:00.643232-04:00 olivil sudo: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/tail -n 30 /var/log/auth.log  
2026-08-16T19:10:00.644598-04:00 olivil sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)  
kali@olivil: ~  
$
```

This showed both the failed and successful testuser authentication/session activity

3. Add the INTEL305 test rule

Added the temporary audit rule:

```
sudo auditctl -w /tmp/intel305-test -p wa -k intel305-test
```

The system displayed:

old style watch rules are slower

This was a warning, not an error.

4. Generate an audit event

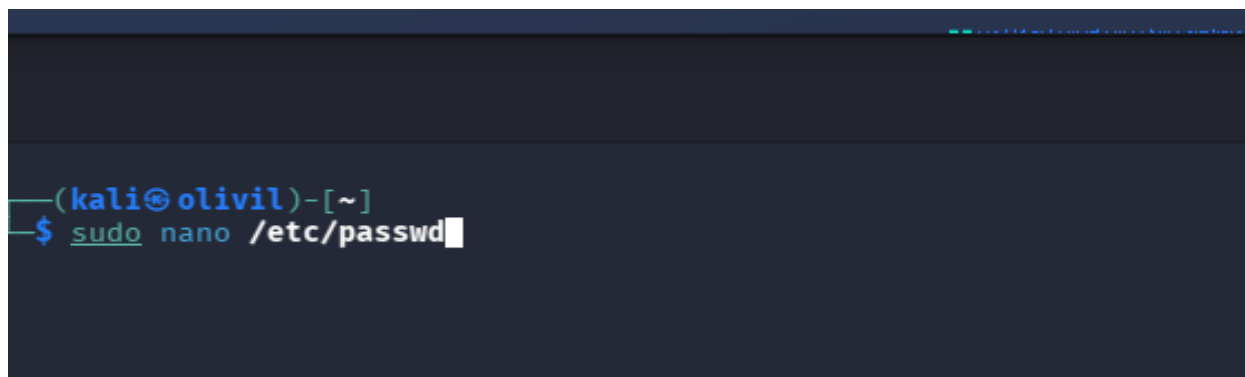
```
sudo touch /tmp/intel305-test
```

5. Search for the INTEL305 audit event

```
sudo ausearch -k intel305-test
```

6. Remove the temporary INTEL305 rule

```
sudo auditctl -d -w /tmp/intel305-test -p wa -k intel305-test
```



```
14-
s2-
h V
kali
Session Actions Edit View Help
GNU nano 9.1 /etc/passwd *
pipewire:x:987:987:system user for pipewire:/nonexistent:/usr/sbin/nologin
lightdm:x:109:110:Light Display Manager:/var/lib/lightdm:/bin/false
statd:x:110:65534::/var/lib/nfs:/usr/sbin/nologin
saned:x:111:111::/var/lib/saned:/usr/sbin/nologin
polkitd:x:986:986:User for polkitd:/usr/sbin/nologin
rtkit:x:112:112:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:985:985:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
pcscd:x:984:984:PC/SC Smart Card Daemon:/usr/sbin/nologin
mysql:x:113:114:MariaDB Server:/nonexistent:/bin/false
geoclue:x:114:115::/var/lib/geoclue:/usr/sbin/nologin
Debian-snmpp:x:115:116::/var/lib/snmpp:/bin/false
sshd:x:116:117::/nonexistent:/usr/sbin/nologin
cups-pk-helper:x:117:120:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
redsocks:x:118:121::/var/run/redsocks:/usr/sbin/nologin
_gophish:x:119:123::/var/lib/gophish:/usr/sbin/nologin
iodine:x:120:65534::/run/iodine:/usr/sbin/nologin
miredo:x:121:65534::/var/run/miredo:/usr/sbin/nologin
redis:x:122:124::/var/lib/redis:/usr/sbin/nologin
postgres:x:123:125:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
mosquitto:x:124:126::/var/lib/mosquitto:/usr/sbin/nologin
inetsim:x:125:127::/var/lib/inetsim:/usr/sbin/nologin
_gvm:x:126:128::/var/lib/openvas:/usr/sbin/nologin
kali:x:1000:1000::/home/kali:/usr/bin/zsh
snort:x:127:131:Snort IDS:/var/log/snort:/usr/sbin/nologin
_dhcpd:x:980:980:DHCP Client Daemon:/var/lib/dhcpd:/sbin/nologin
iperf3:x:128:133::/nonexistent:/usr/sbin/nologin
testuser:x:1001:1001::1,3,4,0:/home/testuser:/bin/bash
Debian-exim:x:129:134::/var/spool/exim4:/usr/sbin/nologin
sbuidl:x:130:135:Debian source builder:/var/lib/sbuidl:/bin/bash
debian-tor:x:131:136::/var/lib/tor:/bin/false
stunnel:x:979:979:stunnel service system account:/var/run/stunnel:/usr/sbin/nologin
freerad:x:132:137::/etc/freeradius:/usr/sbin/nologin
student1:x:1002:1002::,/home/student1:/bin/bash
alice:x:1004:1004::,/home/alice:/bin/bash
bob:x:1005:1005::,/home/bob:/bin/bash
charlie:x:1006:1006::,/home/charlie:/bin/bash
olivil:x:1008:1008::,/home/olivil:/bin/bash
john:x:1009:1009::/home/john:/bin/sh
mary:x:1010:1010::/home/mary:/bin/sh
paul:x:1011:1011::/home/paul:/bin/sh
Ahmed:x:1012:1012::/home/Ahmed:/bin/sh

# INTEL30D audit test

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
kali-linux-2025.4-vmware-a... X
kali@olivil: ~
Session Actions Edit View Help

(kali@olivil)-[~]
$ sudo auditctl -l
-w /var/log/auth.log -p wa -k user-logins
-w /etc/passwd -p wa -k passwd-modifications

(kali@olivil)-[~]
$ sudo auditctl -w /tmp/intel305-test -p wa -k intel305-test
old style watch rules are slower

(kali@olivil)-[~]
$ sudo touch /tmp/intel305-test

(kali@olivil)-[~]
$ sudo ausearch -k intel305-test

time→Sat Aug 15 15:12:27 2026
type=PROCTITLE msg=audit(1786821147.822:874): proctitle=617564697463746C002D77002F746D702F696E74656C3330352
D74657374002D70007761002D6B00696E74656C3330352D74657374
type=PATH msg=audit(1786821147.822:874): item=0 name="/tmp/" inode=1 dev=00:28 mode=041777 ouid=0 ogid=0 rd
ev=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1786821147.822:874): cwd="/home/kali"
type=SOCKADDR msg=audit(1786821147.822:874): saddr=10000000000000000000000000000000
type=SYSCALL msg=audit(1786821147.822:874): arch=c000003e syscall=44 success=yes exit=1088 a0=4 a1=7ffe1e9f
a4b0 a2=440 a3=0 items=1 ppid=35408 pid=35409 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsg
id=0 tty=pts2 ses=3 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1786821147.822:874): auid=1000 ses=3 subj=unconfined op=add_rule key="intel305
-test" list=4 res=1

time→Sat Aug 15 15:15:06 2026
type=PROCTITLE msg=audit(1786821306.888:909): proctitle=746F756368002F746D702F696E74656C3330352D74657374
type=PATH msg=audit(1786821306.888:909): item=1 name="/tmp/intel305-test" inode=383 dev=00:28 mode=0100644
ouid=0 ogid=0 rdev=00:00 nametype=CREATE cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(1786821306.888:909): item=0 name="/tmp/" inode=1 dev=00:28 mode=041777 ouid=0 ogid=0 rd
ev=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1786821306.888:909): cwd="/home/kali"
type=SYSCALL msg=audit(1786821306.888:909): arch=c000003e syscall=257 success=yes exit=3 a0=ffffffffffffff9
c a1=7ffc003b265b a2=941 a3=1b6 items=2 ppid=36734 pid=36735 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 eg
id=0 sgid=0 fsgid=0 tty=pts2 ses=3 comm="touch" exe="/usr/bin/touch" subj=unconfined key="intel305-test"

(kali@olivil)-[~]
$
```

A temporary INTEL305 audit rule was created to demonstrate file monitoring. A test file was accessed, and ausearch successfully retrieved the resulting audit event. This confirmed that the auditd service was functioning correctly and recording monitored file activity. The temporary test rule was then removed.

Part 4 Reboot and Verification

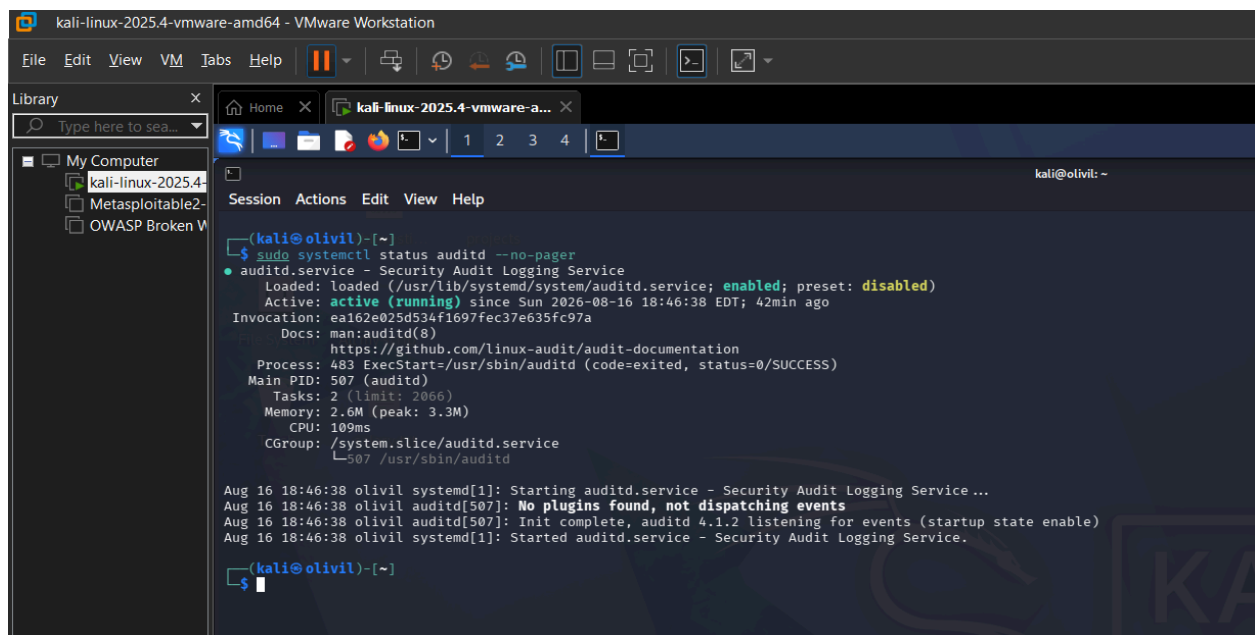
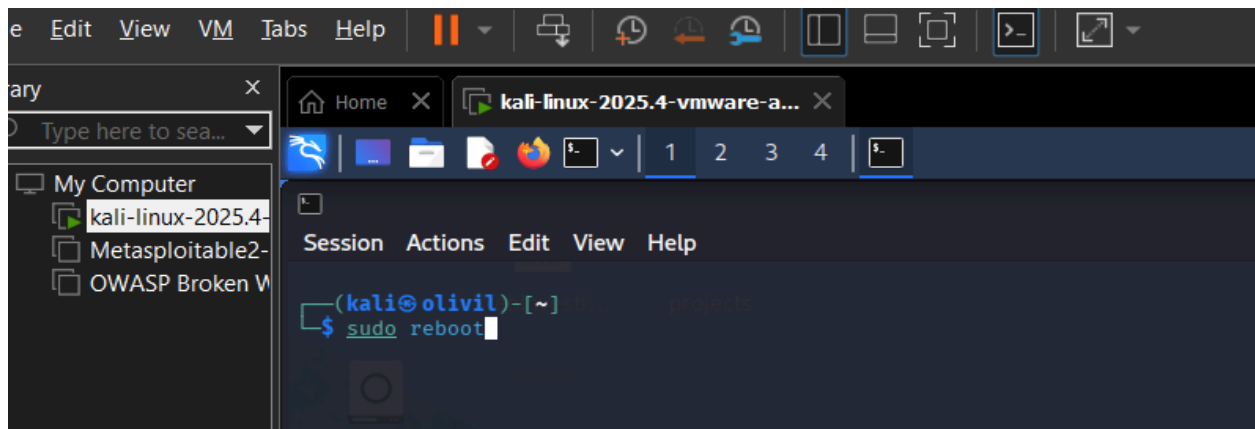
Commands Used:

Reboot

`sudo reboot`

After logging back into Kali

`sudo systemctl status auditd --no-pager`



The system was rebooted to verify persistence of the audit configuration. After restarting Kali, the `auditd` service was checked to confirm that it remained operational.

Lab 4: Advanced User Access Control and Security Auditing

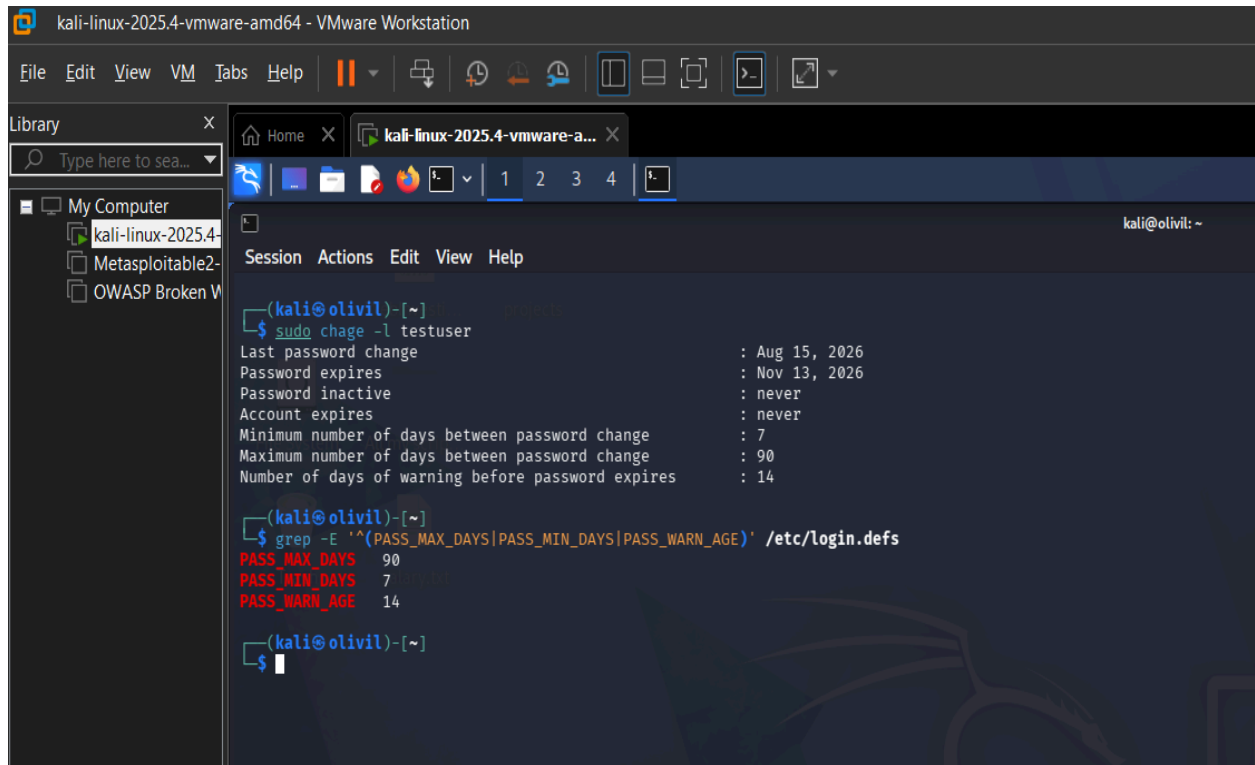
Part 1: Password Policy Configuration

Exercise 1.1: configure Password Aging

Commands used:

```
sudo chage -l testuser
```

```
grep -E '^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE)'  
/etc/login.defs
```



```
(kali@olivil)-[~]
$ sudo chage -l testuser
Last password change           : Aug 15, 2026
Password expires                : Nov 13, 2026
Password inactive               : never
Account expires                 : never
Minimum number of days between password change : 7
Maximum number of days between password change : 90
Number of days of warning before password expires : 14

(kali@olivil)-[~]
$ grep -E '(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE)' /etc/login.defs
PASS_MAX_DAYS 90
PASS_MIN_DAYS 7
PASS_WARN_AGE 14

(kali@olivil)-[~]
$
```

testuser was verified to have a 7-day minimum password age, 90-day maximum password age, and 14-day warning period. The system-wide /etc/login.defs settings matched these required values.

Question 1.1: Why is it important to set a minimum number of days between password changes?

A minimum password age prevents users from repeatedly changing their password in a short period just to bypass password-history rules and return to an old password. It helps ensure that passwords are kept for a reasonable period and improves account security.

Question 1.2: What are the security implications of setting PASS_MAX_DAYS too high or too low?

- Too high: A compromised or weak password may remain valid for a long time, increasing the risk of unauthorized access.
 - Too low: Users are forced to change passwords too frequently, which can lead to password fatigue and encourage users to choose weaker or predictable passwords.
-

Exercise 1.2: Implement Password Complexity Requirements

Commands used

```
sudo apt update
sudo apt install libpam-pwquality -y
sudo nano /etc/security/pwquality.conf
```

Added to `pwquality.conf`:

```
minlen = 12
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
difok = 3
maxrepeat = 2
usercheck = 1
```

Tested the password policy with:

```
sudo passwd testuser
```

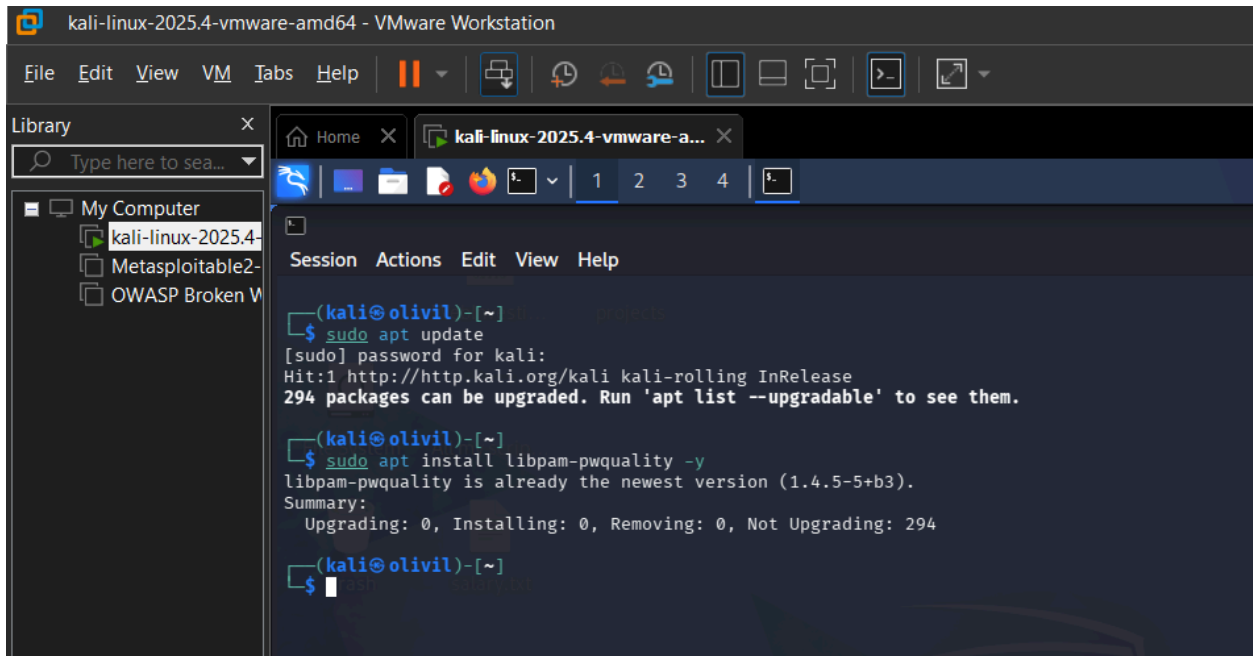
Weak password tested:

```
password
```

It was rejected with a dictionary-check warning.

A compliant password was then tested:

BlueSky@2026!

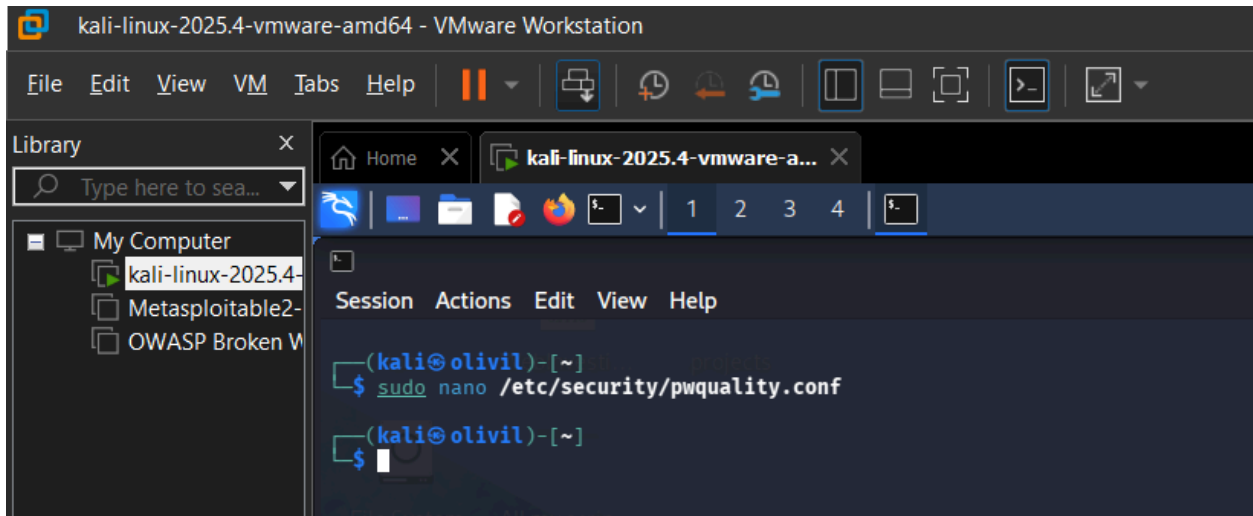


The screenshot shows a VMware Workstation window titled "kali-linux-2025.4-vmware-amd64 - VMware Workstation". The interface includes a menu bar (File, Edit, View, VM, Tabs, Help), a toolbar with icons for file operations and VM control, and a Library pane on the left. The Library pane shows a tree view under "My Computer" with items: "kali-linux-2025.4-", "Metasploitable2-", and "OWASP Broken V...". The main terminal window has a title bar "kali-linux-2025.4-vmware-a..." and a menu bar (Session, Actions, Edit, View, Help). The terminal output shows a user prompt "(kali@olivil)-[~]\$", followed by the command `sudo apt update`. The output indicates that 294 packages can be upgraded. The user then runs `sudo apt install libpam-pwquality -y`, and the output states that the package is already the newest version (1.4.5-5+b3). The summary shows 0 upgrades, 0 installations, 0 removals, and 294 packages not upgrading. The terminal ends with another user prompt "(kali@olivil)-[~]\$" and a cursor.

```
(kali@olivil)-[~] $ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
294 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@olivil)-[~] $ sudo apt install libpam-pwquality -y
libpam-pwquality is already the newest version (1.4.5-5+b3).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 294

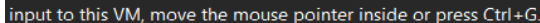
(kali@olivil)-[~] $
```

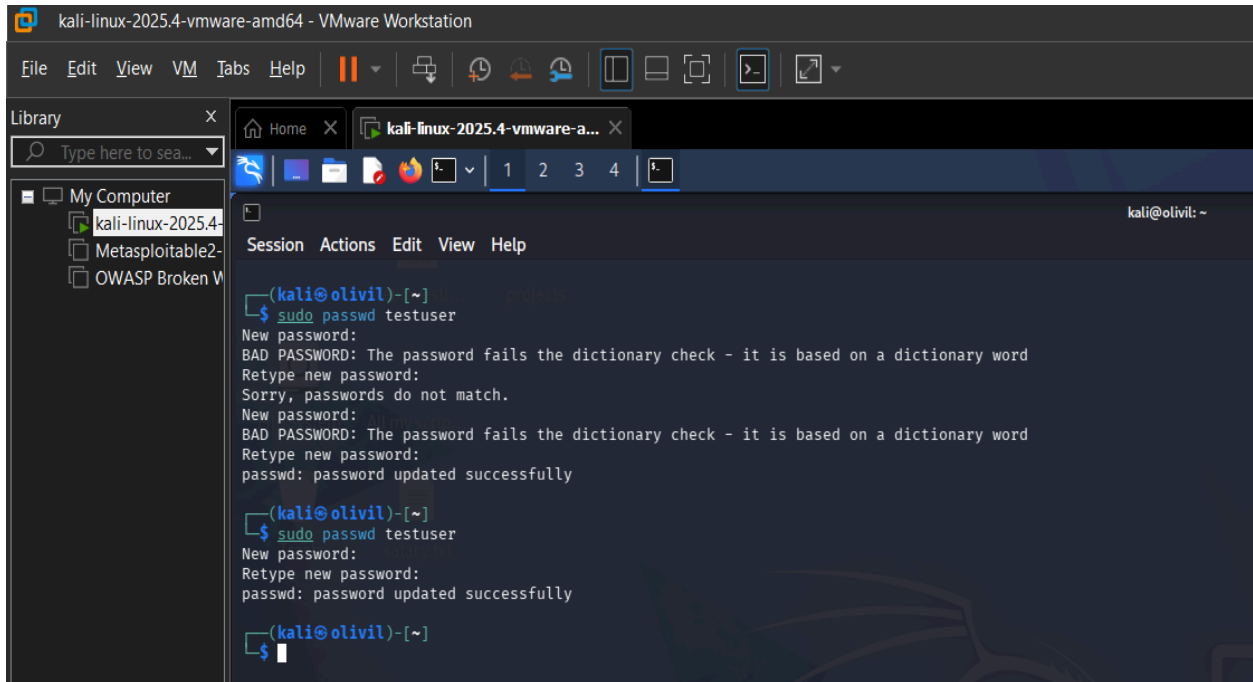


The screenshot shows the same VMware Workstation window. The Library pane is identical. The terminal window title bar is "kali-linux-2025.4-vmware-a...". The menu bar is the same. The terminal output shows the user prompt "(kali@olivil)-[~]\$", followed by the command `sudo nano /etc/security/pwquality.conf`. The terminal ends with another user prompt "(kali@olivil)-[~]\$" and a cursor.

```
(kali@olivil)-[~] $ sudo nano /etc/security/pwquality.conf

(kali@olivil)-[~] $
```



A screenshot of a Kali Linux virtual machine running in VMware Workstation. The terminal window shows a user attempting to set a password for the 'testuser' using the 'passwd' command. The first attempt is rejected because the password is a dictionary word. The second attempt is also rejected for the same reason. The third attempt is successful. The terminal output is as follows:

```
(kali@olivil)-[~]  
$ sudo passwd testuser  
New password:  
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word  
Retype new password:  
Sorry, passwords do not match.  
New password:  
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word  
Retype new password:  
passwd: password updated successfully  
  
(kali@olivil)-[~]  
$ sudo passwd testuser  
New password:  
Retype new password:  
passwd: password updated successfully  
  
(kali@olivil)-[~]  
$
```

I configured the password complexity requirements and tested the policy with both weak and strong passwords. The weak password was rejected, while the compliant password was accepted successfully.

Question 1.3: What happens when you try to set a password that doesn't meet the complexity requirements?

When a password does not meet the complexity requirements, the system rejects it and displays an error explaining that the password is too weak or fails a required check.

Question 1.4: How do password complexity requirements help prevent brute-force attacks?

Password complexity makes passwords harder to guess by requiring a combination of different character types and sufficient length. This increases the number of possible password combinations, making brute-force attacks more difficult.

Part 2: Account Security Auditing

Exercise 2.1: Identify Inactive User Accounts

Step 1 Identify user login activity

Command used:

```
sudo lastlog | grep -v "Never"
```

Issue encountered:

Kali returned:

Now Used:

```
last -a
```

I checked the available login history to identify active and inactive user accounts. The original `lastlog` command was unavailable on this Kali system, so I used `last -a` instead without making system changes.

Step 2 Check account activity and password aging

Command used:

```
sudo chage -l testuser
```

Result: testuser had its password changed on August 17, 2026, with a 90-day maximum password age and 14-day warning period.

Step 3 Check for accounts with empty passwords

Command used:

```
sudo awk -F: '($2 == "") {print $1}' /etc/shadow
```

Result: No output.

Step 4 Identify accounts with UID 0

Command used:

```
awk -F: '($3 == "0") {print $1}' /etc/passwd
```

Result:

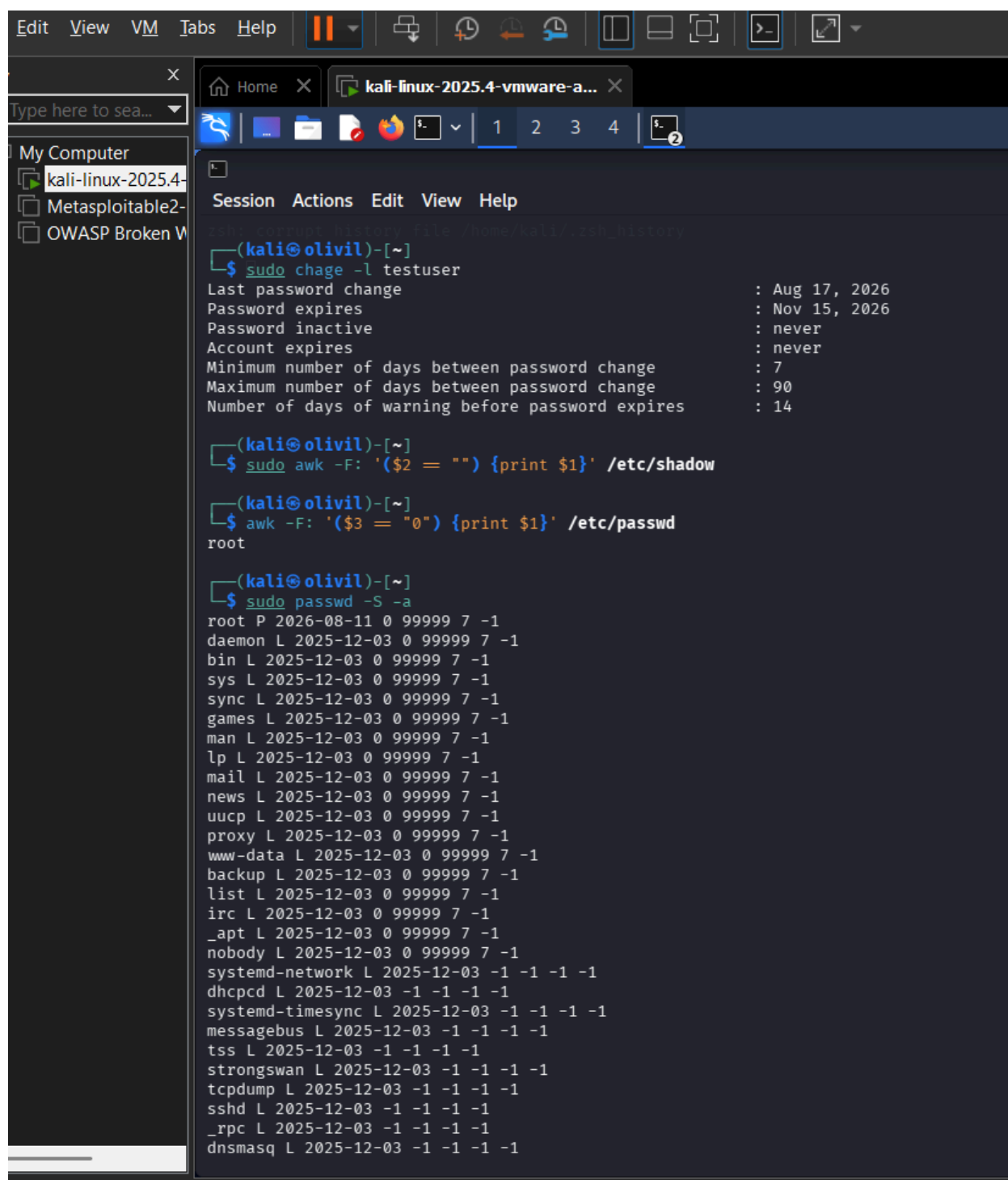
```
root
```

Step 5 Check password status of all users

Command used:

```
sudo passwd -S -a
```

Result: The command showed the password status of system and user accounts. Most system/service accounts were locked (L), while active accounts such as kali, testuser, student1, alice, bob, charlie, olivil, john, mary, and paul showed P



The screenshot shows a Kali Linux terminal window with the following commands and output:

```
ssh <script> history file /home/kali/.ssh_history
(kali@olivil)-[~]
$ sudo chage -l testuser
Last password change           : Aug 17, 2026
Password expires               : Nov 15, 2026
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 7
Maximum number of days between password change : 90
Number of days of warning before password expires : 14

(kali@olivil)-[~]
$ sudo awk -F: '($2 == "") {print $1}' /etc/shadow

(kali@olivil)-[~]
$ awk -F: '($3 == "0") {print $1}' /etc/passwd
root

(kali@olivil)-[~]
$ sudo passwd -S -a
root P 2026-08-11 0 99999 7 -1
daemon L 2025-12-03 0 99999 7 -1
bin L 2025-12-03 0 99999 7 -1
sys L 2025-12-03 0 99999 7 -1
sync L 2025-12-03 0 99999 7 -1
games L 2025-12-03 0 99999 7 -1
man L 2025-12-03 0 99999 7 -1
lp L 2025-12-03 0 99999 7 -1
mail L 2025-12-03 0 99999 7 -1
news L 2025-12-03 0 99999 7 -1
uucp L 2025-12-03 0 99999 7 -1
proxy L 2025-12-03 0 99999 7 -1
www-data L 2025-12-03 0 99999 7 -1
backup L 2025-12-03 0 99999 7 -1
list L 2025-12-03 0 99999 7 -1
irc L 2025-12-03 0 99999 7 -1
_apt L 2025-12-03 0 99999 7 -1
nobody L 2025-12-03 0 99999 7 -1
systemd-network L 2025-12-03 -1 -1 -1 -1
dhcpcd L 2025-12-03 -1 -1 -1 -1
systemd-timesync L 2025-12-03 -1 -1 -1 -1
messagebus L 2025-12-03 -1 -1 -1 -1
tss L 2025-12-03 -1 -1 -1 -1
strongswan L 2025-12-03 -1 -1 -1 -1
tcpdump L 2025-12-03 -1 -1 -1 -1
sshd L 2025-12-03 -1 -1 -1 -1
_rpc L 2025-12-03 -1 -1 -1 -1
dnsmasq L 2025-12-03 -1 -1 -1 -1
```

After reviewing the password status of all accounts and confirmed that most system accounts are locked while active user accounts have passwords configured. This helped identify which accounts are available for normal authentication.

Question 2.1: Why are inactive accounts a security risk?

Inactive accounts can become security vulnerabilities because attackers may exploit unused accounts to gain unauthorized access without being noticed. They also increase the number of accounts that need to be monitored and secured.

Question 2.2: What actions should be taken for accounts that haven't been used in 90+ days?

Review the account to confirm whether it is still needed. If it is no longer required, disable or remove it; if it is still needed, verify the owner and consider resetting the password and reviewing its permissions.

Exercise 2.2: Audit User Permissions and Group Memberships

Commands used

1. Check all groups for a specific user

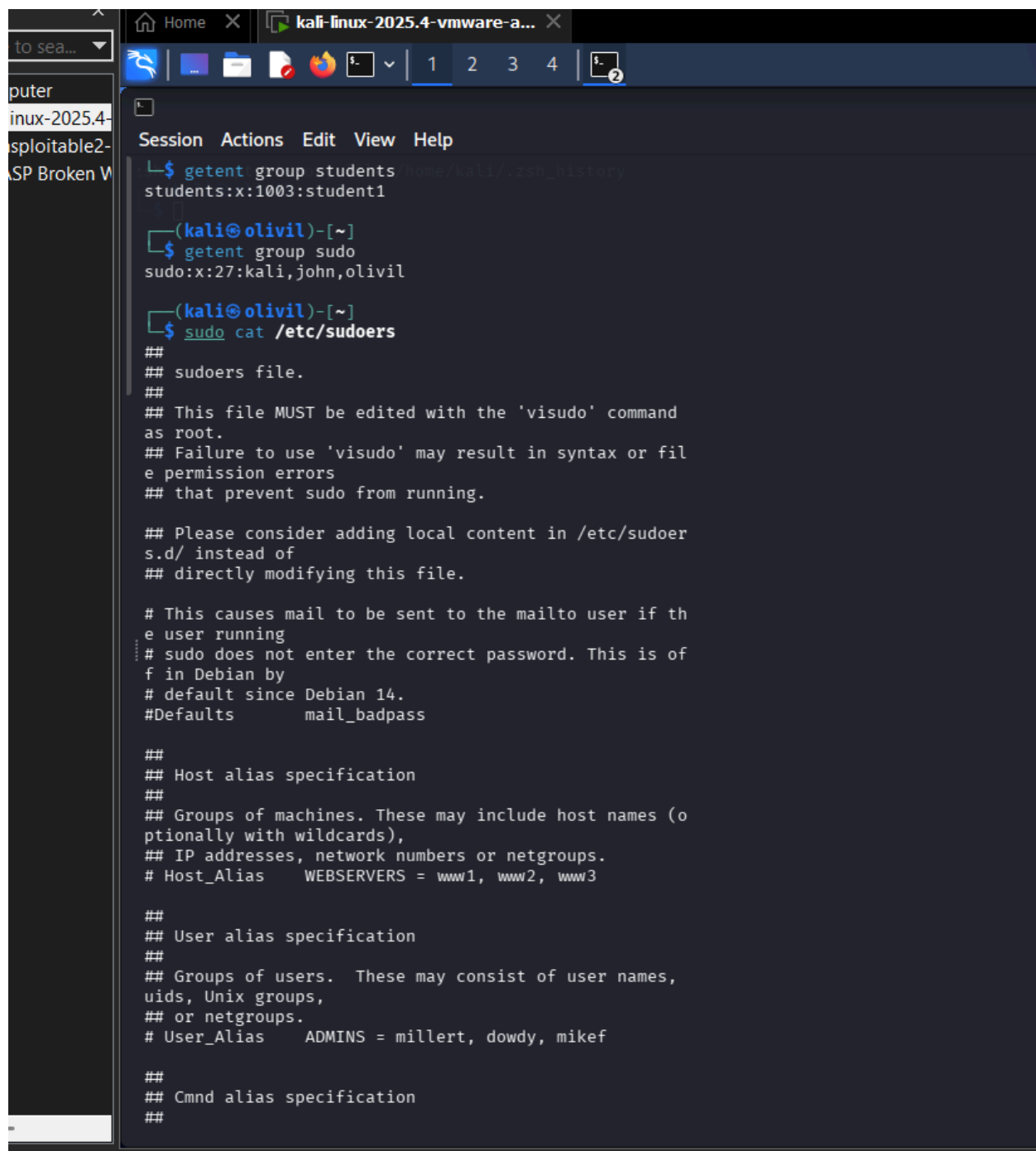
```
getent group students
```

2. Check the sudo group

```
getent group sudo
```

3. Review the sudoers file configuration

```
sudo cat /etc/sudoers
```



The screenshot shows a Kali Linux terminal window with the following commands and output:

```
Session Actions Edit View Help
└─$ getent group students home/kali/.ssh_history
students:x:1003:student1

(kali@olivil)-[~]
└─$ getent group sudo
sudo:x:27:kali,john,olivil

(kali@olivil)-[~]
└─$ sudo cat /etc/sudoers
##
## sudoers file.
##
## This file MUST be edited with the 'visudo' command
as root.
## Failure to use 'visudo' may result in syntax or fil
e permission errors
## that prevent sudo from running.

## Please consider adding local content in /etc/sudoer
s.d/ instead of
## directly modifying this file.

# This causes mail to be sent to the mailto user if th
e user running
# sudo does not enter the correct password. This is of
f in Debian by
# default since Debian 14.
#Defaults      mail_badpass

##
## Host alias specification
##
## Groups of machines. These may include host names (o
ptionally with wildcards),
## IP addresses, network numbers or netgroups.
# Host_Alias    WEBSERVERS = www1, www2, www3

##
## User alias specification
##
## Groups of users. These may consist of user names,
uids, Unix groups,
## or netgroups.
# User_Alias    ADMINS = millert, dowdy, mikef

##
## Cmnd alias specification
##
```

4. List users with valid login shells

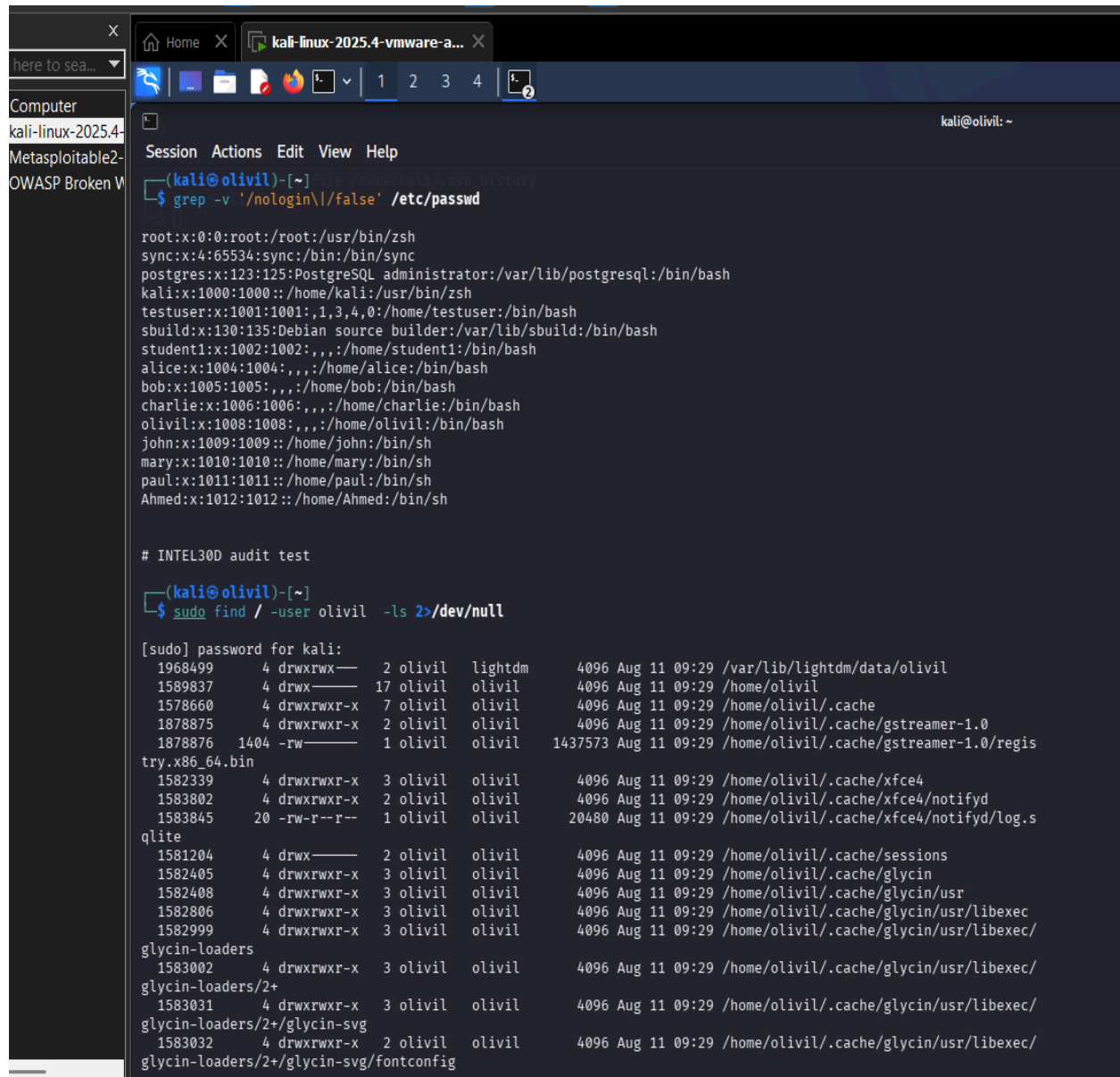
`grep -v '/nologin|/false' /etc/passwd`

5. Find files owned by specific users:

`sudo find / -user olivil -ls 2>/dev/null`

6. Find SUID files (potential security risk)

`sudo find / -perm -4000 -type f -ls 2>/dev/null`



```
(kali@olivil)-[~]
$ grep -v '/nologin|/false' /etc/passwd

root:x:0:0:root:/root:/usr/bin/zsh
sync:x:4:65534:sync:/bin:/bin/sync
postgres:x:123:125:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
kali:x:1000:1000::/home/kali:/usr/bin/zsh
testuser:x:1001:1001:,,,:/home/testuser:/bin/bash
sbuidl:x:130:135:Debian source builder:/var/lib/sbuidl:/bin/bash
student1:x:1002:1002:,,,:/home/student1:/bin/bash
alice:x:1004:1004:,,,:/home/alice:/bin/bash
bob:x:1005:1005:,,,:/home/bob:/bin/bash
charlie:x:1006:1006:,,,:/home/charlie:/bin/bash
olivil:x:1008:1008:,,,:/home/olivil:/bin/bash
john:x:1009:1009:/home/john:/bin/sh
mary:x:1010:1010:/home/mary:/bin/sh
paul:x:1011:1011:/home/paul:/bin/sh
Ahmed:x:1012:1012:/home/Ahmed:/bin/sh

# INTEL30D audit test

(kali@olivil)-[~]
$ sudo find / -user olivil -ls 2>/dev/null

[sudo] password for kali:
1968499      4 drwxrwx---  2 olivil  lightdm      4096 Aug 11 09:29 /var/lib/lightdm/data/olivil
1589837      4 drwx----- 17 olivil  olivil       4096 Aug 11 09:29 /home/olivil
1578660      4 drwxrwxr-x  7 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache
1878875      4 drwxrwxr-x  2 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/gstreamer-1.0
1878876     1404 -rw-----  1 olivil  olivil     1437573 Aug 11 09:29 /home/olivil/.cache/gstreamer-1.0/registries
1582339      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/xfce4
1583802      4 drwxrwxr-x  2 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/xfce4/notifd
1583845     20 -rw-r--r--  1 olivil  olivil     20480 Aug 11 09:29 /home/olivil/.cache/xfce4/notifd/log.s
qlite
1581204      4 drwx-----  2 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/sessions
1582405      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin
1582408      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin/usr
1582806      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin/usr/libexec
1582999      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin/usr/libexec/
glycin-loaders
1583002      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin/usr/libexec/
glycin-loaders/2+
1583031      4 drwxrwxr-x  3 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin/usr/libexec/
glycin-loaders/2+/glycin-svg
1583032      4 drwxrwxr-x  2 olivil  olivil       4096 Aug 11 09:29 /home/olivil/.cache/glycin/usr/libexec/
glycin-loaders/2+/glycin-svg/fontconfig
```

Reviewed user groups, sudo privileges, valid login accounts, files owned by olivil, and SUID files on the system. The checks helped identify privileged users and files that could require additional security monitoring. The audit checks were completed successfully.

Question 2.3: Why are SUID files a potential security concern?

SUID files run with the permissions of the file's owner, often **root**, even when executed by a normal user. If a vulnerable SUID program is exploited, an attacker could potentially gain elevated/root privileges.

Question 2.4: What is the principle of least privilege, and how does it apply to user management?

The principle of least privilege means users should receive only the permissions they need to perform their tasks. In user management, this means limiting administrative privileges, controlling group membership, and avoiding unnecessary access to sensitive files and systems.

Part 3: User Activity Monitoring

Exercise 3.1: Analyze Authentication Logs

Commands used

1. View successful login history

```
sudo last -a
```

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@olivil)-[~]
$ sudo last -a

[sudo] password for kali:
kali      tty7          Mon Aug 17 02:07 - still logged in      :0
lightdm   tty7          Mon Aug 17 02:06 - 02:07 (00:00)      :0
testuser  pts/0         Sun Aug 16 19:08 - 19:09 (00:00)      :0
kali      tty7          Sun Aug 16 18:47 - still logged in      :0
lightdm   tty7          Sun Aug 16 18:47 - 18:47 (00:00)      :0
kali      tty7          Sat Aug 15 21:48 - still logged in      :0
lightdm   tty7          Sat Aug 15 21:48 - 21:48 (00:00)      :0
testuser  pts/1         Sat Aug 15 14:20 - 14:20 (00:00)      :0
kali      tty7          Sat Aug 15 14:01 - still logged in      :0
lightdm   tty7          Sat Aug 15 14:01 - 14:01 (00:00)      :0
testuser  pts/2         Sat Aug 15 09:29 - 09:30 (00:01)      :0
kali      tty7          Sat Aug 15 09:02 - still logged in      :0
lightdm   tty7          Sat Aug 15 09:02 - 09:02 (00:00)      :0
kali      pts/5         Sat Aug 15 08:44 - 08:46 (00:01)      ::1
lightdm   tty8          Sat Aug 15 08:11 - 08:12 (00:00)      :1
kali      pts/4         Sat Aug 15 07:58 - 08:59 (01:00)      ::1
kali      pts/3         Sat Aug 15 07:37 - 07:38 (00:01)      ::1
kali      tty7          Sat Aug 15 07:06 - still logged in      :0
lightdm   tty7          Sat Aug 15 06:56 - 07:06 (00:10)      :0
kali      tty7          Tue Aug 11 09:50 - still logged in      :0
lightdm   tty7          Tue Aug 11 09:50 - 09:50 (00:00)      :0
kali      pts/0         Tue Aug 11 09:46 - still logged in      :0
olivil    tty7          Tue Aug 11 09:29 - still logged in      :0
lightdm   tty7          Tue Aug 11 09:29 - 09:29 (00:00)      :0
lightdm   tty7          Tue Aug 11 08:56 - still logged in      :0
olivil    console       Tue Aug 11 08:52 - 08:55 (00:03)      :0
lightdm   tty7          Tue Aug 11 08:33 - still logged in      :0
lightdm   tty7          Tue Aug 11 07:31 - still logged in      :0
lightdm   tty7          Mon Aug 10 21:04 - still logged in      :0
lightdm   tty7          Mon Aug 10 20:56 - still logged in      :0
lightdm   tty7          Mon Aug 10 20:36 - still logged in      :0
lightdm   tty7          Mon Aug 10 19:54 - still logged in      :0
lightdm   tty7          Mon Aug 10 19:46 - still logged in      :0
```

2. Attempt to view failed login history

`sudo last -b`

This did **not work on your Kali system** because your `last` utility uses `wtmpdb`, which does not support the `-b` option.

3. Search SSH logs for failed attempts

```
sudo journalctl _SYSTEMD_UNIT=ssh.service | grep -i "failed"
```

4. Search the system journal for authentication failures

```
sudo journalctl | grep -Ei "failed password|authentication failure|invalid user"
```

5. Monitor the authentication log in real time

```
sudo tail -f /var/log/auth.log
```

6. Count failed-password attempts by user

```
sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-5)}' | sort | uniq -c  
| sort -nr
```

7. Check specifically for failed root login attempts

```
sudo grep "root" /var/log/auth.log | grep "Failed"
```

8. Monitor currently logged-in users

```
w
```

9. Display detailed information about logged-in users

```
who -a
```

10. View the last 20 login records

```
last -n 20
```

```
(kali@olivil)-[~]
```

```
$ sudo last -b
```

```
[sudo] password for kali:
```

```
last: invalid option -- 'b'
```

```
Usage: wtmpdb [command] [options]
```

```
Commands: last, boot, boottime, rotate, shutdown, import
```

```
Options for last:
```

```
-a, --hostlast      Display hostnames as last entry
-d, --dns           Translate IP addresses into a hostname
-f, --file FILE     Use FILE as wtmpdb database
-F, --fulltimes     Display full times and dates
-i, --ip           Translate hostnames to IP addresses
-j, --json          Generate JSON output
-n, --limit N, -N   Display only first N entries
-p, --present TIME  Display who was present at TIME
-R, --nohostname   Don't display hostname
-S, --service       Display PAM service used to login
-s, --since TIME    Display who was logged in after TIME
-t, --until TIME    Display who was logged in until TIME
-w, --fullnames     Display full IP addresses and user and domain names
-x, --system        Display system shutdown entries
--time-format FORMAT Display timestamps in the specified FORMAT:
                    notime|short|full|iso
[username ...]      Display only entries matching these arguments
[tty ...]           Display only entries matching these arguments
TIME must be in the format "YYYY-MM-DD HH:MM:SS"
```

```
Options for boot (writes boot entry to wtmpdb):
```

```
-f, --file FILE     Use FILE as wtmpdb database
```

kali@olivil: ~

(kali@olivil)-[~]

\$ sudo journalctl _SYSTEMD_UNIT=ssh.service | grep -i "failed"~

Journal file /var/log/journal/b69758c0cad3481e967dcad827001d56/system@000658b9f79bae7f-6fae008802c6d6f3.journal~ is truncated, ignoring file.

(kali@olivil)-[~]

\$ sudo journalctl | grep -Ei "failed password|authentication failure|invalid user"

Journal file /var/log/journal/b69758c0cad3481e967dcad827001d56/system@000658b9f79bae7f-6fae008802c6d6f3.journal~ is truncated, ignoring file.

Jul 09 16:18:28 kali lightdm[1496]: pam_unix(lightdm:auth): authentication failure; logname= uid=0 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=kali

Jul 10 05:40:29 kali xfce4-screensaver-dialog[220887]: pam_unix(xfce4-screensaver:auth): authentication failure; logname=kali uid=1000 euid=1000 tty=:0.0 ruser= rhost= user=kali

Jul 10 05:45:36 kali xfce4-screensaver-dialog[223354]: pam_unix(xfce4-screensaver:auth): authentication failure; logname=kali uid=1000 euid=1000 tty=:0.0 ruser= rhost= user=kali

Jul 10 06:33:15 kali sudo[2539]: pam_unix(sudo:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=kali

Jul 14 01:05:25 kali sudo[68881]: pam_unix(sudo:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=kali

Jul 14 05:10:01 kali xfce4-screensaver-dialog[70290]: pam_unix(xfce4-screensaver:auth): authentication failure; logname=kali uid=1000 euid=1000 tty=:0.0 ruser= rhost= user=kali

Jul 17 23:50:29 kali xfce4-screensaver-dialog[25802]: pam_unix(xfce4-screensaver:auth): authentication failure; logname=kali uid=1000 euid=1000 tty=:0.0 ruser= rhost= user=kali

Aug 07 13:03:09 kali lightdm[1443]: pam_unix(lightdm:auth): authentication failure; logname= uid=0 euid=0 tty=:0 ruser= rhost= user=olivil

Aug 07 13:35:14 olivil su[11504]: pam_unix(su-l:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/2 ruser=kali rhost= user=john

Aug 09 10:28:37 olivil sudo[3043]: pam_unix(sudo:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=kali

Aug 09 11:10:17 olivil su[24296]: pam_unix(su-l:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/2 ruser=kali rhost= user=olivil

Aug 09 11:17:09 olivil su[27769]: pam_unix(su-l:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=olivil

Aug 09 11:17:34 olivil su[28057]: pam_unix(su-l:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=olivil

Aug 10 17:54:40 olivil su[29979]: pam_unix(su-l:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=mary

Aug 10 17:59:28 olivil su[31366]: pam_unix(su-l:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=mary

Aug 10 18:58:24 olivil lightdm[1455]: pam_unix(lightdm:auth): authentication failure; logname= uid=0 euid=0 tty=:0 ruser= rhost= user=kali

Aug 10 19:01:16 olivil lightdm[1457]: pam_unix(lightdm:auth): authentication failure; logname= uid=0 euid=0 tty=:0 ruser= rhost= user=kali

Aug 10 19:04:54 olivil lightdm[1453]: pam_unix(lightdm:auth): authentication failure; logname= uid=0 euid=0 tty=:0 ruser= rhost= user=kali

Aug 10 19:05:03 olivil lightdm[1458]: pam_unix(lightdm:auth): authentication failure; logname= uid=0 euid=0 tty=:0 ruser= rhost= user=kali

Question 3.1: What patterns might indicate a brute-force attack attempt?

- Many failed login attempts within a short period.
- Repeated attempts against the same username.
- Attempts against multiple usernames from the same IP address.
- Login attempts occurring continuously or at regular short intervals.
- Attempts coming from an unknown or suspicious IP address.
- Repeated attempts against privileged accounts such as root.
- Numerous authentication failures followed by a successful login, which could indicate that the attacker eventually guessed the password.

Question 3.2: Why is it important to monitor root login attempts?

Monitoring root login attempts is important because an attacker who successfully gains access to root can potentially:

- Modify or delete system files.
- Create or modify user accounts.
- Install malicious software.
- Disable security controls.
- Access sensitive information.
- Establish persistence.
- Take complete control of the system.

Exercise 3.2: Implement Account Lockout Policies

The prescribed `pam_tally2` method was not implemented because `pam_tally2` is unavailable on the current Kali Linux installation. The system provides `pam_faillock` instead. To avoid modifying the system-wide PAM authentication configuration and potentially disrupting existing authentication, this exercise was skipped.

Question 3.3: What are the trade-offs between security and usability when implementing account lockout policies?

Account lockout improves security by stopping repeated password-guessing attempts. However, it can inconvenience legitimate users who enter the wrong password and may allow attackers to deliberately lock accounts. A reasonable limit and lockout time helps balance security and usability.

Part 4: Sudo Access Management

Exercise 4.1: Configure Fine-Grained Sudo Permissions

Commands used:

1. Check the test user

```
id locktest
```

2. Check existing sudo permissions

```
sudo -l -U locktest
```

3. Back up the sudoers file

```
sudo cp /etc/sudoers /etc/sudoers.backup
```

4. Verify the backup

```
sudo ls -l /etc/sudoers /etc/sudoers.backup
```

5. Check Apache installation

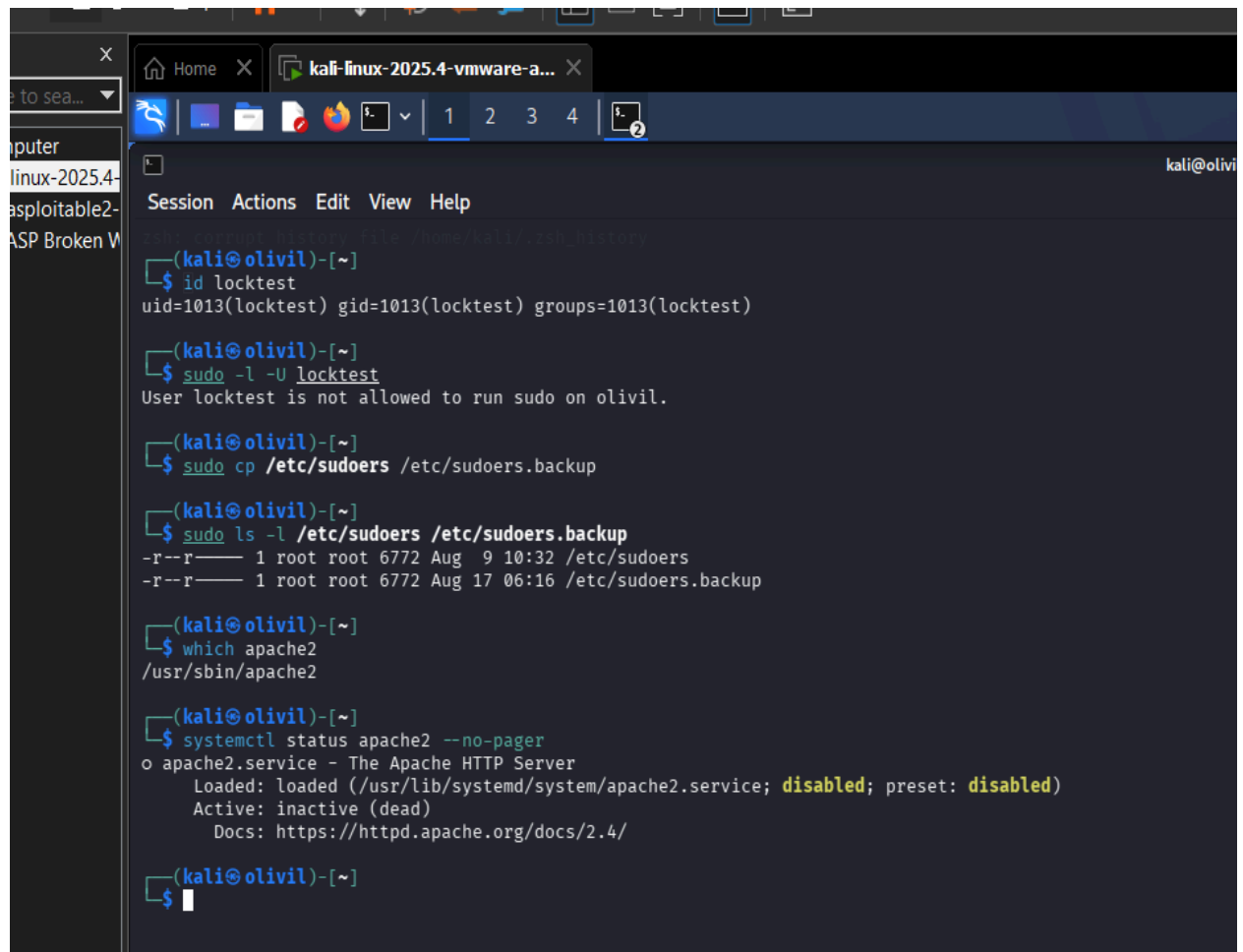
which apache2

6. Check Apache status

```
systemctl status apache2 --no-pager
```

7. Safely edit sudoers

```
sudo visudo
```



```
kali@olivil:~$ id locktest
uid=1013(locktest) gid=1013(locktest) groups=1013(locktest)

kali@olivil:~$ sudo -l -U locktest
User locktest is not allowed to run sudo on olivil.

kali@olivil:~$ sudo cp /etc/sudoers /etc/sudoers.backup

kali@olivil:~$ sudo ls -l /etc/sudoers /etc/sudoers.backup
-r--r--r-- 1 root root 6772 Aug  9 10:32 /etc/sudoers
-r--r--r-- 1 root root 6772 Aug 17 06:16 /etc/sudoers.backup

kali@olivil:~$ which apache2
/usr/sbin/apache2

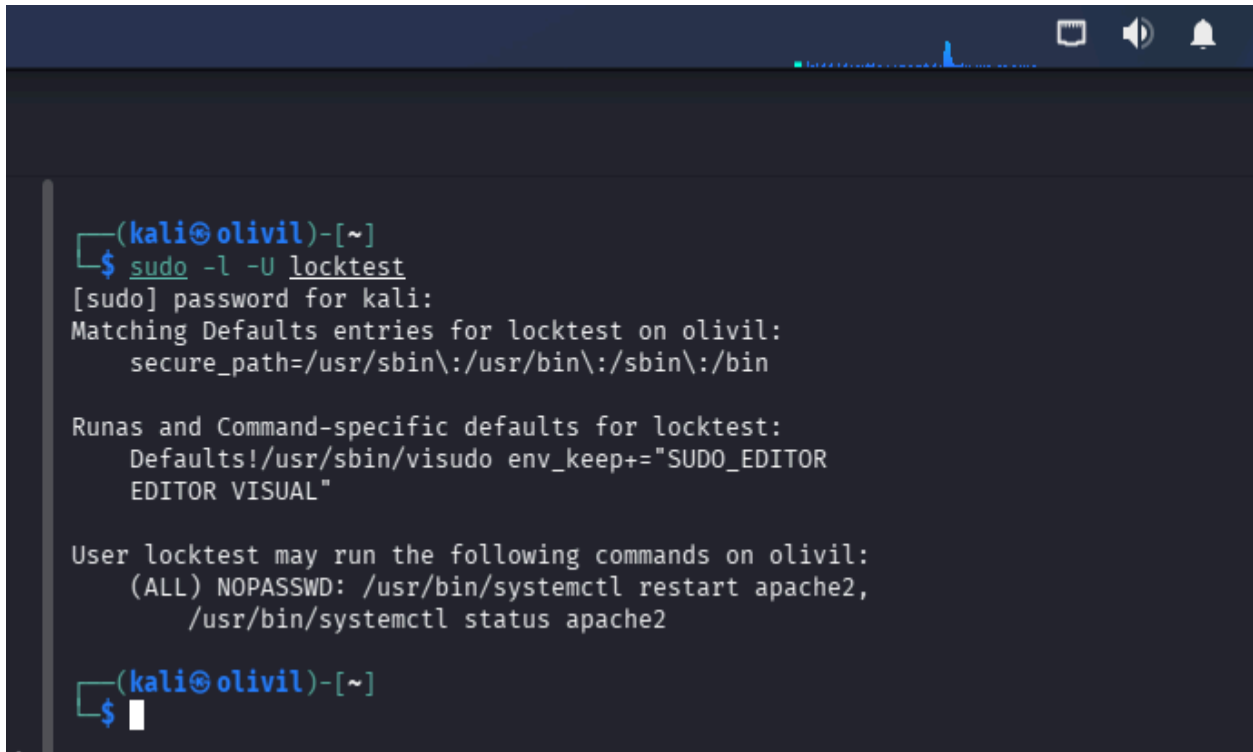
kali@olivil:~$ systemctl status apache2 --no-pager
o apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: https://httpd.apache.org/docs/2.4/

kali@olivil:~$
```

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ sudo visudo  
[sudo] password for kali:  
  
(kali@olivil)-[~]  
$  
  
kali@olivil: ~  
  
GNU nano 9.1 /etc/sudoers.tmp *  
## Some package scripts run a huge number of commands, which is made  
## slower by these options and also can clutter up the logs.  
# Defaults!PKGMAN !intercept, !log_subcmds  
##  
## Uncomment to disable PAM silent mode. Otherwise messages by PAM  
## modules such as pam_faillock will not be printed.  
# Defaults !pam_silent  
##  
## Uncomment for sudo to require the password of the target user to  
## allow changing users. By default, with this commented, sudo asks for  
## the password of the calling user.  
## Before uncommenting this, make sure that all your local configuration  
## that is intended to escalate privileges with the credentials of the  
## calling user have an explicit "Defaults:<entity> !targetpw" line.  
# Defaults targetpw  
##  
## Runas alias specification  
##  
##  
## User privilege specification  
##  
root ALL=(ALL:ALL) ALL  
  
# Allow members of group sudo to execute any command.  
# This is going to become optional at some point in the future, so please  
# consider adding a file in /etc/sudoers.d/zz_group_sudo containing this if  
# you want the sudo group to work. See also Bug#1136732.  
Defaults:%wheel !targetpw  
%sudo ALL=(ALL:ALL) ALL  
  
## Read drop-in files from /etc/sudoers.d. Consider setting  
## !targetpw in your snippets, so that Debian might change the  
## default at some time in the future.  
## See sudoers(5) for more information on "@include" directives:  
@includedir /etc/sudoers.d  
  
nary ALL=(ALL) NOPASSWD: /usr/bin/apt update, /usr/bin/apt upgrade  
  
paul ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart mysql  
  
locktest ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart apache2, /usr/bin/systemctl status apache2  
  
G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set M  
X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-6 Copy
```

8. Verify the sudo rule

sudo -l -U locktest



```
(kali@olivil)-[~]
$ sudo -l -U locktest
[sudo] password for kali:
Matching Defaults entries for locktest on olivil:
    secure_path=/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for locktest:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR
EDITOR VISUAL"

User locktest may run the following commands on olivil:
    (ALL) NOPASSWD: /usr/bin/systemctl restart apache2,
                  /usr/bin/systemctl status apache2

(kali@olivil)-[~]
$
```

it shows:

(ALL) NOPASSWD: /usr/bin/systemctl restart apache2, /usr/bin/systemctl status apache2

9. Switch to locktest

su - locktest

10. Confirm you are actually locktest

whoami

11. Check the user's sudo permissions

sudo -l

12. Test Apache status command

sudo systemctl status apache2 --no-pager

13. Test Apache restart command

`sudo systemctl restart apache2`

14. Test SSH command

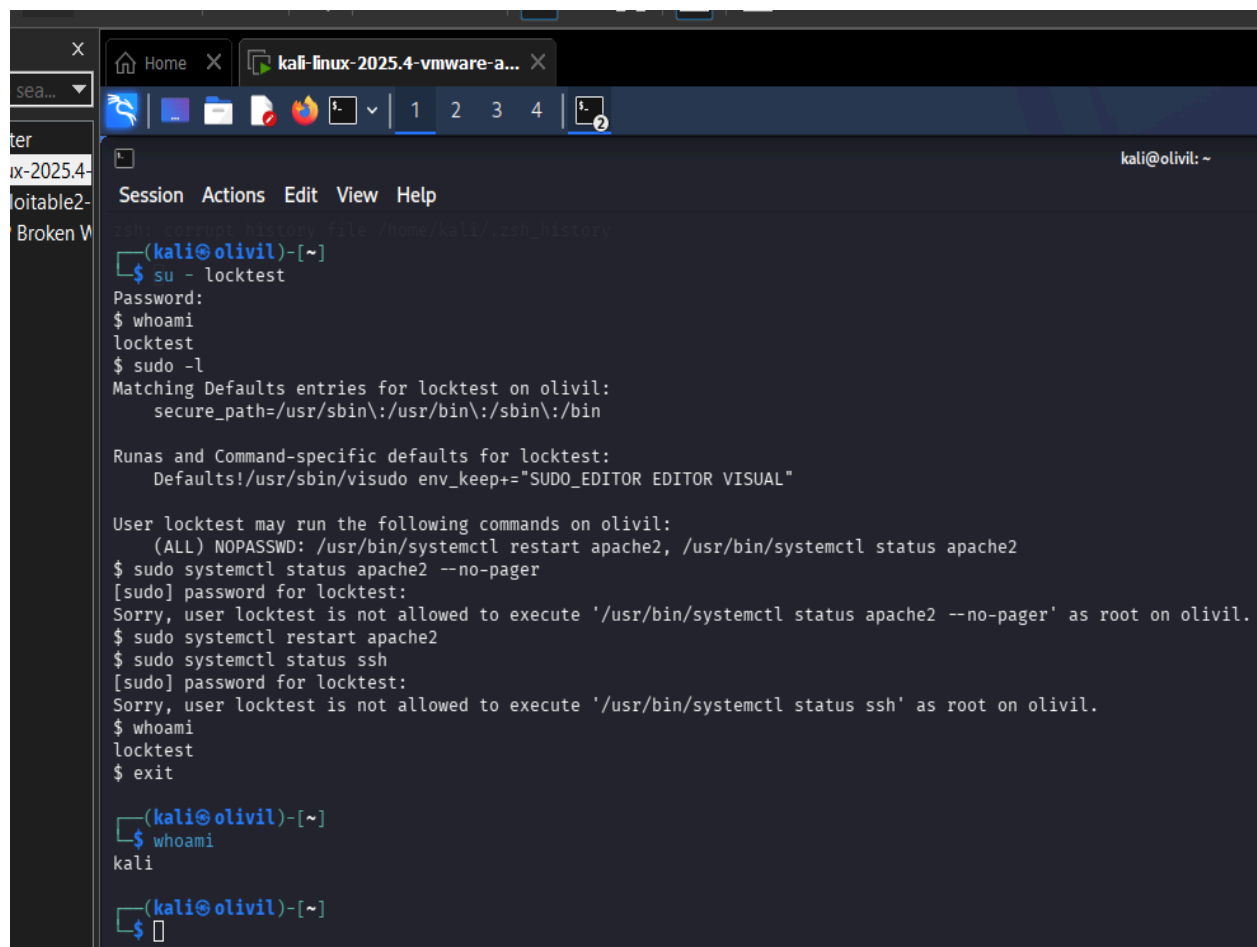
`sudo systemctl status ssh`

4. Returned to kali

`exit`

Then:

`whoami`



```
(kali@olivil)-[~]
$ su - locktest
Password:
$ whoami
locktest
$ sudo -l
Matching Defaults entries for locktest on olivil:
    secure_path=/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for locktest:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User locktest may run the following commands on olivil:
    (ALL) NOPASSWD: /usr/bin/systemctl restart apache2, /usr/bin/systemctl status apache2
$ sudo systemctl status apache2 --no-pager
[sudo] password for locktest:
Sorry, user locktest is not allowed to execute '/usr/bin/systemctl status apache2 --no-pager' as root on olivil.
$ sudo systemctl restart apache2
$ sudo systemctl status ssh
[sudo] password for locktest:
Sorry, user locktest is not allowed to execute '/usr/bin/systemctl status ssh' as root on olivil.
$ whoami
locktest
$ exit

(kali@olivil)-[~]
$ whoami
kali

(kali@olivil)-[~]
$
```

I configured fine-grained sudo permissions for the locktest account using visudo. The account was granted passwordless access to restart and check the

status of Apache, while unauthorized commands were restricted. I tested the permissions using the `locktest` account and verified the sudo configuration successfully. After completing the exercise, Apache was stopped and returned to its previous disabled state

Question 4.1: Why is it recommended to use `visudo` instead of directly editing `/etc/sudoers`?

`visudo` checks the `sudoers` file for syntax errors before saving, helping prevent mistakes that could disable sudo access or lock administrators out.

Question 4.2: What are the security implications of using `NOPASSWD` in sudo configurations?

`NOPASSWD` improves convenience but reduces security because commands can be run with elevated privileges without password verification. If an account is compromised, an attacker may immediately gain access to the allowed privileged commands.

Part 5: Security Compliance and Best Practices

Exercise 5.1: Perform a Security Audit Checklist

Created a script to automate security checks

Commands Used:

```
cd ~  
nano security_audit.sh
```

```
025.4-
ble2-
ken V

Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@olivil)-[~]
$ cd ~
(kali@olivil)-[~]
$ cd ~
nano security_audit.sh
```

Pasted into nano file:

```
#!/bin/bash
# User Access Security Audit Script

echo "=== User Access Security Audit ==="
echo ""

echo "1. Checking for users with UID 0:"
awk -F: '($3 == "0") {print $1}' /etc/passwd
echo ""

echo "2. Checking for accounts with empty passwords:"
sudo awk -F: '($2 == "") {print $1}' /etc/shadow
echo ""

echo "3. Checking for users with sudo access:"
getent group sudo
echo ""

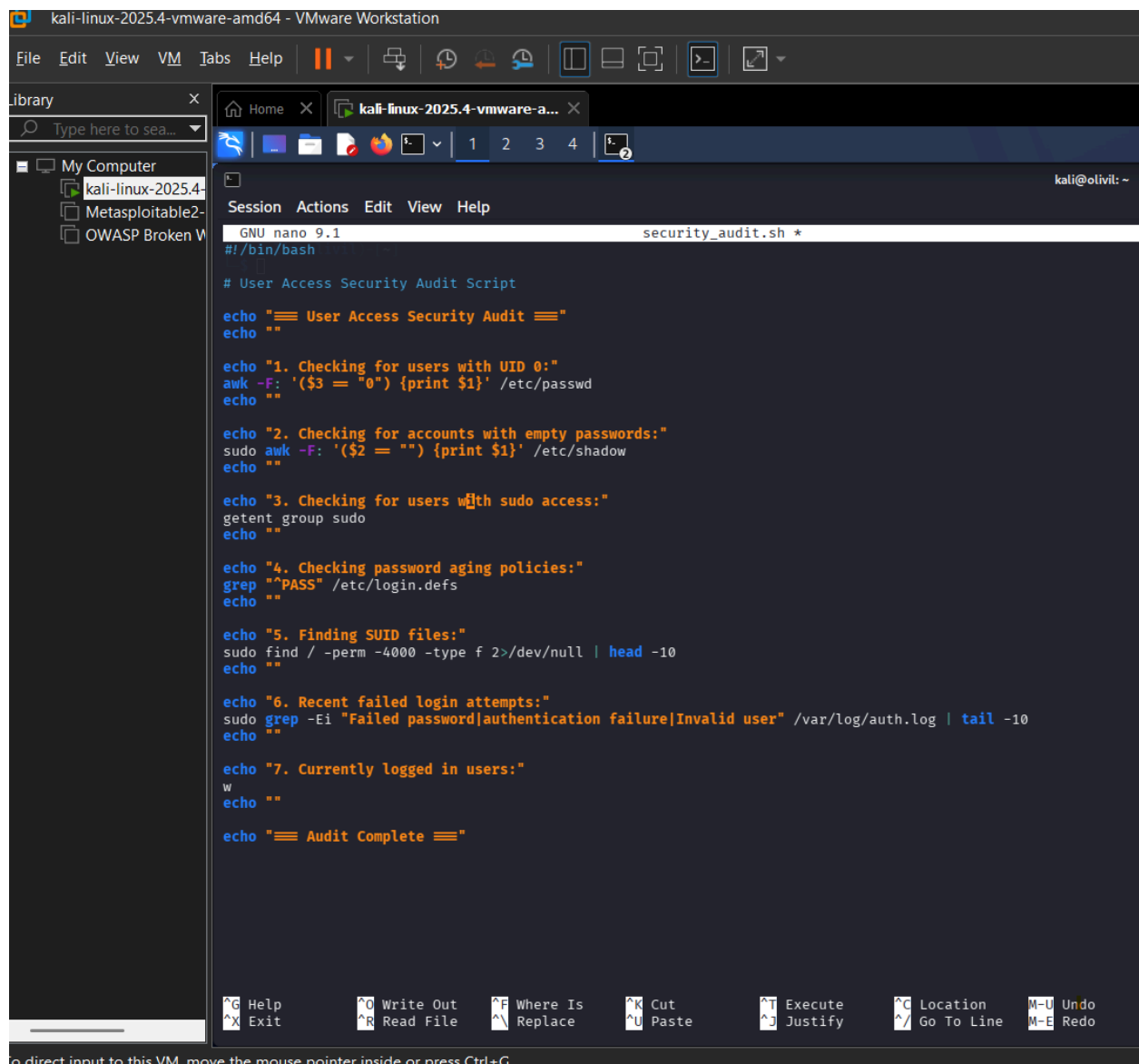
echo "4. Checking password aging policies:"
grep "^PASS" /etc/login.defs
echo ""

echo "5. Finding SUID files:"
sudo find / -perm -4000 -type f 2>/dev/null | head -10
echo ""
```

```
echo "6. Recent failed login attempts:"  
sudo lastb | head -10  
echo ""
```

```
echo "7. Currently logged in users:"  
w  
echo ""
```

```
echo "=== Audit Complete ==="
```



The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The terminal window displays the execution of a script named `security_audit.sh`. The script performs several security checks, including checking for users with UID 0, accounts with empty passwords, users with sudo access, password aging policies, SUID files, failed login attempts, and currently logged-in users. The script concludes with a message indicating the audit is complete.

```
kali-linux-2025.4-vmware-amd64 - VMware Workstation  
File Edit View VM Tabs Help  
Library  
Type here to search  
My Computer  
kali-linux-2025.4  
Metasploitable2  
OWASP Broken V  
kali@olivil: ~  
Session Actions Edit View Help  
GNU nano 9.1 security_audit.sh *  
#!/bin/bash  
  
# User Access Security Audit Script  
  
echo "=== User Access Security Audit ==="  
echo ""  
  
echo "1. Checking for users with UID 0:"  
awk -F: '($3 == "0") {print $1}' /etc/passwd  
echo ""  
  
echo "2. Checking for accounts with empty passwords:"  
sudo awk -F: '($2 == "") {print $1}' /etc/shadow  
echo ""  
  
echo "3. Checking for users with sudo access:"  
getent group sudo  
echo ""  
  
echo "4. Checking password aging policies:"  
grep "^PASS" /etc/login.defs  
echo ""  
  
echo "5. Finding SUID files:"  
sudo find / -perm -4000 -type f 2>/dev/null | head -10  
echo ""  
  
echo "6. Recent failed login attempts:"  
sudo grep -Ei "Failed password|authentication failure|Invalid user" /var/log/auth.log | tail -10  
echo ""  
  
echo "7. Currently logged in users:"  
w  
echo ""  
  
echo "=== Audit Complete ==="  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo  
to direct input to this VM, move the mouse pointer inside or press Ctrl+G
```

Saved the script = Ctrl O, Exit = X

Made it executable

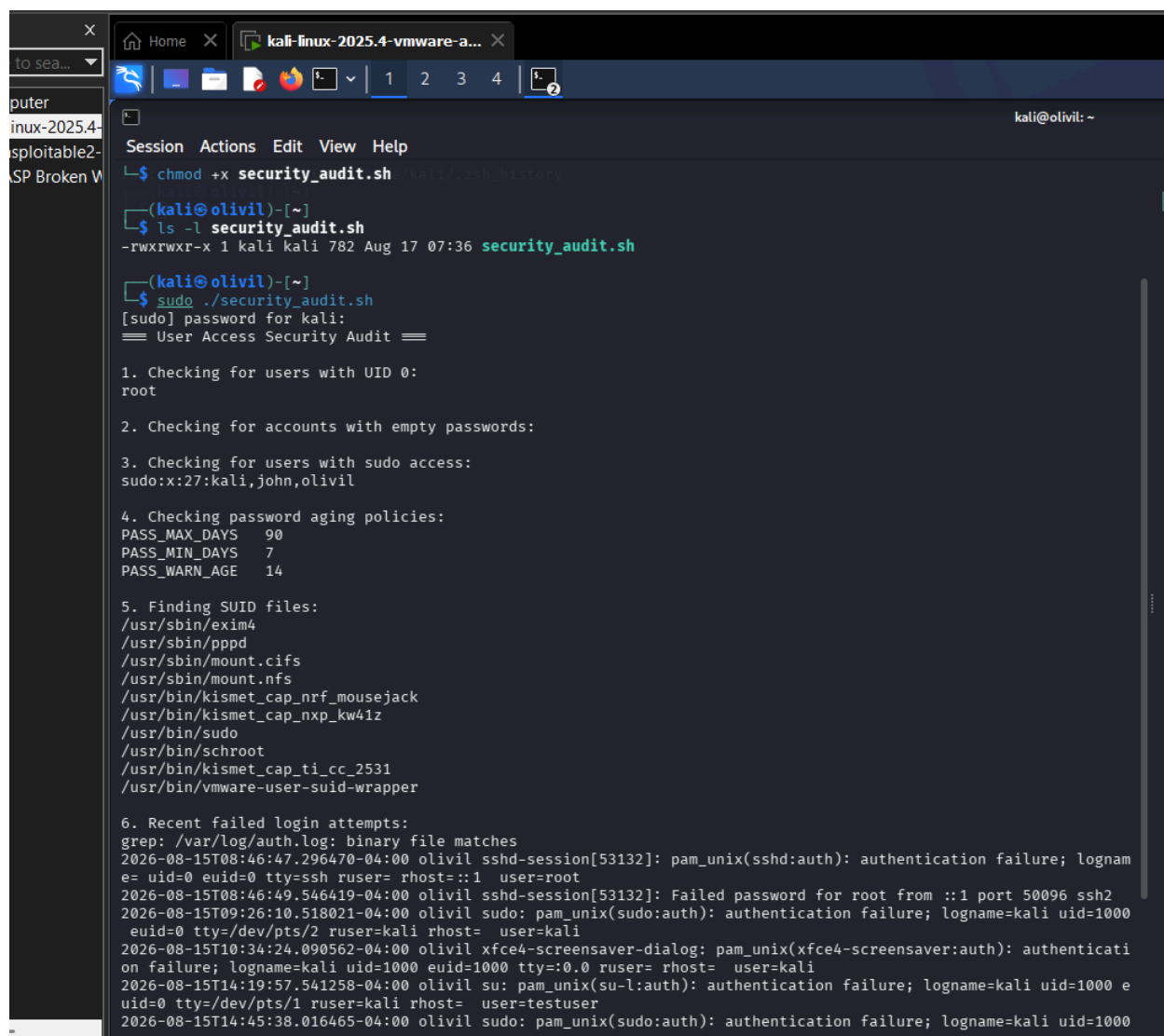
```
chmod +x security_audit.sh
```

Verified:

```
ls -l security_audit.sh
```

Ran the Audit File:

```
sudo ./security_audit.sh
```

A screenshot of a terminal window titled 'kali@olivil: ~'. The terminal shows the execution of a security audit script. The user first runs 'chmod +x security_audit.sh' and then 'sudo ./security_audit.sh'. The script outputs a series of checks: 1. Checking for users with UID 0: root. 2. Checking for accounts with empty passwords. 3. Checking for users with sudo access: sudo:x:27:kali,john,olivil. 4. Checking password aging policies: PASS_MAX_DAYS 90, PASS_MIN_DAYS 7, PASS_WARN_AGE 14. 5. Finding SUID files: /usr/sbin/exim4, /usr/sbin/pppd, /usr/sbin/mount.cifs, /usr/sbin/mount.nfs, /usr/bin/kismet_cap_nrf_mousejack, /usr/bin/kismet_cap_nxp_kw41z, /usr/bin/sudo, /usr/bin/schroot, /usr/bin/kismet_cap_ti_cc_2531, /usr/bin/vmware-user-suid-wrapper. 6. Recent failed login attempts: grep: /var/log/auth.log: binary file matches. The output shows several failed login attempts for root and kali users via SSH and sudo. The terminal window has a dark background with light-colored text. The top of the window shows a browser-like interface with tabs and a search bar.

created the automated security check bash script and it ran successfully

Question 5.1: What additional checks would you add to this security audit script?

I would add checks for inactive accounts, expired passwords, failed login attempts by IP address, unusual user accounts, SSH configuration, unauthorized cron jobs, and recently modified system files.

Exercise 5.2: Implement User Access Review Process

1. Create a user access report:

Commands Used:

```
#!/bin/bash
```

```
# Generate user access report
```

```
echo "User Access Report - $(date)" > user_access_report.txt
```

```
echo "===== " >>
```

```
user_access_report.txt
```

```
echo "" >> user_access_report.txt
```

```
echo "All User Accounts:" >> user_access_report.txt
```

```
awk -F: '{print $1 " (UID: " $3 ")"}' /etc/passwd >> user_access_report.txt
```

```
echo "" >> user_access_report.txt
```

```
echo "Users with Shell Access:" >> user_access_report.txt
```

```
grep -v '/nologin\|/false' /etc/passwd | awk -F: '{print $1}' >>
```

```
user_access_report.txt
```

```
echo "" >> user_access_report.txt
```

```
echo "Last Login Information:" >> user_access_report.txt
```

```
sudo lastlog >> user_access_report.txt
```

```
cat user_access_report.txt
```

The screenshot shows a Kali Linux terminal window with a dark theme. The window title is "kali-linux-2025.4-vmware-a...". The terminal displays a script named "user_access_report.sh" being executed in nano 9.1. The script generates a report in "user_access_report.txt" with the following content:

```
GNU nano 9.1 user_access_report.sh *
#!/bin/bash

# Generate user access report

echo "User Access Report - $(date)" > user_access_report.txt
echo "===== " >> user_access_report.txt
echo "" >> user_access_report.txt

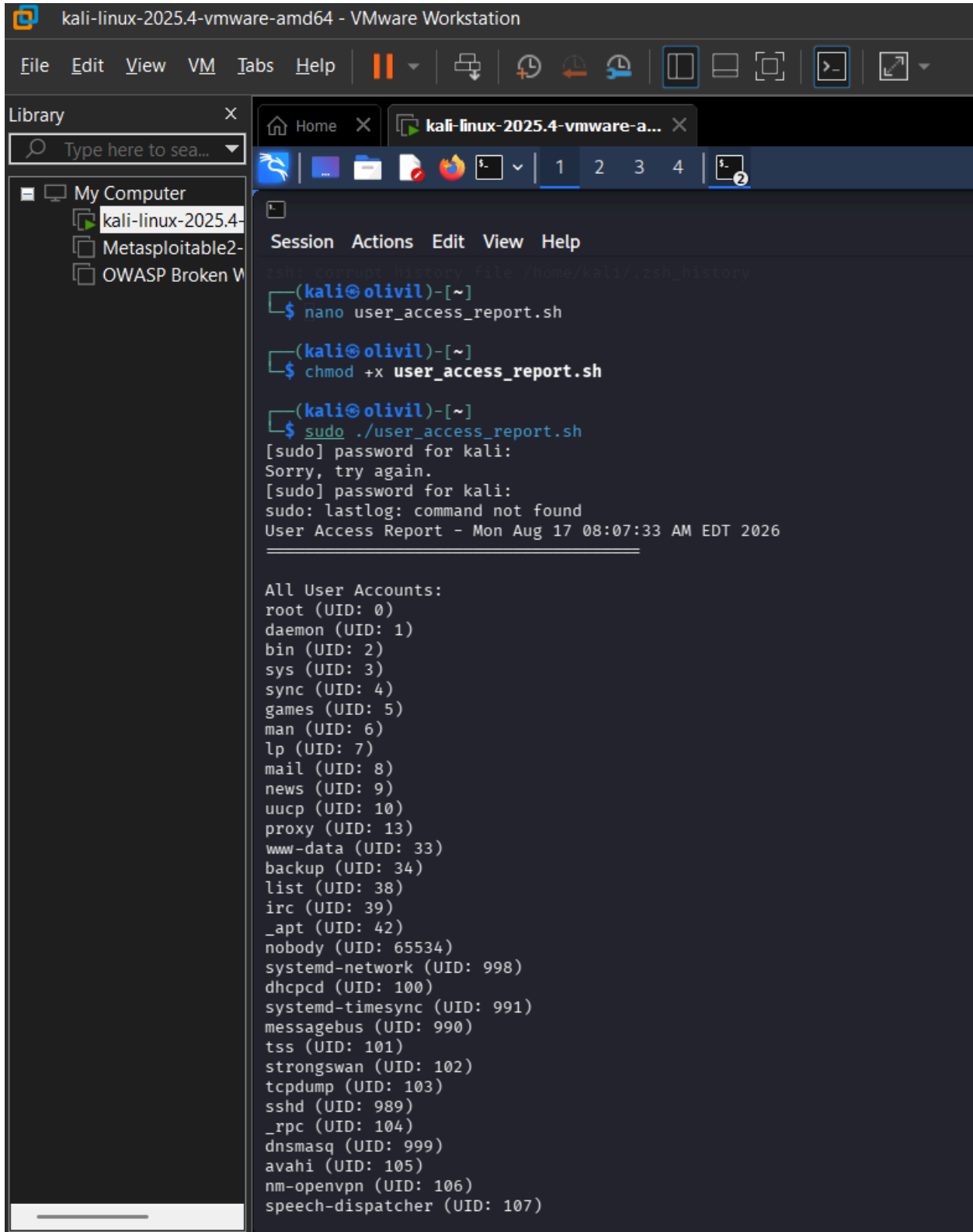
echo "All User Accounts:" >> user_access_report.txt
awk -F: '{print $1 " (UID: " $3 ")"}' /etc/passwd >> user_access_report.txt
echo "" >> user_access_report.txt

echo "Users with Shell Access:" >> user_access_report.txt
grep -v '/nologin\\|/false' /etc/passwd | awk -F: '{print $1}' >> user_access_report.txt
echo "" >> user_access_report.txt

echo "Last Login Information:" >> user_access_report.txt
sudo lastlog >> user_access_report.txt

cat user_access_report.txt
```

The terminal output shows the script is running successfully, and the final command "cat user_access_report.txt" is being executed, displaying the generated report content.



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

2. Document findings and recommendations:
Create a file security_recommendations.txt with your findings.

```
kali@olivil: ~  
GNU nano 9.1 security_recommendations.txt *  
Security Recommendations  
1. Review user accounts regularly and remove unnecessary accounts.  
2. Disable inactive or unused accounts.  
3. Enforce strong password and password-aging policies.  
4. Review sudo privileges and apply least-privilege access.  
5. Monitor failed login attempts for suspicious activity.  
6. Review SUID files and remove unnecessary SUID permissions.  
7. Monitor privileged accounts and root access.
```

```
kali@olivil: ~  
(kali@olivil)-[~]  
$ nano security_recommendations.txt  
(kali@olivil)-[~]  
$ cat security_recommendations.txt  
Security Recommendations  
1. Review user accounts regularly and remove unnecessary accounts.  
2. Disable inactive or unused accounts.  
3. Enforce strong password and password-aging policies.  
4. Review sudo privileges and apply least-privilege access.  
5. Monitor failed login attempts for suspicious activity.  
6. Review SUID files and remove unnecessary SUID permissions.  
7. Monitor privileged accounts and root access.  
(kali@olivil)-[~]  
$
```

Question 5.2: How often should user access reviews be conducted in a production environment?

User access reviews should be conducted at least quarterly, with additional reviews whenever employees change roles, leave the organization, or receive new privileges.